



Facultad de Ingeniería en
Ciencias de la Computación y Telecomunicaciones
U.A.G.R.M.


Somos ingeniería!



PARCIAL.- 2

APLICACIÓN WEB Y MÓVIL DISTRIBUIDA PARA LA ORQUESTACIÓN DE WORKFLOWS Y AUTOMATIZACIÓN DE TRÁMITES INSTITUCIONALES A PARTIR DE MODELADO VISUAL BPM, ASISTENCIA CONVERSACIONAL INTEGRADA

Estudiante: Flores Quispe Nathaly Madeline

Docente: M.Sc. Martínez Canedo Rolando Antonio

Asignatura: Ingeniería De Software 1– INF412 SA

Santa Cruz - Bolivia

Tabla de contenido

Perfil	4
Introducción	4
Objetivo	5
Objetivo General	5
Objetivos Específicos	5
Descripción del Problema.....	7
Alcance	8
Requisitos Funcionales y No Funcionales	11
Requisitos Funcionales (RF)	11
Requisitos No Funcionales (RNF).....	12
PARTE I	14
1.1 Fundamentación Teórica	14
1.1.1 Ingeniería de Software Asistida por computadora (C.A.S.E).....	14
1.1.2 Desarrollo de Software basado en componentes.....	14
1.1.3 Arquitectura de Software	16
1.1.4 Evolucion y Mantenimiento del Software	17
1.1.4.1 Mantenimiento Correctivo	17
1.1.4.2 Mantenimiento Adaptativo.....	18
1.1.4.3 Mantenimiento Preventivo.....	18
1.1.4.4 Mantenimiento Perfectivo.....	18
1.1.5 Ingenieria Humana (Experiencia del Usuario - UX).....	19
1.1.6 Inteligencia Artificial Aplicada en el desarrollo del Software	20
1.1.7 El proceso de Desarrollo de Software (PUDS)	22
1.1.8 Lenguaje de Modelado Unificado (UML 2.5)	23
1.1.9. Desarrollo Móvil Multiplataforma (Flutter)	25
1.1.10. Modelo C4.....	25
1.1.11 Sistemas de Gestión Documental Electrónica (EDMS).....	26
1.1.12 Computación e Infraestructura en la Nube (Amazon Web Services - AWS)	26
1.1.13 Redes Neuronales y Aprendizaje Profundo (Deep Learning / NLP)	27
Parte II – Proceso de desarrollo	28
2.1 Flujo de Trabajo: Captura de Requisitos.....	28
2.1.1 Actores y casos de uso	28
2.1.1.1 Actores	28

2.1.1.2 Casos de Uso	29
2.1.1.3 Priorización de Casos de Uso	30
2.1.1.4 Detallar casos de uso.....	32
2.1.1.5 Estructura de modelo de caso de uso.....	54
2.2. Flujo de Trabajo: Análisis.....	56
2.2.1 Análisis de Arquitectura	56
2.2.1.1 Identificación de Paquetes	56
2.2.1.2 Relación de Paquetes y Casos de Uso.....	57
2.2.1.3 Vista de Paquetes	60
2.2.1.4 Análisis de Casos de Uso	62
2.2.1.4.1 Diagramas de Comunicación	62
2.2.1.4.2 Diagrama de Secuencia	66
2.2.1.5 Análisis de Paquetes.....	78
2.3 Flujo de Trabajo: Diseño	79
2.3.1 Diseño de arquitectura (Modelo C4)	79
3.3.1.1 Diagrama de Contexto	79
3.3.1.2 Diagrama de Contenedores.....	80
3.3.1.3 Diagrama de Componentes	80
2.3.2 Diseño físico (Modelo de Despliegue)	81
2.3.2.1 Diseño de Datos Logico.....	81
Diagrama de Clases	81
Mapeo	81
2.3.3.2 Diseño de Datos Fisico.....	83
Tabla de Volumen.....	83
2.4 Flujo de Trabajo: Implementación	91
2.4.1 Implementación de la arquitectura del Sistema	91
2.5 Flujo de Trabajo: Pruebas.....	92
Parte III	100
Bibliografía	100
Anexos.....	101
Estándares de codificación utilizados.....	101
Repositorio:.....	102
Web:.....	103
Manual de Usuario:.....	103

Perfil

Introducción

En el actual entorno de gestión organizacional, donde la agilidad operativa, la transparencia de los procesos y la automatización son esenciales, las herramientas tradicionales para el seguimiento de trámites resultan poco eficientes ante las nuevas exigencias de empresas de servicios como CRE, COTAS o entidades bancarias. La gestión de políticas de negocio, especialmente en flujos que involucran múltiples departamentos técnicos, legales y administrativos, demanda un control dinámico que evite tareas repetitivas de supervisión manual y reduzca el tiempo significativo perdido en la localización de expedientes o estados de solicitudes. Asimismo, la alta dependencia de documentación física genera una acumulación masiva de archivos ineficiente, requiriendo la evolución hacia repositorios digitales centralizados, seguros y de alta velocidad en la nube.

Simultáneamente, los diagramas de actividades UML han demostrado ser elementos fundamentales durante la etapa de diseño de procesos, pero rara vez se aprovechan directamente para ejecutar y controlar el flujo de trabajo funcional. Con la necesidad de una coordinación departamental distribuida, se hace imperativo contar con entornos colaborativos en tiempo real que permitan a los funcionarios operar mediante monitores proactivos, editar simultáneamente expedientes digitales y rediseñar flujos de forma conjunta. A la par, el acceso a estos sistemas debe garantizar resiliencia absoluta, permitiendo que las aplicaciones móviles operen en modo offline y las plataformas web actúen como Aplicaciones Web Progresivas (PWA) ante fallas de conectividad a internet.

En este contexto, el desarrollo de una aplicación web y móvil colaborativa que integre la automatización de workflows basados en UML, gestión documental en infraestructura cloud (AWS S3) e Inteligencia Artificial avanzada (Deep Learning y NLP) representa una solución innovadora y necesaria. Esta plataforma eliminará los puntos ciegos de atención al cliente mediante un agente inteligente de admisión por voz y prompts, guiará el enrutamiento predictivo para mitigar riesgos de cuellos de botella y permitirá a los administradores generar reportes dinámicos interactivos a través de lenguaje natural, optimizando drásticamente la productividad institucional y la experiencia del usuario final.

Objetivo

Objetivo General

Desarrollar una aplicación web y móvil colaborativa y resiliente (PWA y modo offline) que permita diseñar políticas de negocio mediante diagramas de actividades UML, gestionar de forma automática el seguimiento de trámites en tiempo real, y administrar expedientes mediante un sistema de gestión documental centralizado en la nube (AWS S3); integrando un motor de Inteligencia Artificial basado en Deep Learning y NLP para la admisión automatizada de usuarios, el enrutamiento predictivo de flujos con análisis de riesgo, y la generación dinámica de reportes interactivos, facilitando la operatividad de los funcionarios y la transparencia institucional.

Objetivos Específicos

- Implementar un sistema de autenticación y gestión de roles basado en control de acceso restrictivo (RBAC) para diferenciar las capacidades de los clientes, administradores y funcionarios operativos según su departamento correspondiente.
- Desarrollar un motor de flujos (Workflow Engine) en Spring Boot que permita crear, cargar, guardar y ejecutar de manera dinámica estructuras de procesos secuenciales, alternativas (con compuertas de decisión), iterativas y paralelas.
- Desarrollar una herramienta gráfica colaborativa en tiempo real basada en WebSockets que permita el diseño visual de las políticas de negocio utilizando la notación de diagramas de actividades UML de calle.
- Integrar un Agente Inteligente de Admisión basado en NLP y Deep Learning en el microservicio de Python, capaz de procesar prompts de texto o comandos de voz del cliente final para clasificar su caso y asignarle automáticamente la política de negocio correspondiente de manera digital.
- Diseñar e implementar un Repositorio de Gestión Documental Digital centralizado en la nube mediante AWS S3, que soporte el almacenamiento de múltiples formatos (Word, Excel, PDF, imágenes, videos) estructurado de manera independiente por política y cliente, bajo un control estricto de privilegios departamentales de lectura y modificación.
- Implementar un motor de formularios dinámicos y parametrizables que genere automáticamente interfaces específicas de carga de datos para cada área o departamento involucrado en el nodo activo de la política de negocio.
- Desarrollar un monitor de actividades proactivo web mediante Angular transformándolo en una Aplicación Web Progresiva (PWA), que organice las tareas operativas utilizando un sistema de semáforos visuales (Rojo para trámites nuevos, Amarillo en proceso y Verde para completados) con actualización en tiempo real mediante WebSockets.
- Habilitar el funcionamiento 100% offline en la aplicación móvil desarrollada en Flutter, permitiendo que los funcionarios registren tareas y los clientes realicen el seguimiento externo de la ubicación de su trámite mediante el escaneo de códigos QR y la descarga final de documentos en PDF sin depender críticamente de una conexión activa a internet.

- Construir un Motor Inteligente de Enrutamiento y Análisis de Riesgo en Python mediante Deep Learning, que analice el historial de transiciones para predecir la mejor ruta de los trámites, anticipar demoras, detectar anomalías de comportamiento y priorizar automáticamente la asignación de recursos.
- Desarrollar un generador dinámico de reportes interactivos por lenguaje natural (Text-to-MongoQuery) en Python, que permita al jefe de políticas solicitar informes específicos mediante texto o audio (definiendo datos, criterios y formatos de salida en Word, Excel o PDF) e interactúe proactivamente en caso de detectar ambigüedades.
- Garantizar la consistencia, persistencia y alta disponibilidad de la arquitectura desacoplada mediante el uso de bases de datos NoSQL (MongoDB), orquestación de microservicios e infraestructura cloud escalable.

Descripción del Problema

Actualmente, gestionar políticas de negocio y trámites complejos en empresas de servicios implica una dependencia crítica de procesos manuales o sistemas estáticos, lo que representa una barrera para la agilidad organizacional. Los equipos de trabajo pierden tiempo valioso en la comunicación interna y en el seguimiento manual de formularios que deben transitar por múltiples departamentos como técnico, legal o almacén. Esta falta de automatización genera "puntos ciegos" donde ni el funcionario ni el cliente conocen con certeza el estado o la ubicación exacta de un trámite, provocando retrasos significativos. A esto se suma que los trámites generan una inmensa cantidad de documentación física o archivos dispersos; almacenarlos en sistemas de archivos locales o servidores tradicionales resulta costoso, sumamente lento en velocidad de respuesta y propenso a pérdidas, al carecer de un repositorio documental centralizado, eficiente y con control estricto de privilegios.

Por otro lado, existe una desconexión crítica entre el diseño estratégico (diagramas de actividades UML) y la operación diaria del funcionario, quien a menudo debe lidiar con monitores pasivos que requieren actualizaciones constantes (F5) para visualizar nuevas tareas. Esta desconexión retrasa los tiempos de atención, genera errores en el enrutamiento de la información y dificulta la detección oportuna de cuellos de botella que saturan la capacidad operativa. Además, los sistemas convencionales obligan a la existencia de ventanillas físicas de atención al cliente para la recepción de solicitudes y validación de requisitos; esto satura las oficinas, genera colas innecesarias y delega la asignación de flujos al criterio falible del personal de recepción, en lugar de contar con un mecanismo inteligente de admisión automatizada.

Asimismo, los supervisores y jefes de políticas de negocio carecen de herramientas analíticas predictivas capaces de anticipar demoras, evaluar riesgos de estancamiento o priorizar dinámicamente los recursos institucionales antes de que ocurra el colapso de un departamento. Esta rigidez operativa se extiende a la toma de decisiones estratégicas, donde la obtención de reportes de gestión depende de formatos estáticos predefinidos; los gerentes no pueden solicitar consultas analíticas personalizadas sobre la marcha, viéndose obligados a depender de personal técnico para bases de datos, lo que ralentiza drásticamente la capacidad de reacción empresarial ante anomalías.

La falta de una solución integral que permita diseñar políticas interactivas, centralizar la gestión documental en la nube (AWS S3) bajo entornos resilientes web/móvil (offline/PWA) y explotar la Inteligencia Artificial mediante modelos de Deep Learning y NLP, representa una limitación crítica en entornos competitivos. Resolver este problema estructural permitirá eliminar los puntos de atención físicos mediante agentes inteligentes de admisión por voz y prompts, automatizar el enrutamiento y análisis predictivo de riesgo de cuellos de botella, agilizar el llenado de formularios mediante el procesamiento de informes y democratizar la analítica gerencial a través de la generación dinámica de reportes por lenguaje natural, transformando radicalmente la productividad institucional y la experiencia del usuario final.

Alcance

- **Módulo de Gestión de Usuarios y Roles (RBAC)**
Permite registrar, iniciar y cerrar sesión de manera segura. Administra el acceso restrictivo diferenciando entre Clientes, Funcionarios (asociados a departamentos específicos) y Administradores de políticas, garantizando que cada usuario acceda únicamente a las funciones, flujos y formularios que le competen por área.
- **Módulo de Gestión de Políticas de Negocio**
Permite crear, guardar, cargar y eliminar políticas de negocio dinámicas. Los administradores pueden parametrizar estructuras de flujo secuenciales, alternativas, iterativas y paralelas para modelar cualquier trámite organizacional, definiendo además las etiquetas de contexto del proceso y los requisitos de admisión obligatorios u opcionales asociados.
- **Módulo de Edición de Workflows (Editor Visual)**
Incluye un canvas interactivo basado en la notación de diagramas de actividades UML de calle (*swimlanes*). Permite organizar actividades, asignar departamentos responsables y definir la lógica de tránsito del trámite de forma intuitiva.
- **Módulo de Diseño Asistido por IA (Prompts)**
Permite la creación y edición de diagramas mediante comandos de texto o voz. La inteligencia artificial, alojada en el microservicio de Python, interpreta los prompts del administrador para dibujar nodos, conexiones y transiciones automáticamente.
- **Módulo de Edición Colaborativa en Tiempo Real**
Permite que múltiples administradores trabajen en el diseño de una misma política de negocio de manera simultánea. El sistema sincroniza los cambios de forma continua mediante WebSockets, resolviendo conflictos concurrentes de datos en tiempo real.
- **Módulo de Monitor de Actividades Proactivo (Web PWA)**
Proporciona a los funcionarios un panel visual web implementado en Angular y transformado en una Aplicación Web Progresiva (PWA) para garantizar resiliencia ante cortes de red. Las tareas se organizan mediante un sistema de semáforos interactivos: Rojo (Recién Llegado/Pendiente), Amarillo (En proceso) y Verde (Ejecutados/Atendidos). Este módulo elimina las actualizaciones manuales (\$F5\$), refrescando la información instantáneamente a través de WebSockets.
- **Módulo de Generación Dinámica de Formularios**
A partir de la política diseñada, el sistema renderiza automáticamente interfaces de entrada de datos específicas para cada área o departamento involucrado en el nodo activo del trámite. Estos formularios permiten a los funcionarios registrar su labor y derivar el expediente al siguiente nivel de manera automática.
- **Módulo de Procesamiento de Informes con IA**
Permite a los funcionarios cargar resúmenes operativos textuales o notas de voz sobre su labor. El microservicio de Python procesa la información en lenguaje natural mediante NLP para realizar el llenado automático de los campos en el formulario del sistema, agilizando la ejecución del trámite.

- **Módulo de Repositorio de Gestión Documental Digital (AWS S3)**
Implementa un sistema de gestión documental centralizado para digitalizar el 99% de los expedientes físicos. Soporta el almacenamiento de archivos en múltiples formatos (Word, Excel, PDF, imágenes y videos) utilizando la infraestructura de Amazon S3 para garantizar velocidad de respuesta, reducción de costos y alta disponibilidad. Los documentos se organizan de forma independiente estructurados por Política de Negocio y Cliente, manejando un registro de auditoría estricto (quién envió, cuándo y quién modificó) y asignación estricta de privilegios departamentales (Lectura, Escritura y Modificación) con soporte de edición colaborativa paralela.
- **Módulo de Agente Inteligente de Admisión (Deep Learning / NLP)**
Reemplaza la ventanilla física de atención al cliente mediante un agente inteligente virtual accesible por web y móvil. Este componente utiliza modelos de procesamiento de lenguaje natural (NLP) para interactuar con el usuario final a través de comandos de voz o prompts de texto, clasificando la problemática planteada por el cliente para determinar y poner en marcha de manera automatizada la política de negocio correcta que corresponda al trámite. Gestiona de manera interactiva la solicitud y carga de los requisitos definidos en la política.
- **Módulo Motor Inteligente de Enrutamiento y Análisis de Riesgo**
Evoluciona el motor de estados transaccional mediante el uso de Deep Learning. Este módulo analiza el histórico de transiciones para predecir la ruta óptima de cada trámite, calcular de forma predictiva el riesgo de demoras o estancamientos, detectar anomalías de comportamiento en el flujo operativo y priorizar de manera automatizada las asignaciones de recursos institucionales y trámites críticos.
- **Módulo Generador Dinámico de Reportes por Lenguaje Natural**
Proporciona al jefe de políticas un mecanismo interactivo para la obtención de métricas avanzadas y KPIs flexibles. A través de instrucciones de texto o audio procesadas con NLP, el administrador define los datos que requiere, los criterios o condiciones de filtrado y el formato de salida deseado (Word, Excel, PDF o Pantalla). El asistente de IA tiene la capacidad de interactuar proactivamente y repreguntar al usuario en caso de detectar ambigüedades en la instrucción antes de generar el reporte.
- **Módulo de Seguimiento Móvil Resiliente (Cliente / Modo Offline)**
Desarrollado en Flutter, este módulo está diseñado para operar al 100% en modo offline, permitiendo al cliente realizar el seguimiento de su trámite, visualizar requisitos y consultar estados sin depender críticamente de una conexión activa a internet. Habilita el escaneo de códigos QR para verificar la ubicación del expediente en tiempo real y, al completarse el ciclo del flujo, permite la descarga directa del documento de resolución final en formato PDF firmado digitalmente.
- **Módulo de Notificaciones y Sincronización (Asistente)**
Implementa un servicio de notificaciones proactivo que alerta instantáneamente a los involucrados en las plataformas web y móvil sobre el progreso de los formularios y

transiciones de estado, evitando que el flujo se detenga por falta de supervisión operativa.

- **Módulo de Seguridad y Persistencia Cloud**

Aplica cifrado de datos y control de acceso estricto, garantizando la persistencia de las políticas, documentos y trámites distribuidos en una base de datos NoSQL (MongoDB) alojada en la nube para asegurar alta disponibilidad. Asimismo, se consolida una arquitectura de microservicios donde el núcleo del negocio es gestionado en Spring Boot, delegando las tareas analíticas predictivas, el procesamiento de lenguaje natural y el enrutamiento inteligente de Deep Learning a un servidor independiente en Python, conectados mediante solicitudes HTTP REST de alta eficiencia.

Requisitos Funcionales y No Funcionales

Requisitos Funcionales (RF)

Son las funciones específicas que el sistema debe ejecutar obligatoriamente para satisfacer las necesidades de los usuarios.

- **RF-01: Control de Acceso Basado en Roles (RBAC):** El sistema debe autenticar a los usuarios de forma segura y restringir los accesos diferenciando entre Clientes, Administradores de políticas y Funcionarios (asociados estrictamente a un departamento operativo).
- **RF-02: Editor Visual de Políticas de Negocio (Workflows):** El sistema debe proveer un canvas interactivo basado en la notación de diagramas de actividades UML de calle (*swimlanes*) para modelar flujos organizacionales en tiempo real.
- **RF-03: Soporte de Variables Estructurales de Flujo:** El motor del workflow debe ser capaz de procesar y ejecutar estructuras lógicas complejas de tipo secuencial, alternativo (con compuertas de decisión), iterativo y paralelo (procesos simultáneos).
- **RF-04: Diseño Asistido por Prompts de IA (Voz/Texto):** El sistema debe permitir al administrador dictar o escribir instrucciones en lenguaje natural para que el microservicio de IA dibuje automáticamente nodos, relaciones y dependencias en el diagrama de actividades.
- **RF-05: Edición Colaborativa Simultánea de Workflows:** El entorno de diseño de políticas de negocio debe permitir que múltiples administradores editen simultáneamente un mismo diagrama, sincronizando los cambios en las pantallas concurrentes de forma inmediata.
- **RF-06: Generación Dinámica de Formularios por Área:** El software debe renderizar interfaces de captura de datos parametrizables y específicas para cada departamento según la actividad activa en la política de negocio.
- **RF-07: Monitor Proactivo de Actividades (Sistema de Semáforos):** El sistema debe mostrar a los funcionarios un panel automatizado que organice sus tareas mediante etiquetas de colores interactivos: Rojo (Pendientes/Recién llegó), Amarillo (En proceso) y Verde (Atendidas/Ejecutadas), actualizándose instantáneamente sin requerir refresco manual (\$F5\$).
- **RF-08: Procesamiento Inteligente de Informes Textuales:** El sistema debe permitir a los funcionarios cargar resúmenes operativos o notas de voz en lenguaje natural, extrayendo los datos clave para realizar el llenado automático de los campos del formulario activo.
- **RF-09: Repositorio Digital de Gestión Documental (AWS S3):** El sistema debe permitir la carga, almacenamiento y recuperación de archivos multidiseño (Word, Excel, PDF, imágenes, videos) asociados a un trámite, organizados de manera independiente estructurados por Política de Negocio y Cliente.
- **RF-10: Control de Privilegios y Edición Paralela de Documentos:** El repositorio documental debe restringir las acciones sobre cada archivo (Lectura, Escritura,

Modificación) de acuerdo al departamento del funcionario y permitir que múltiples usuarios autorizados editen simultáneamente archivos de texto o cálculo en flujos concurrentes.

- **RF-11: Agente Inteligente de Admisión Automatizada (NLP):** El sistema debe proveer una interfaz virtual interactiva para el cliente final (vía texto o comandos de voz) que analice su problemática en lenguaje natural, verifique el cumplimiento de requisitos de admisión opcionales u obligatorios, y ponga en marcha automáticamente el workflow correspondiente.
- **RF-12: Enrutamiento Inteligente y Análisis Predictivo de Riesgo:** El motor del sistema debe procesar las marcas de tiempo de las transiciones para predecir la ruta óptima de un trámite, anticipar el riesgo de cuellos de botella antes de que ocurran y detectar anomalías en el rendimiento operativo de los departamentos.
- **RF-13: Generador Dinámico de Reportes por Lenguaje Natural:** El sistema debe permitir al jefe de políticas solicitar reportes analíticos personalizados especificando mediante voz o texto tres parámetros obligatorios: Datos a extraer, Criterios de filtrado y Formato de salida (Word, Excel, PDF, Pantalla); debiendo repreguntar al usuario interactivamente en caso de ambigüedad.
- **RF-14: Seguimiento Externo Móvil Resiliente (Código QR y PDF):** El cliente debe poder escanear un código QR desde la aplicación móvil para visualizar el estado y ubicación de su trámite en tiempo real, e implementar la descarga directa del documento de resolución final en formato PDF.

Requisitos No Funcionales (RNF)

Son las propiedades, cualidades y restricciones técnicas que debe cumplir el sistema para garantizar su calidad.

- **RNF-01: Arquitectura de Microservicios Desacoplada:** El sistema debe estar compuesto por un backend transaccional de negocio implementado en Spring Boot y un microservicio independiente en Python enfocado exclusivamente en las tareas de Inteligencia Artificial (Deep Learning y NLP).
- **RNF-02: Persistencia Flexible NoSQL:** La gestión de datos estructurados dinámicos (workflows de diagramas de actividades y formularios dinámicos por área) debe ejecutarse sobre una base de datos documental MongoDB para asegurar flexibilidad de esquemas libres.
- **RNF-03: Almacenamiento en la Nube Escalable y de Bajo Costo (AWS S3):** Toda la documentación digital generada a lo largo del ciclo de vida del trámite debe ser almacenada de forma externa en buckets de Amazon S3 para maximizar la velocidad de respuesta y mitigar los costos de almacenamiento local de archivos pesados.
- **RNF-04: Sincronización Concurrente de Alta Velocidad (Tiempo Real):** La actualización de los semáforos en los monitores proactivos y la edición colaborativa simultánea (tanto de diagramas como de documentos) deben gestionarse mediante

WebSockets para asegurar tiempos de respuesta inferiores a 1 segundo entre los terminales conectados.

- **RNF-05: Resiliencia de Red y Multiplataforma:** La interfaz web administrativa y operativa debe implementarse en Angular bajo el estándar de **Aplicación Web Progresiva (PWA)** para tolerar caídas de red, mientras que la interfaz móvil del cliente debe desarrollarse en Flutter con soporte completo de funcionamiento **100% offline** (almacenamiento local temporal y sincronización automática al recuperar la conectividad).
- **RNF-06: Portabilidad y Contenedorización:** Los servicios del backend (Spring Boot y Python FastAPI) deben estar empaquetados en contenedores independientes de Docker para simplificar su despliegue y alta disponibilidad en entornos cloud.
- **RNF-07: Trazabilidad y Seguridad de Datos (Auditoría):** Todas las transiciones de trámites y modificaciones de documentos en el repositorio deben registrarse estrictamente en una bitácora inmutable (marcas de tiempo, identificador de usuario y tipo de acción), aplicando cifrado en tránsito y en reposo sobre la información sensible corporativa.

PARTE I

1.1 Fundamentación Teórica

1.1.1 Ingeniería de Software Asistida por computadora (C.A.S.E)

La Ingeniería de Software Asistida por Computadora (CASE, por sus siglas en inglés Computer-Aided Software Engineering) se refiere al conjunto de herramientas y métodos que apoyan el desarrollo de software mediante la automatización de actividades en las distintas fases del ciclo de vida del software. Estas herramientas buscan mejorar la productividad, calidad y mantenimiento de los sistemas desarrollados, reduciendo errores humanos y facilitando la gestión de proyectos complejos (Pressman, 2010).

El concepto de CASE surge como respuesta a la creciente necesidad de manejar proyectos de software de manera más eficiente, estandarizada y controlada. Las herramientas CASE permiten automatizar tareas como el análisis, diseño, codificación, pruebas y documentación, proporcionando entornos integrados que apoyan a los desarrolladores en cada etapa (Sommerville, 2011).

Existen diferentes tipos de herramientas CASE, clasificadas generalmente en tres categorías:

- Herramientas CASE superiores (Upper CASE): Enfocadas en las fases iniciales como análisis y diseño.
- Herramientas CASE inferiores (Lower CASE): Orientadas a la generación de código, pruebas y mantenimiento.
- Herramientas CASE integradas (I-CASE): Combinan funcionalidades de ambos tipos, abarcando todo el ciclo de vida del software (Pressman, 2010).

El uso de estas herramientas proporciona beneficios significativos, como la estandarización de procesos, mejora en la trazabilidad de requisitos, aumento en la reutilización de componentes y reducción en los tiempos de desarrollo. Sin embargo, también implica desafíos, como la curva de aprendizaje, el costo de implementación y la necesidad de adaptación a metodologías específicas (Sommerville, 2011).

1.1.2 Desarrollo de Software basado en componentes

El Desarrollo de Software Basado en Componentes (Component-Based Software Development, CBSD) es un paradigma que promueve la construcción de sistemas mediante la integración de componentes previamente desarrollados y probados, en lugar de desarrollar cada funcionalidad desde cero. Este enfoque se basa en el principio de reutilización, donde los componentes son unidades modulares, independientes y encapsuladas que ofrecen servicios a través de interfaces bien definidas (Szyperski, 2002).

El CBSD surge como una respuesta a los desafíos del desarrollo tradicional, buscando mejorar la eficiencia, reducir costos y acortar los tiempos de entrega mediante la reutilización sistemática de software. En este modelo, un componente es considerado como una unidad funcional autónoma que puede ser desplegada e integrada dentro de diferentes aplicaciones, siempre que se respeten las interfaces establecidas (Brown & Wallnau, 1998).

- **Características del Desarrollo Basado en Componentes** Entre las principales características del CBSD se destacan en este proyecto:

- **Reutilización:** La capacidad de integrar componentes existentes en nuevos sistemas. En esta plataforma, se manifiesta en el uso de librerías y componentes visuales preestablecidos para el lienzo del editor de diagramas UML, disminuyendo drásticamente el esfuerzo de desarrollo y diseño desde cero.
 - **Modularidad:** Los componentes son diseñados para ser independientes. El sistema divide de forma estricta sus responsabilidades en componentes de backend (servicios independientes en Spring Boot para lógica de estado, notificaciones y métricas) y componentes de frontend (vistas aisladas en Angular y Flutter), facilitando el mantenimiento y la escalabilidad del software de manera aislada (Szyperski, 2002).
 - **Interoperabilidad:** Los componentes pueden comunicarse entre sí mediante estándares definidos. En este proyecto, el backend (Spring Boot), la web (Angular) y la aplicación móvil (Flutter) logran una comunicación fluida e interoperable mediante el intercambio de datos estructurados en formato JSON a través de una API REST y protocolos WebSocket, independientemente de los lenguajes nativos de cada plataforma.
 - **Encapsulamiento:** Los detalles internos de cada componente están ocultos. Tanto la lógica de persistencia en MongoDB como los estados internos de procesamiento del motor de *workflows* exponen únicamente interfaces públicas bien definidas (endpoints controlados), protegiendo la integridad de las reglas del negocio.
- **Ventajas del CBSD** El uso del desarrollo basado en componentes proporciona múltiples beneficios en el ecosistema del proyecto. La reducción del tiempo de desarrollo permitió ensamblar las bandejas de entrada, los formularios reactivos dinámicos y los paneles semafóricos a partir de la reutilización de elementos preexistentes y validados. Además, se mejora la calidad general del software, dado que frameworks arquitectónicos como Spring Boot, Angular y Flutter basan su ciclo de vida en componentes altamente probados a nivel global, lo que simplifica el mantenimiento y la actualización futura sin afectar el comportamiento del motor central del workflow (Heineman & Councill, 2001).
 - **Desafíos y Consideraciones** A pesar de sus ventajas, el CBSD también presenta ciertos desafíos que se gestionaron rigurosamente en el diseño del sistema. La correcta selección de módulos externos (como las librerías gráficas para el canvas interactivo de UML) fue crucial para asegurar los requisitos funcionales y de rendimiento. Asimismo, fue necesario controlar estrictamente aspectos de compatibilidad, seguridad y dependencias heterogéneas entre los paquetes administrados para garantizar una integración limpia en producción (Crnkovic et al., 2011).
 - **CBSD en el Contexto del Proyecto** En la arquitectura del sistema, el CBSD se consolidó como una estrategia clave para enfrentar la demanda de una solución rápida y altamente flexible. El backend saca provecho de la inyección de dependencias y beans modulares de **Spring Boot**; el frontend web utiliza la arquitectura orientada a

componentes y módulos reactivos de **Angular** para reutilizar los elementos de los formularios interactivos; y la aplicación móvil se estructuró sobre el árbol de widgets altamente combinables de **Flutter**. Toda la gestión de estas piezas funcionales y de dependencias externas de software se administró de manera automatizada a través de gestores de componentes a gran escala como **Maven** (para Java) y **npm** (para JavaScript).

1.1.3 Arquitectura de Software

La Arquitectura de Software es una disciplina fundamental dentro de la ingeniería de software que se encarga de definir la estructura general de un sistema, estableciendo cómo se organizan sus componentes, cómo interactúan entre sí y qué principios guían su diseño y evolución. Según Bass, Clements y Kazman (2013), la arquitectura de software representa "la estructura o estructuras de un sistema, conformadas por componentes de software, las propiedades visibles externamente de esos componentes y las relaciones entre ellos".

Una buena arquitectura proporciona una base sólida sobre la cual se puede desarrollar, mantener y escalar un sistema de software de manera eficiente. Actúa como un plano maestro que guía a los desarrolladores, asegurando que las decisiones técnicas estén alineadas con los objetivos funcionales y no funcionales del proyecto, como el rendimiento, la seguridad, la escalabilidad, la mantenibilidad y la interoperabilidad (Rozanski & Woods, 2012).

- **Principios de la Arquitectura de Software** Los principales principios que rigen la arquitectura de software e impulsan este proyecto incluyen:
 - **Modularidad:** Dividir el sistema en componentes independientes. El proyecto se segmenta en tres módulos globales claramente delimitados en la raíz del espacio de trabajo: un backend centralizador, una interfaz web y una aplicación móvil.
 - **Abstracción:** Ocultar la complejidad interna. El motor de estados del workflow procesa internamente la lógica de grafos relacionales y las transiciones dinámicas del trámite, exponiendo hacia los frontends únicamente interfaces de servicios limpias (APIs) para interactuar con los datos.
 - **Separación de responsabilidades:** Asignar funciones específicas. Se delimita estrictamente la lógica de control visual y captura de formularios (delegada a Angular y Flutter) de la persistencia de datos, auditoría, el broker de WebSockets y las reglas de negocio (centralizadas en Spring Boot).
 - **Reutilización:** Diseñar componentes reutilizables. Se implementa en la construcción de los widgets de interfaz móvil, las plantillas gráficas del editor de actividades UML y el renderizador genérico de campos de formularios (Bass et al., 2013).
 - **Flexibilidad y escalabilidad:** Permitir la adaptación a cambios futuros. La elección de una base de datos NoSQL basada en documentos JSON flexibles dota a la arquitectura de la adaptabilidad necesaria para que cada política de

negocio evolucione o altere su estructura de datos sin corromper el motor central.

- **Estilos y Patrones de Arquitectura en el Proyecto** La arquitectura del sistema adopta un enfoque híbrido combinando estilos y patrones arquitectónicos probados para solucionar problemas recurrentes en el diseño de software (Rozanski & Woods, 2012):
 - **Estilo Cliente-Servidor Multicapa:** Se establece un modelo donde múltiples clientes concurrentes (la web en Angular y la aplicación móvil en Flutter) solicitan recursos y sincronización transaccional a un servidor centralizado (Spring Boot). Este backend implementa a su vez una arquitectura interna en capas: capa de controladores (Endpoints REST), capa de servicios (lógica de negocio y motor de workflow) y capa de acceso a datos (Persistencia).
 - **Patrón Modelo-Vista-Controlador (MVC):** Utilizado de forma nativa por los frameworks cliente para desacoplar de manera eficiente el estado de los trámites y formularios (Modelo), el renderizado semafórico de las bandejas de entrada (Vista), y los manejadores de eventos o captura de voz (Controlador).
 - **Patrón Repositorio:** Implementado en la capa de persistencia de Spring Boot para abstraer los mecanismos de consulta hacia MongoDB, permitiendo operaciones fluidas sobre el historial (*timeline*) y las colecciones del sistema.
- **Importancia de la Arquitectura de Software** Una arquitectura bien definida permite anticipar problemas potenciales, mejorar la toma de decisiones técnicas y facilita la comunicación entre los distintos actores del proyecto (desarrolladores, arquitectos, stakeholders). En el contexto de este sistema de gestión de políticas de negocio, el diseño arquitectónico inicial establece el marco técnico indispensable para garantizar requisitos no funcionales críticos, tales como la consistencia en el procesamiento de flujos paralelos (*Fork/Join*), la baja latencia en la edición colaborativa de diagramas mediante WebSockets y el rendimiento bajo carga incremental de trámites en ejecución (Bass et al., 2013).

1.1.4 Evolucion y Mantenimiento del Software

El mantenimiento del software es un elemento crítico dentro del ciclo de vida de los sistemas, abarcando todas las modificaciones, optimizaciones y correcciones realizadas sobre una aplicación tras su puesta en marcha o despliegue inicial. Según Sommerville (2011), el software no es un producto estático, sino un ente dinámico que debe evolucionar de forma continua para seguir siendo útil y operativo dentro de una organización.

1.1.4.1 Mantenimiento Correctivo

El mantenimiento correctivo se refiere a las acciones tomadas para solucionar errores, defectos o fallos detectados en el software después de su puesta en producción. Estos errores pueden ser de tipo funcional, de rendimiento o de seguridad, y suelen ser descubiertos por los usuarios finales o durante auditorías técnicas (Sommerville, 2011). En el contexto de este proyecto, el mantenimiento correctivo se activa de forma reactiva

ante fallas críticas en el motor de estados del *workflow*, tales como interrupciones en el canal de comunicación por WebSockets al editar diagramas, errores de persistencia al mapear documentos JSON corruptos en MongoDB, o bloqueos inesperados en las colas de procesamiento que impidan cambiar el color del semáforo visual en las bandejas de entrada de los funcionarios.

1.1.4.2 Mantenimiento Adaptativo

El mantenimiento adaptativo consiste en modificar el software para que pueda operar correctamente en un entorno cambiante. Esto incluye la adaptación a nuevas plataformas, sistemas operativos, cambios en la normativa legal, políticas de seguridad, o actualizaciones de hardware y software con los que el sistema interactúa (Pressman, 2010). Para este sistema de automatización, este mantenimiento asegura la compatibilidad operativa frente a cambios externos e institucionales, tales como la alteración en las reglas de las políticas de negocio de empresas locales, modificaciones en las estructuras de los formularios requeridos por ley, actualizaciones en los motores de ejecución de navegadores web (afectando a Angular), o nuevas API de control del sistema operativo Android que obliguen a realizar ajustes de compatibilidad en el entorno de Flutter.

1.1.4.3 Mantenimiento Preventivo

El mantenimiento preventivo tiene un enfoque proactivo y busca anticiparse a posibles fallos futuros, mejorando aspectos internos del software que podrían derivar en problemas a largo plazo. Esto incluye la refactorización de código, optimización de procesos, eliminación de redundancias y mejoras en la documentación (Sommerville, 2011). En la plataforma desarrollada, las actividades preventivas se centran en la reestructuración y optimización de los índices de consulta en MongoDB para evitar futuras ralentizaciones en el historial de trámites, la limpieza de código redundante en los gestores de estado reactivos de las interfaces, y el robustecimiento de las validaciones de datos antes de que el flujo pase de un carril (*swimlane*) a otro, reduciendo sustancialmente los costos de soporte técnico futuro.

1.1.4.4 Mantenimiento Perfectivo

El mantenimiento perfectivo está orientado a la mejora continua del software, no necesariamente porque existan fallos, sino para optimizar su rendimiento, agregar nuevas funcionalidades o mejorar la experiencia del usuario (IEEE, 2010). En el alcance de este primer ciclo, este mantenimiento se traduce en acciones estratégicas encaminadas a pulir el diseño de la interfaz de usuario, optimizar la velocidad de renderizado del editor gráfico de actividades UML, minimizar la latencia en las actualizaciones de las bandejas de entrada, o expandir las capacidades del panel de control para que los administradores visualicen estadísticas del rendimiento de los funcionarios en tiempo real, garantizando la competitividad técnica del producto de software.

1.1.5 Ingeniería Humana (Experiencia del Usuario - UX)

La Ingeniería Humana, en el contexto del desarrollo de software, se refiere al estudio y aplicación de principios que buscan optimizar la interacción entre los seres humanos y los sistemas computacionales. Esta disciplina se centra en diseñar sistemas que sean intuitivos, eficientes y seguros para los usuarios, considerando sus capacidades, limitaciones y expectativas. Dentro de esta área, uno de los conceptos más relevantes es la Experiencia del Usuario (UX, por sus siglas en inglés), la cual juega un papel fundamental en la aceptación y éxito de cualquier producto software (Norman, 2013).

La Experiencia del Usuario (UX) se define como el conjunto de percepciones y respuestas que una persona experimenta al interactuar con un sistema, producto o servicio. No solo abarca aspectos funcionales, sino también emocionales, estéticos y de usabilidad. Según la ISO 9241-210 (2010), UX es "la percepción y respuesta de una persona que resulta del uso o anticipación del uso de un producto, sistema o servicio".

- **Principios de la Ingeniería Humana en UX Aplicados al Proyecto** La Ingeniería Humana aplicada a la experiencia del usuario busca garantizar que los sistemas sean diseñados de manera centrada en el usuario (User-Centered Design), lo cual se ve reflejado en las dos interfaces del sistema (Angular y Flutter) considerando los siguientes aspectos:
 - **Usabilidad:** Facilidad con la que el usuario aprende a manejar el sistema. En el entorno web (Angular), se traduce en un editor gráfico altamente intuitivo que permite arrastrar y soltar componentes para modelar diagramas de actividades UML sin requerir conocimientos técnicos avanzados de programación.
 - **Accesibilidad:** Capacidad del sistema para ser utilizado por diferentes perfiles. Tanto el portal web como la aplicación móvil estructuran sus elementos visuales y formularios dinámicos con contrastes adecuados y jerarquías claras, facilitando que cualquier funcionario de la organización pueda operar sus tareas asignadas sin barreras cognitivas.
 - **Eficiencia:** Minimizar el esfuerzo necesario para completar tareas. Se optimiza el flujo de trabajo mediante un diseño de bandejas de entrada que elimina clics innecesarios. El motor de *workflows* pre-carga los formularios reactivos y datos esenciales del trámite de manera automática según el carril o departamento correspondiente.
 - **Satisfacción:** Generar una experiencia agradable. El uso de transiciones fluidas en la aplicación móvil y una interfaz limpia libre de saturación de texto genera un entorno de trabajo digital cómodo y moderno para el personal operativo.
 - **Consistencia:** Mantener patrones de diseño coherentes. Se implementa una misma línea gráfica, tipográfica y de comportamiento operativo entre la aplicación de escritorio/web y la aplicación móvil, asegurando que un botón, un estado o un formulario se interpreten exactamente de la misma manera en cualquier pantalla (Krug, 2014).

- **Importancia de la Experiencia del Usuario en el Sistema** La UX se ha convertido en un factor determinante en el éxito de los sistemas modernos. Un software de gestión de procesos técnicamente robusto puede fracasar si los funcionarios no logran entender su flujo. En esta plataforma, problemas comunes de los sistemas administrativos tradicionales —como interfaces confusas o flujos de aprobación complicados— se resuelven mediante la incorporación de un código de colores semáforo (rojo, amarillo y verde). Esta alerta visual reduce drásticamente la carga cognitiva de los trabajadores, permitiéndoles identificar instantáneamente la prioridad de sus tareas pendientes, mitigando errores operativos y evitando el abandono del sistema (Garrett, 2011).
- **Metodologías y Herramientas en UX Utilizadas en el Primer Parcial** El diseño de experiencias efectivas para este sistema de políticas de negocio implicó el uso de metodologías de ingeniería humana clave durante el Ciclo 1 (Norman, 2013):
 - **Design Thinking:** Proceso iterativo que permitió empatizar con las necesidades reales de los departamentos organizacionales, definiendo con precisión cómo estructurar el panel de control y las bandejas para agilizar la interacción diaria del funcionario.
 - **Prototipado:** Creación de maquetas visuales previas en herramientas de diseño para evaluar el comportamiento del lienzo interactivo y la distribución de las calles o carriles (*swimlanes*) de responsabilidad antes de iniciar su codificación en Angular.
 - **Pruebas de Usabilidad:** Evaluaciones prácticas preliminares donde se analizó la velocidad y facilidad con la que un usuario simulado podía completar un formulario dinámico o reasignar un trámite, identificando oportunidades críticas para simplificar la interfaz.
 - **Personas y Escenarios:** Representación de los perfiles típicos de usuario que interactúan con la plataforma —desde el Administrador que modela las políticas en la web, hasta el Funcionario de campo que procesa las actividades en la aplicación móvil—, guiando con precisión cada decisión de diseño visual y de navegación del software.

1.1.6 Inteligencia Artificial Aplicada en el desarrollo del Software

La Inteligencia Artificial (IA) ha revolucionado múltiples industrias, y el desarrollo de sistemas empresariales no es la excepción. La IA aplicada en los sistemas de gestión se refiere al uso de algoritmos, técnicas de aprendizaje automático (*Machine Learning*), procesamiento de lenguaje natural (PLN) y reconocimiento de voz para automatizar, optimizar y mejorar la interacción del usuario con el sistema y agilizar la ejecución del ciclo de vida de los procesos de negocio (Russell & Norvig, 2020).

La incorporación de IA en las plataformas de flujos de trabajo (*workflows*) permite desde la edición guiada de diagramas hasta la asistencia interactiva en el llenado de información compleja. Esta sinergia entre IA y los sistemas de software institucionales no solo incrementa

la productividad de los funcionarios, sino que también abre la puerta a nuevas formas de construir entornos operativos más inteligentes, adaptativos y eficientes (Jordan & Mitchell, 2015).

- **Aplicaciones de la IA en la Plataforma de Políticas de Negocio** En el diseño y ejecución de este sistema, las ramas de la Inteligencia Artificial se aplican de manera específica en los siguientes componentes del software:
 - **Interacción y Edición de Diagramas mediante Comandos de Voz:** En lugar de depender exclusivamente de periféricos tradicionales, el sistema integra servicios de IA conversacional y reconocimiento de voz en el editor gráfico de Angular. Esto permite que el administrador interactúe con el lienzo UML y realice modificaciones estructurales en la política de negocio utilizando instrucciones verbales directas, las cuales son interpretadas y ejecutadas por el software en tiempo real.
 - **Procesamiento de Lenguaje Natural (PLN) en Formularios:** El sistema saca provecho de técnicas de PLN para facilitar el llenado fiable de los formularios dinámicos por parte de los funcionarios. Los algoritmos asisten en la interpretación de descripciones extensas o entradas de texto libre, automatizando la estructuración de los datos requeridos en cada carril o nodo de atención del trámite administrativo.
 - **Asistencia Inteligente al Funcionario:** Mediante la integración de modelos conversacionales adaptados, la plataforma provee un mecanismo de asistencia interactiva que guía al usuario en la resolución de sus tareas pendientes (etiquetas rojas y amarillas), minimizando la tasa de error en la introducción de datos y optimizando los flujos de respuesta institucionales.
- **Beneficios de la IA en el Sistema de Gestión de Procesos** La implementación de servicios cognitivos dentro del motor de *workflows* proporciona múltiples ventajas operativas directas (Russell & Norvig, 2020):
 - **Automatización de tareas repetitivas de captura de datos,** permitiendo que los funcionarios agilicen el procesamiento de los trámites y se enfoquen en la toma de decisiones estratégicas de su departamento.
 - **Reducción drástica de errores humanos en el modelado y ejecución de las políticas,** gracias a la supervisión continua, validación de campos e interfaces guiadas inteligentemente.
 - **Aceleración del tiempo de atención de cada trámite,** disminuyendo el rozamiento en la interfaz de usuario mediante comandos de voz y asistencia automatizada.
 - **Mejora continua en la experiencia de usuario (UX),** ya que los componentes de asistencia conversacional facilitan que el personal de la organización

interactúe con el software de forma natural, intuitiva y con una curva de aprendizaje mínima.

1.1.7 El proceso de Desarrollo de Software (PUDS)

El Proceso Unificado de Desarrollo de Software (PUDS, o simplemente UP por sus siglas en inglés *Unified Process*) es un marco de trabajo de ingeniería de software extensible, adaptado a sistemas complejos, que se caracteriza por ser guiado por Casos de Uso, centrado en la arquitectura, e iterativo e incremental (Jacobson, Booch & Rumbaugh, 1999). Este proceso establece un marco sistemático y controlado para planificar, diseñar, construir y asegurar productos de software de alta calidad tecnológica (Pressman, 2010).

El propósito principal del PUDS es organizar y mitigar los riesgos del trabajo de desarrollo mediante la generación de versiones iterativas ejecutables. En el contexto de este proyecto, el enfoque iterativo permitió que al finalizar el Ciclo 1 se consolidara una versión operativa e incremental real —lista para producción— que resuelve el núcleo transaccional del negocio antes de evolucionar hacia capacidades cognitivas (Sommerville, 2011).

- **Fases Estructurales del Proceso Unificado (PUDS)**

A diferencia de los modelos lineales tradicionales, el PUDS organiza el esfuerzo de desarrollo en cuatro fases dinámicas consecutivas, dentro de las cuales se ejecutan disciplinas técnicas de forma iterativa (Jacobson et al., 1999):

- **Fase de Inicio (*Inception*):** Consiste en la definición del alcance del proyecto, identificación de los actores y delimitación de los primeros Casos de Uso del sistema de políticas de negocio. En esta etapa se establecieron las restricciones tecnológicas iniciales (Backend en Spring Boot, Web en Angular, Móvil en Flutter y base de datos en MongoDB).
- **Fase de Elaboración (*Elaboration*):** Se centra en la mitigación de los riesgos arquitectónicos mayores y la especificación detallada de los requisitos funcionales. Durante esta fase se diseñó el plano maestro de la arquitectura cliente-servidor multicapa y se validó la viabilidad técnica del motor de estados del *workflow* y la comunicación por WebSockets.
- **Fase de Construcción (*Construction*):** Es la fase donde se desarrolla el código fuente de forma intensiva y se ensamblan los componentes modulares. En el alcance de este primer ciclo, abarcó la programación del editor visual UML, el motor transaccional de estados, los formularios reactivos dinámicos y la lógica semafórica de control de tiempos para los funcionarios.
- **Fase de Transición (*Transition*):** Orientada a asegurar que el software esté listo para los usuarios finales. Implica el despliegue del sistema en el entorno de producción, la ejecución de pruebas de usabilidad y la validación de que el incremento del Ciclo 1 sea una versión iterativa completamente funcional y operativa en producción (Pressman, 2010).

- **Disciplinas Técnicas del PUDS Aplicadas**

Durante el desarrollo de la primera iteración ejecutable del sistema, se interactuó de manera concurrente sobre las disciplinas clave del Proceso Unificado (Sommerville, 2011):

- **Modelado del Negocio y Requisitos:** Captura de las necesidades operativas de las organizaciones mediante Casos de Uso estructurados para definir cómo deben interactuar los administradores y funcionarios con las políticas de negocio.
- **Análisis y Diseño:** Traducción de los requisitos en planos técnicos de software. Se modelaron los diagramas de clases, secuencias lógicas y estados para estructurar las colecciones de MongoDB y los servicios de Spring Boot.
- **Implementación y Pruebas:** Codificación limpia de los componentes de backend y frontend bajo el paradigma basado en componentes (CBSD) y validación continua mediante pruebas funcionales para garantizar la consistencia en el enrutamiento de los trámites y la edición colaborativa en tiempo real.

1.1.8 Lenguaje de Modelado Unificado (UML 2.5)

El Lenguaje de Modelado Unificado (UML, por sus siglas en inglés Unified Modeling Language) es un estándar de la industria ampliamente utilizado en la ingeniería de software para la especificación, visualización, construcción y documentación de los artefactos de un sistema de software. UML proporciona un conjunto de notaciones gráficas que permiten representar de manera clara y estructurada los diferentes aspectos de un sistema, facilitando la comunicación técnica entre los desarrolladores, arquitectos y los distintos actores del proyecto (Booch, Rumbaugh, & Jacobson, 2005).

Desarrollado inicialmente en la década de 1990 y adoptado como estándar oficial por el Object Management Group (OMG) en 1997, UML ha evolucionado continuamente. Para los propósitos operativos de esta plataforma, se toma como referencia estricta la especificación de **UML 2.5 en adelante**, garantizando que las estructuras semánticas y notaciones visuales modeladas cumplan con los estándares internacionales de la ingeniería de sistemas modernos (OMG, 2017).

- **Propósitos de UML en la Plataforma de Workflows**

La aplicación de UML en el desarrollo y funcionamiento de este proyecto cumple de forma directa con los cuatro propósitos fundamentales de la disciplina (Booch et al., 2005):

- **Visualización:** Facilita la comprensión intuitiva de la lógica de los trámites institucionales mediante interfaces gráficas comprensibles para cualquier funcionario.
- **Especificación:** Define con precisión matemática el comportamiento transaccional de las reglas del negocio, los tipos de datos de los formularios y las restricciones de los flujos de trabajo.

- **Construcción:** El grafo visual modelado en el lienzo interactivo sirve como plano directo para que el motor de Spring Boot calcule los cambios de estado y las transiciones operativas.
- **Documentación:** Genera un registro técnico riguroso de la persistencia de datos en MongoDB, las secuencias de edición colaborativa en tiempo real y la arquitectura del sistema.

- **Tipos de Diagramas UML Implementados**

Para la construcción y el funcionamiento de la plataforma de políticas de negocio, se seleccionaron y aplicaron diagramas específicos de las dos grandes categorías de UML (OMG, 2017):

- **Diagramas Estructurales (Enfoque Estático):**
 - **Diagrama de Clases:** Utilizado para diseñar la estructura interna del backend en Spring Boot, modelando las entidades, relaciones, servicios y los repositorios necesarios para mapear las políticas y los trámites.
 - **Diagrama de Componentes:** Empleado para documentar la organización modular del monorrepositorio, ilustrando cómo interactúan el backend, el cliente web en Angular y el cliente móvil en Flutter.
- **Diagramas de Comportamiento (Enfoque Dinámico):**
 - **Diagrama de Casos de Uso:** Diseñado en la fase de inicio para capturar los requisitos funcionales del Administrador (diseño de políticas, creación de usuarios) y del Funcionario (atención de trámites, llenado de formularios).
 - **Diagrama de Secuencia:** Utilizado para modelar la comunicación asíncrona en tiempo real mediante WebSockets, detallando el intercambio de mensajes entre los clientes y el servidor durante las sesiones colaborativas.
 - **Diagrama de Actividades (El Núcleo del Sistema):** Este diagrama representa el componente central de la aplicación. El editor visual interactivo de Angular se construyó específicamente como una herramienta CASE orientada a permitir que el usuario dibuje Diagramas de Actividades de UML 2.5. El sistema procesa e interpreta sus elementos nativos: Nodos de Inicio y Fin, Actividades (tareas de las oficinas), Nodos de Decisión (rombos de flujos alternativos), y Barras de Sincronización (*Fork* y *Join* para flujos simultáneos o paralelos), organizados obligatoriamente dentro de Calles o Carriles (*Swimlanes*) que delimitan la responsabilidad de cada departamento de la empresa.

- **Importancia de UML**

El uso estricto de la notación UML aporta múltiples beneficios críticos en esta plataforma. Al unificar el diseño visual de los procesos empresariales bajo un estándar mundial de ingeniería, se garantiza que cualquier política de negocio modelada por un administrador sea universalmente entendible y ejecutable. Asimismo, faculta al software para validar la integridad lógica de los diagramas creados —detectando errores de diseño o terminales faltantes antes de su puesta en marcha—, traduciendo el modelado visual en un motor transaccional robusto, automatizado y libre de fallos conceptuales en producción (Bass et al., 2013).

1.1.9. Desarrollo Móvil Multiplataforma (Flutter)

El desarrollo móvil moderno ha evolucionado hacia la adopción de frameworks multiplataforma que permiten compilar aplicaciones nativas de alto rendimiento para múltiples sistemas operativos (iOS, Android) a partir de una única base de código fuente. En este ecosistema, **Flutter** se destaca como un kit de desarrollo de software (SDK) de código abierto creado por Google, caracterizado por su motor de renderizado propio y su arquitectura basada completamente en un árbol de widgets altamente combinables (Nader, 2021).

- **Características y Ventajas en el Proyecto:**

- **Modo Offline-First (Resiliencia en Campo):** En el contexto de esta plataforma de workflows, Flutter provee la infraestructura reactiva necesaria para gestionar estados locales mediante almacenamiento local aislado. Esto faculta al funcionario para rellenar formularios y avanzar trámites en campo sin depender de una conexión activa a internet.
- **Hot Reload Extremo:** Agiliza significativamente las iteraciones del ciclo de desarrollo al permitir visualizar modificaciones en la interfaz de usuario en tiempo real sobre el dispositivo sin necesidad de reiniciar el estado de la aplicación.
- **Interoperabilidad Nativa:** Se comunica de forma fluida con servicios de geolocalización o almacenamiento del dispositivo, empaquetando la información en estructuras JSON listas para ser sincronizadas asincrónicamente con el backend en Spring Boot.

1.1.10. Modelo C4

El Modelo C4 es un enfoque de diseño estructurado que busca resolver la complejidad de la documentación arquitectónica mediante la abstracción de los sistemas de software en niveles jerárquicos de detalle (Contexto, Contenedores, Componentes y Código). A diferencia de los diagramas de despliegue tradicionales de UML, C4 funciona como un mapa interactivo (un "Google Maps" del software) que permite a los ingenieros y stakeholders entender la distribución técnica sin perderse en detalles de bajo nivel (Brown, 2018).

- **Niveles Aplicados en el Ciclo 2:**

- **Nivel 1: Contexto:** Muestra cómo el sistema de políticas de negocio interactúa de forma global con los funcionarios, clientes y sistemas externos de notificación.

- **Nivel 2: Contenedores:** Detalla la separación física de las aplicaciones del monorrepositorio, ilustrando cómo el Front-end (Angular) y la App Móvil (Flutter) consumen el núcleo transaccional (Spring Boot), el cerebro cognitivo (Python FastAPI), y cómo interactúan con las bases de datos NoSQL y el almacenamiento en la nube de AWS S3.
- **Nivel 3: Componentes:** Desglosa la organización de los controladores, servicios e interceptores internos que dan vida al puente de comunicación entre Java y Python.

1.1.11 Sistemas de Gestión Documental Electrónica (EDMS)

Un Sistema de Gestión Documental (EDMS, por sus siglas en inglés Electronic Document Management System) es una infraestructura de software diseñada para rastrear, almacenar, organizar y controlar el ciclo de vida de los documentos electrónicos dentro de una organización, asegurando la transición hacia procesos completamente digitalizados (99% papel cero). Estos sistemas proveen capacidades avanzadas para el manejo de metadatos, control de versiones, flujos de aprobación paralelos e integración de políticas de seguridad restrictivas (Smith, 2018).

- **Características Implementadas en el Incremento del Workflow:**
 - **Soporte de Variedad de Formatos:** Capacidad de ingesta directa de archivos heterogéneos generados a lo largo de un trámite institucional, incluyendo documentos de texto (.docx, .pdf), hojas de cálculo (.xlsx), imágenes de planos y evidencias en video.
 - **Matriz de Privilegios por Nodo Estructural:** Control de acceso granular enlazado directamente a los carriles organizacionales del diagrama UML. Permite definir de forma estricta qué dpto. tiene autorización para subir archivos, quién posee permisos de modificación colaborativa síncrona y qué roles están limitados a solo lectura.
 - **Pista de Auditoría Rigorosa (Audit Logs):** Almacenamiento inmutable de los metadatos asociados a cada archivo. El sistema registra obligatoriamente los atributos de control: identidad del emisor, fecha y hora de carga, modificaciones efectuadas y registro de visualizaciones para fines de transparencia institucional.

1.1.12 Computación e Infraestructura en la Nube (Amazon Web Services - AWS)

La computación en la nube representa la disponibilidad bajo demanda de los recursos del sistema informático, especialmente el almacenamiento de datos y la capacidad de cómputo, sin una gestión directa por parte del usuario. Para afrontar el desafío de almacenar grandes volúmenes de documentación masiva generada por los trámites concurrentes, se adopta la arquitectura en la nube provista por Amazon Web Services (AWS), garantizando alta velocidad de respuesta, tolerancia a fallos y una drástica reducción de costos de infraestructura local (Baron, 2020).

- **Servicios AWS Integrados en el Ecosistema:**
 - **Amazon S3 (Simple Storage Service):** Utilizado como el repositorio binario principal. La plataforma organiza los documentos de forma aislada estructurando buckets lógicos segmentados estrictamente por política_id y cliente_id_tramite_id. El frontend web y móvil consumen este servicio mediante *Presigned URLs* generadas por Spring Boot, permitiendo la subida veloz directa a la nube sin sobrecargar el servidor web.

1.1.13 Redes Neuronales y Aprendizaje Profundo (Deep Learning / NLP)

El Aprendizaje Profundo (*Deep Learning*) es un subcampo del aprendizaje automático (*Machine Learning*) basado en redes neuronales artificiales con múltiples capas de abstracción, especializado en modelar y resolver problemas no lineales de alta complejidad a partir de grandes volúmenes de datos. Dentro de este paradigma, el Procesamiento de Lenguaje Natural (NLP) impulsado por Deep Learning faculta a los sistemas de software para interpretar, analizar y extraer variables estructuradas directamente a partir del lenguaje humano escrito o hablado (Goodfellow, Bengio & Courville, 2016).

- **Modelos Cognitivos Implementados en Python:**
 - **Agente Inteligente Front-Desk Conversacional:** Red neuronal entrenada en procesamiento de texto y audio para interactuar de forma autónoma con el cliente mediante comandos de voz y prompts. Clasifica la problemática libre expresada por el usuario, infiere y asigna la política de negocio UML correcta que debe iniciar, y valida mediante IA si los documentos subidos cumplen sintácticamente con los requisitos obligatorios de la plantilla.
 - **Motor Inteligente de Enrutamiento y Análisis de Riesgo:** Algoritmos predictivos que procesan el histórico relacional de trámites anteriores para estimar la mejor ruta de atención, predecir con exactitud probabilística el riesgo de demoras o cuellos de botella en departamentos específicos, e identificar anomalías estructurales en los tiempos de respuesta del personal.
 - **Generador Dinámico de Reportes por NLP:** Modelo lingüístico que analiza las instrucciones de texto o notas de voz del jefe de políticas, extrayendo de forma estricta tres ejes de información estructural (**Datos requeridos**, **Criterios de filtrado** y **Formatos de salida**). Implementa reglas lógicas conversacionales para repreguntar de forma interactiva en caso de capturar datos incompletos, evitando la interrupción del sistema.

Parte II – Proceso de desarrollo

2.1 Flujo de Trabajo: Captura de Requisitos

2.1.1 Actores y casos de uso

2.1.1.1 Actores

A1. Administrador. - responsable de la configuración estratégica, seguridad y modelado del sistema. Su rol principal consiste en diseñar visualmente los workflows organizacionales, gestionar el registro y roles de los usuarios, parametrizar los formularios dinámicos departamentales, evaluar el rendimiento institucional ante cuellos de botella y auditar los reportes gerenciales del negocio.

A2. Funcionario Operativo. - usuario interno perteneciente a una sección o departamento específico de la organización (Servicio al Cliente, Asesoría Legal, Técnicos de Campo, Almacén, etc.). Su función principal es procesar las tareas asignadas a su bandeja de entrada mediante el monitor proactivo, registrar los datos requeridos en los formularios dinámicos de su competencia y gestionar la documentación digital asociada al ciclo de vida del trámite.

A3. Cliente. - persona externa que inicia una solicitud de trámite. Su rol consiste en proporcionar la información y requisitos de admisión iniciales requeridos por el flujo, realizar el seguimiento externo interactivo de la ubicación y estado de su expediente y descargar el documento de resolución o certificación final provisto por el sistema.

2.1.1.2 Casos de Uso

- CU-01. Autenticación y Gestión de Sesión
- CU-02. Gestión de Departamentos (CRUD)
- CU-03. Modelado Visual de Políticas de Negocio (Editor BPM)
- CU-04. Copilot IA de Diagramas (Diseño por Lenguaje Natural)
- CU-05. Generación de Código UML
- CU-06. Detección de Cuellos de Botella y Optimización IA
- CU-07. Gestión de Bandeja de Entrada
- CU-08. Procesamiento y Estado de Tarea
- CU-09. Notificaciones en Tiempo Real
- CU-10. Inicio de Trámite
- CU-11. Seguimiento Interactivo de Trámite
- CU-12. Generación y Descarga de Resolución (PDF)
- CU-13. Asistente Virtual (WorkBot)
- CU-14. Asistente Front-Desk Inteligente (Clasificación de Solicitud)
- CU-15. Repositorio Documental Digital
- CU-16. Acceso Móvil / PWA Offline
- CU-17. Gestión Documental Colaborativa de Trámites
- CU-18. Extracción Automática de Datos de Documento (IA)
- CU-19. Motor de Enrutamiento Inteligente y Análisis de Riesgo
- CU-20. Generación Dinámica de Reportes por Lenguaje Natural:
- CU-21. Clarificación Conversacional de Reportes
- CU-22. Gestión de Almacenamiento en Nube con AWS S3

2.1.1.3 Priorización de Casos de Uso

Ciclo 1

ID	Caso de Uso	Estado	Prioridad	Riesgo	Actor
CU-01	Autenticación y Gestión de Sesión	Aprobado	Alta	Bajo	Administrador / Funcionario / Cliente
CU-02	Gestión de Departamentos (CRUD)	Aprobado	Normal	Bajo	Administrador
CU-03	Modelado Visual de Políticas de Negocio (Editor BPM)	Aprobado	Alta	Alto	Administrador
CU-04	Copilot IA de Diagramas (Diseño por Lenguaje Natural)	Aprobado	Alta	Alto	Administrador
CU-05	Generación de Código UML	Aprobado	Normal	Medio	Administrador
CU-06	Detección de Cuellos de Botella y Optimización IA	Aprobado	Alta	Alto	Administrador
CU-07	Gestión de Bandeja de Entrada	Aprobado	Alta	Medio	Funcionario Operativo
CU-08	Procesamiento y Estado de Tarea	Aprobado	Alta	Alto	Funcionario Operativo
CU-09	Notificaciones en Tiempo Real	Aprobado	Normal	Medio	Funcionario Operativo
CU-10	Inicio de Trámite	Aprobado	Alta	Medio	Cliente
CU-11	Seguimiento Interactivo de Trámite	Aprobado	Alta	Bajo	Cliente
CU-12	Generación y Descarga de Resolución (PDF)	Aprobado	Normal	Bajo	Cliente
CU-13	Asistente Virtual (WorkBot)	Aprobado	Normal	Bajo	Administrador / Funcionario / Cliente

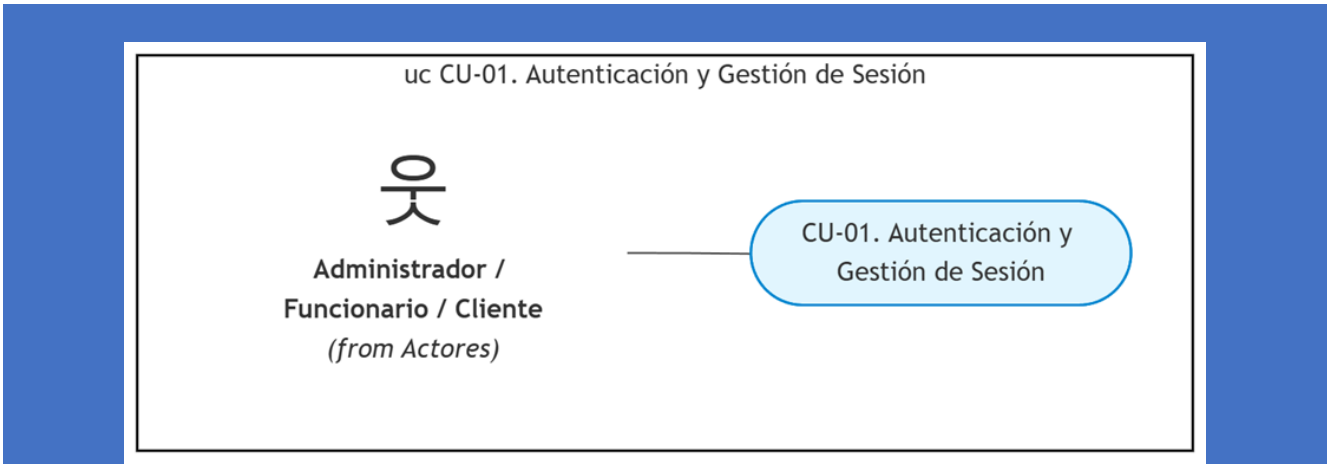
Ciclo 2

ID	Caso de Uso	Estado	Prioridad	Riesgo	Actor
CU-14	Asistente Front-Desk Inteligente (Clasificación de Solicitud)	Aprobado	Alta	Alto	Cliente
CU-15	Repositorio Documental Digital	Aprobado	Alta	Medio	Cliente / Funcionario Operativo
CU-16	Acceso Móvil / PWA Offline	Aprobado	Normal	Alto	Cliente / Funcionario Operativo
CU-17	Gestión Documental Colaborativa de Trámites	Aprobado	Alta	Alto	Funcionario Operativo
CU-18	Extracción Automática de Datos de Documento (IA)	Aprobado	Alta	Alto	Funcionario Operativo
CU-19	Motor de Enrutamiento Inteligente y Análisis de Riesgo	Aprobado	Alta	Alto	Administrador
CU-20	Generación Dinámica de Reportes por Lenguaje Natural:	Aprobado	Alta	Alto	Administrador
CU-21	Clarificación Conversacional de Reportes	Aprobado	Normal	Medio	Administrador
CU-22	Gestión de Almacenamiento en Nube con AWS S3	Aprobado	Alta	Alto	Administrador / Funcionario Operativo

2.1.1.4 Detallar casos de uso

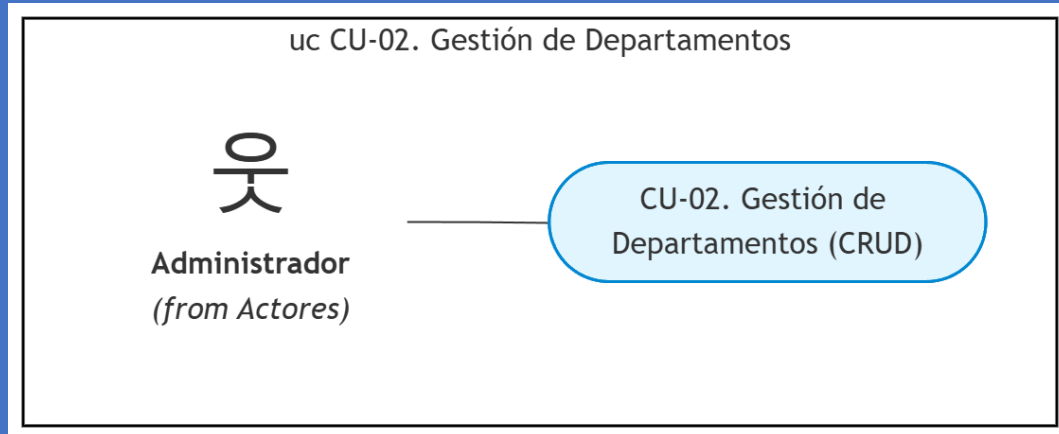
Ciclo 1

CU-01. Autenticación y Gestión de Sesión



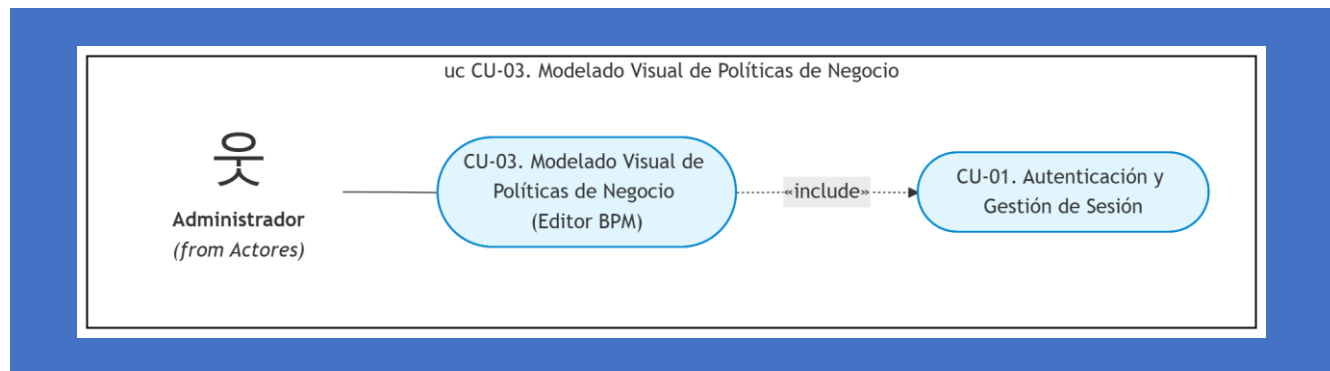
Nombre de Caso de Uso	CU-01. Autenticación y Gestión de Sesión
Propósito	Permitir a los usuarios ingresar y cerrar sesión de forma segura en la plataforma para acceder a las funcionalidades según su rol.
Actores	Administrador, Funcionario Operativo, Cliente
Actor Iniciador	Administrador, Funcionario Operativo, Cliente
Precondición	El usuario debe estar previamente registrado en el sistema.
Flujo Principal	1. El usuario accede al portal web o aplicación móvil. 2. Introduce sus credenciales (correo y contraseña). 3. El sistema valida las credenciales con MongoDB. 4. Se genera el token JWT y se redirige al panel según su rol.
Post Condición	El usuario queda autenticado y con sesión activa.
Excepción	Credenciales incorrectas o cuenta inhabilitada.

CU-02. Gestión de Departamentos (CRUD)



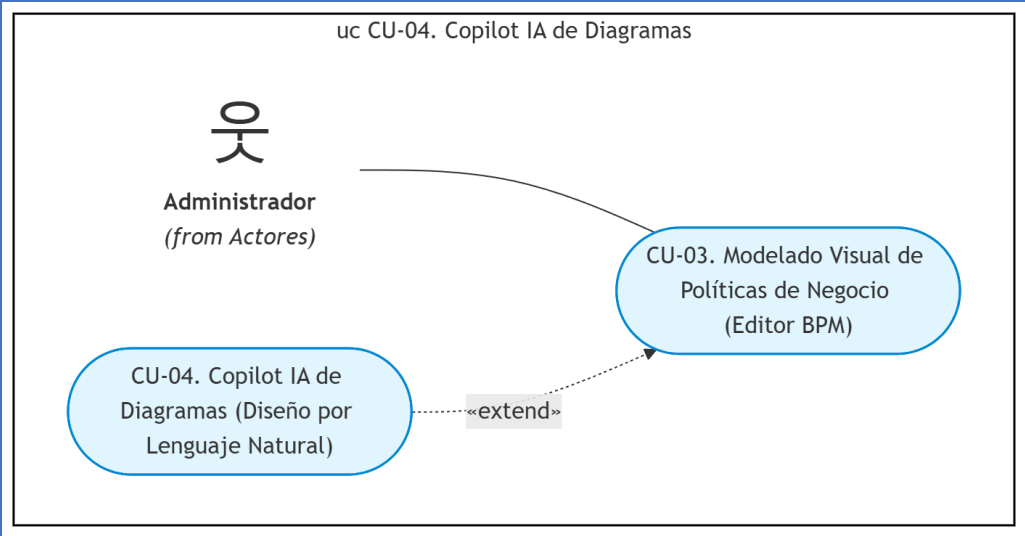
Nombre de Caso de Uso	CU-02. Gestión de Departamentos (CRUD)
Propósito	Permitir al Administrador crear, modificar, consultar y eliminar departamentos organizacionales para definir la asignación de tareas en los workflows.
Actores	Administrador
Actor Iniciador	Administrador
Precondición	El Administrador debe estar autenticado en el sistema.
Flujo Principal	1. El Administrador ingresa a la sección "Gestión de Departamentos". 2. El sistema muestra la lista de departamentos activos. 3. El Administrador puede elegir entre: - <i>Crear</i>: Ingresa el nombre del departamento y descripción. - <i>Editar</i>: Selecciona un departamento existente y modifica sus campos. - <i>Eliminar</i>: Selecciona un departamento y presiona eliminar. 4. El sistema valida los datos y guarda los cambios en MongoDB.
Post Condición	Los departamentos quedan actualizados y disponibles para ser asignados en los workflows.
Excepción	Campos obligatorios vacíos o nombre de departamento duplicado.

CU-03. Modelado Visual de Políticas de Negocio (Editor BPM)



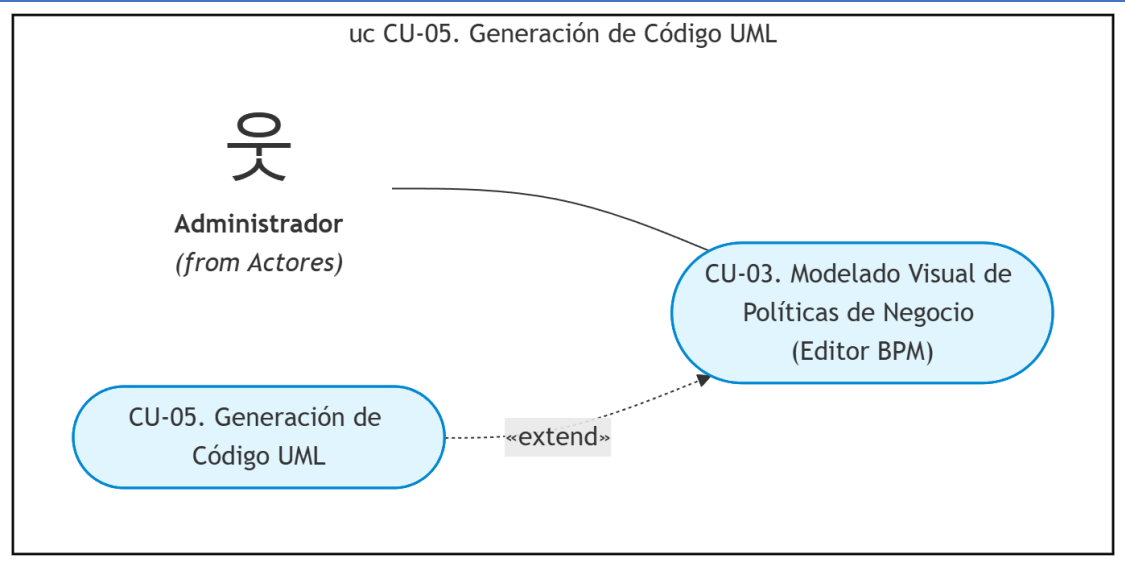
Nombre de Caso de Uso	CU-03. Modelado Visual de Políticas de Negocio (Editor BPM)
Propósito	Permitir al Administrador diseñar visualmente la secuencia de tareas, decisiones y departamentos que componen un trámite.
Actores	Administrador
Actor Iniciador	Administrador
Precondición	El Administrador debe haber iniciado sesión en la plataforma web.
Flujo Principal	1. El Administrador accede a la sección "Editor de Políticas". 2. Utiliza el lienzo interactivo para arrastrar y conectar nodos (Inicio, Tareas, Decisiones, Fin). 3. Asigna cada tarea a un departamento específico. 4. Guarda y activa la política de negocio en el sistema.
Post Condición	La Política de negocio queda registrada en base de datos y lista para iniciar trámites reales basados en ella.
Excepción	Conexiones inválidas (ej. nodos huérfanos) o falta de nombre en la política. Diagrama sin inicio/fin o con nodos desconectados (se advierte en pantalla y se impide guardar).

CU-04. Copilot IA de Diagramas (Diseño por Lenguaje Natural)



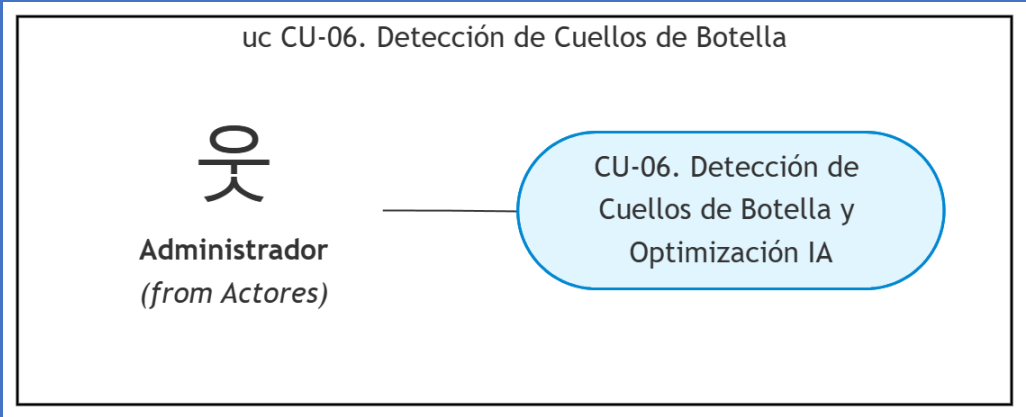
Nombre de Caso de Uso	CU-04. Copilot IA de Diagramas (Diseño por Lenguaje Natural)
Propósito	Permitir al Administrador crear, estructurar y enlazar nodos de un workflow de forma automática en el editor utilizando instrucciones de texto en lenguaje natural.
Actores	Administrador
Actor Iniciador	Administrador
Precondición	El Administrador debe estar dentro del Editor BPM (CU-03) en el navegador.
Flujo Principal	1. El Administrador abre el panel del Copilot en el editor de diagramas. 2. Ingresa un comando escrito (ej: <i>"Agrega un flujo completo: inicio, verificación, aprobación, fin"</i>). 3. El sistema envía la solicitud al backend donde la procesa la IA. 4. La IA genera un JSON con la estructura de nodos y enlaces y la devuelve al editor web. 5. El editor web actualiza instantáneamente el lienzo gráfico con los nodos y relaciones creados.
Post Condición	El workflow se actualiza visualmente en pantalla a partir de la instrucción en lenguaje natural.
Excepción	La IA no comprende el texto (envía un mensaje aclaratorio en el chat del asistente).

CU-05. Generación de Código UML



Nombre de Caso de Uso	CU-05. Generación de Código UML
Propósito	Generar el código de especificación PlantUML para diagramas de actividades a partir del workflow modelado en la aplicación.
Actores	Administrador
Actor Iniciador	Administrador
Precondición	El Administrador debe tener cargada una política de negocio en el editor.
Flujo Principal	1. El Administrador hace clic en la opción "Generar PlantUML" dentro del editor. 2. El backend recopila el listado de nodos, tipos de nodos, conexiones y departamentos de la política en MongoDB. 3. El sistema formatea esta información y la envía a la IA para traducirla a código PlantUML. 4. La IA genera el script estructurado de PlantUML. 5. El sistema presenta el código en pantalla con opción de copiado rápido al portapapeles.
Post Condición	El script PlantUML se genera de forma correcta para que el administrador pueda documentar el proceso.
Excepción	La política de negocio no contiene ningún nodo registrado (se cancela la generación).

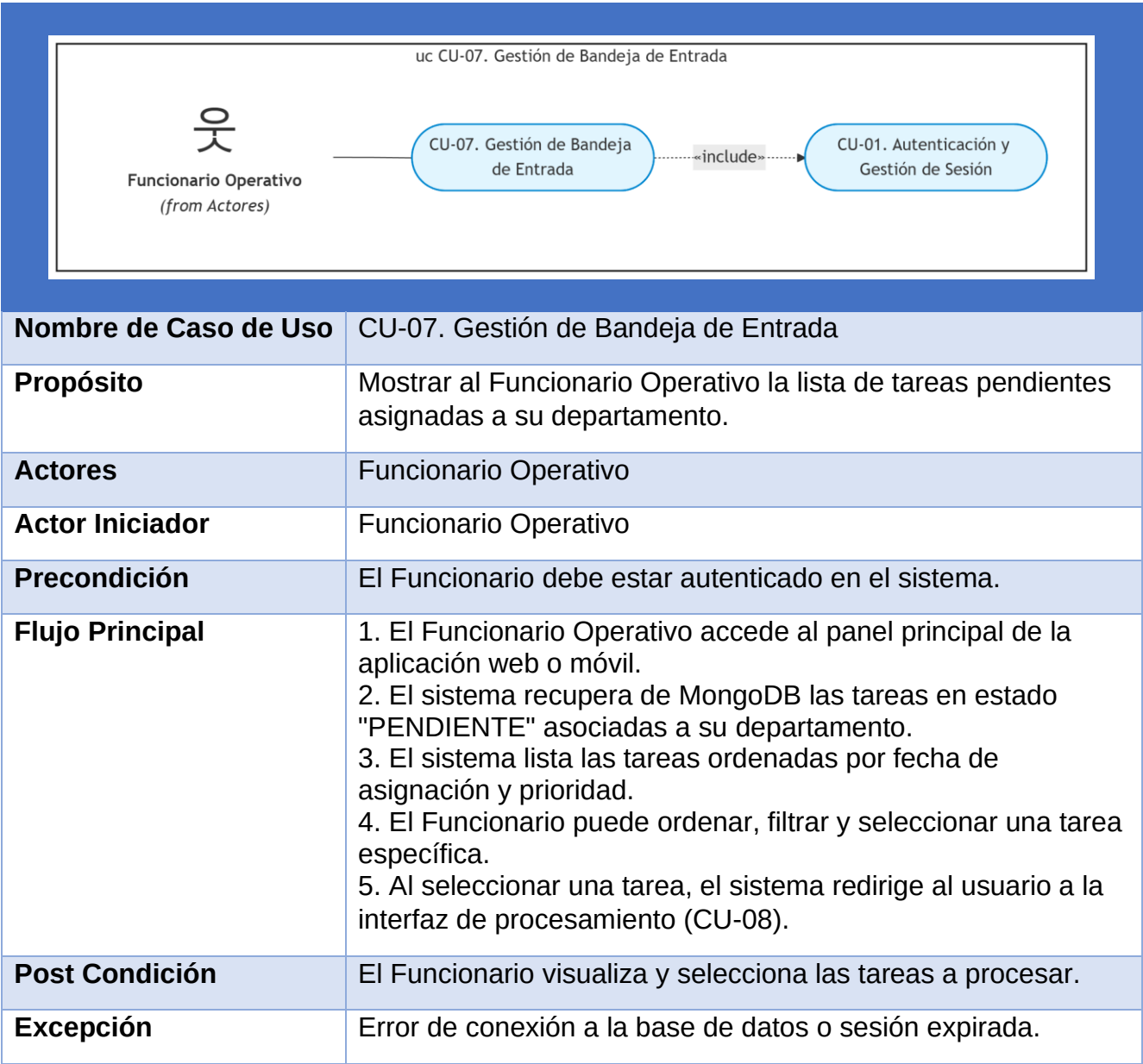
CU-06. Detección de Cuellos de Botella y Optimización IA



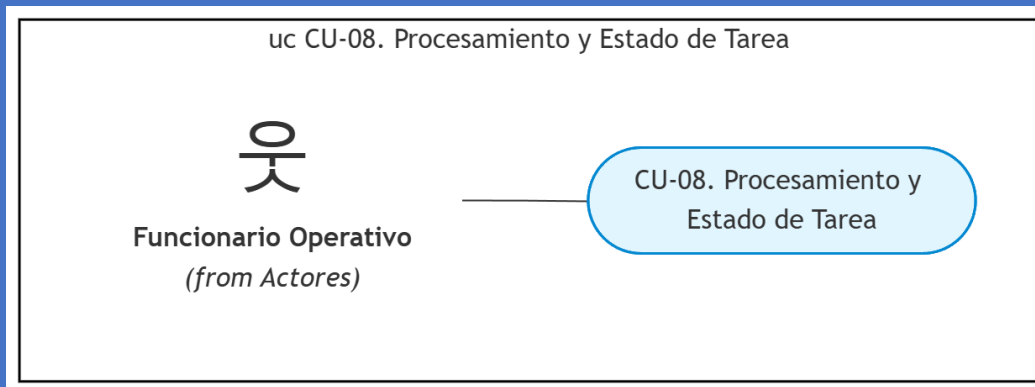
Nombre de Caso de Uso	CU-06. Detección de Cuellos de Botella y Optimización IA
Propósito	Detectar qué nodos de una política superan su tiempo límite establecido, notificar al administrador sobre estas ineficiencias y generar propuestas automáticas de optimización de procesos por medio de IA.
Actores	Administrador
Actor Iniciador	Administrador
Precondición	Deben existir trámites e instancias de tareas completadas registradas en la base de datos.
Flujo Principal	1. El Administrador accede a la sección de Analíticas y selecciona la política a evaluar. 2. El sistema calcula en el backend el promedio de tiempo real de finalización de tareas de cada nodo y lo compara con el tiempoLimiteHoras configurado. 3. Al encontrar excedentes, el sistema dispara notificaciones de alerta tipo "Cuello de botella" al administrador. 4. El backend envía las estadísticas y desviaciones a la IA para su análisis estructural. 5. La IA responde con un informe descriptivo y tres recomendaciones concretas de reingeniería de procesos. 6. El Administrador visualiza el informe de recomendaciones y el panel de analítica.
Post Condición	El sistema detecta los cuellos de botella reales y provee al Administrador de alternativas estratégicas para agilizar el workflow.

Excepción	No hay suficientes datos de tareas completadas para calcular promedios (se indica mensaje de falta de historial).
------------------	---

CU-07. Gestión de Bandeja de Entrada

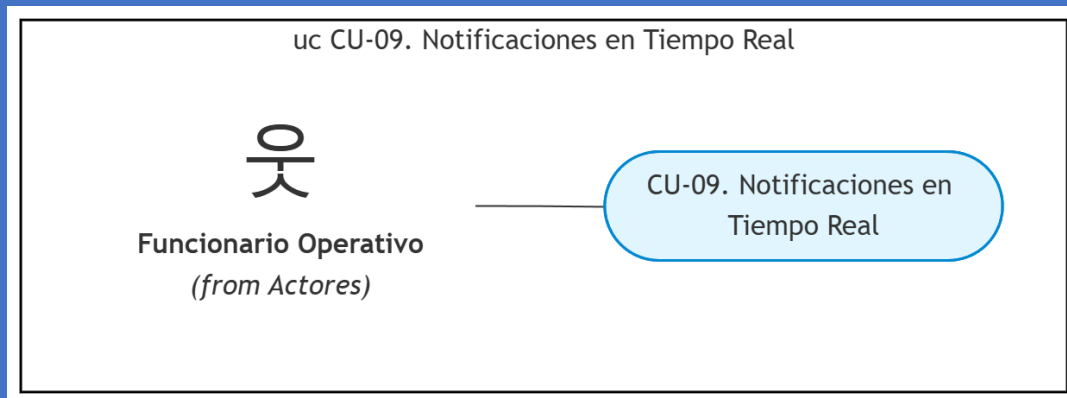


CU-08. Procesamiento y Estado de Tarea



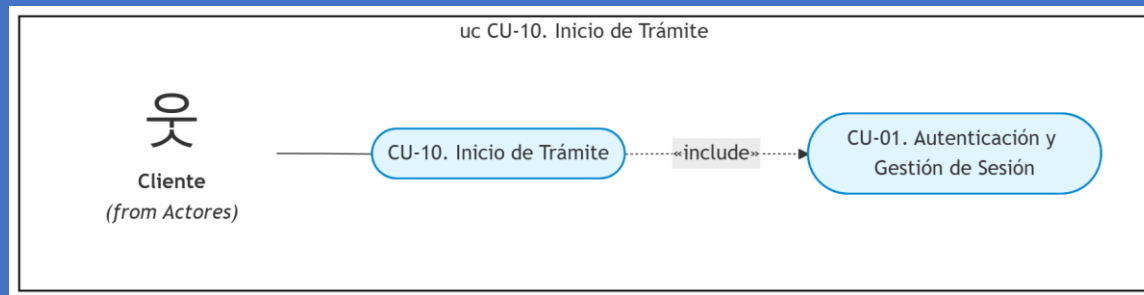
Nombre de Caso de Uso	CU-08. Procesamiento y Estado de Tarea
Propósito	Permitir al Funcionario Operativo procesar las tareas pendientes asignadas a su área de competencia, registrar observaciones y cambiar su estado para hacer avanzar el trámite.
Actores	Funcionario Operativo
Actor Iniciador	Funcionario Operativo
Precondición	El Funcionario Operativo debe tener seleccionada una tarea activa desde su bandeja de entrada (CU-07).
Flujo Principal	1. El Funcionario Operativo accede al detalle de la tarea. 2. El sistema renderiza el formulario correspondiente al nodo del trámite. 3. El Funcionario digita la información solicitada y añade notas aclaratorias. 4. El Funcionario selecciona cambiar el estado de la tarea (ej: "Completar" o "Observar"). 5. El sistema guarda la fecha/hora de finalización, calcula el tiempo transcurrido en minutos y actualiza el estado de la tarea en MongoDB. 6. El motor del workflow evalúa el flujo de la política y genera la tarea del siguiente nodo en el carril correspondiente.
Post Condición	La tarea actual finaliza y el trámite avanza automáticamente al nodo subsiguiente.
Excepción	Campos obligatorios vacíos o falta de permisos en el rol (se bloquea la acción).

CU-09. Notificaciones en Tiempo Real



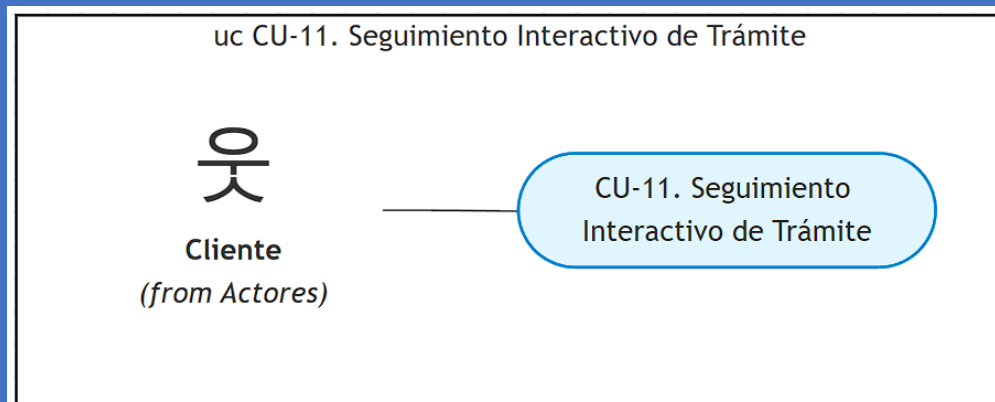
Nombre de Caso de Uso	CU-09. Notificaciones en Tiempo Real
Propósito	Notificar instantáneamente al Funcionario Operativo mediante WebSockets en la aplicación web o alertas Push de Firebase (FCM) en la aplicación móvil en cuanto ingresa una nueva tarea a su departamento.
Actores	Funcionario Operativo
Actor Iniciador	Sistema (Motor de Workflows del Backend)
Precondición	El Funcionario Operativo debe estar conectado a la red y tener su sesión iniciada.
Flujo Principal	1. El motor del backend genera una nueva tarea asignada a un departamento. 2. El sistema detecta el evento de asignación y emite una alerta a través de la conexión activa de WebSockets. 3. Simultáneamente, el servidor invoca al servicio FCM para enviar la notificación Push al dispositivo móvil del funcionario. 4. El frontend web del funcionario detecta el mensaje por WebSocket, reproduce un sonido y actualiza el contador de tareas pendientes de la bandeja sin necesidad de recargar la página.
Post Condición	El funcionario recibe y visualiza la notificación en su dispositivo/navegador de forma inmediata.
Excepción	El funcionario está sin conexión (el sistema guarda la notificación en MongoDB como "no leída" para mostrarla al iniciar sesión).

CU-10. Inicio de Trámite



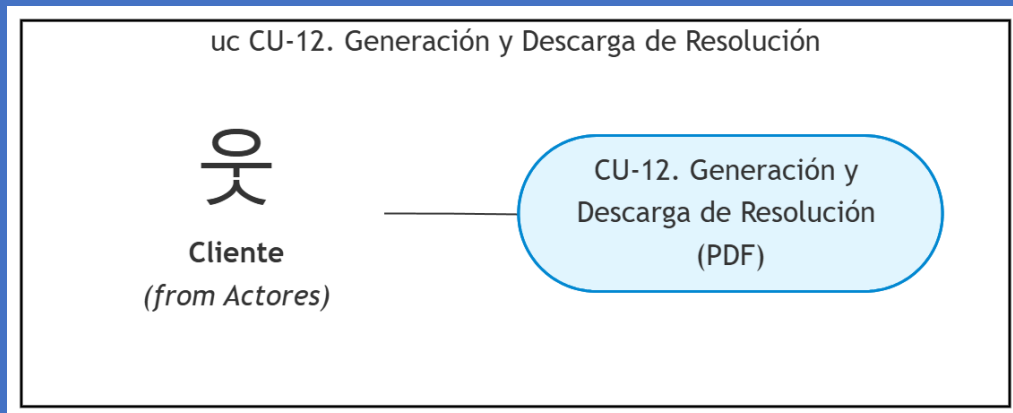
Nombre de Caso de Uso	CU-10. Inicio de Trámite
Propósito	Permitir al cliente iniciar un trámite en la plataforma bajo una política de negocio activa.
Actores	Cliente
Actor Iniciador	Cliente
Precondición	El Cliente debe estar autenticado en el sistema.
Flujo Principal	1. El Cliente ingresa al portal de trámites y selecciona la política a iniciar. 2. Rellena los campos iniciales del formulario. 3. Envía la solicitud. 4. El sistema crea una nueva instancia de Trámite en MongoDB con estado "EN_PROGRESO" y genera la primera tarea.
Post Condición	El trámite es creado y se asigna la primera tarea al departamento correspondiente.
Excepción	La política seleccionada no está activa o faltan datos obligatorios.

CU-11. Seguimiento Interactivo de Trámite



Nombre de Caso de Uso	CU-11. Seguimiento Interactivo de Trámite
Propósito	Permitir al Cliente visualizar gráficamente la ubicación actual de su expediente, el departamento que lo tiene y la línea de tiempo con el historial detallado de tareas completadas.
Actores	Cliente
Actor Iniciador	Cliente
Precondición	El Cliente debe estar autenticado y tener al menos un trámite registrado.
Flujo Principal	1. El Cliente ingresa al portal y hace clic en la sección "Mis Trámites". 2. El sistema consulta a MongoDB y le muestra la lista de sus trámites con sus códigos de referencia únicos. 3. El Cliente selecciona un trámite. 4. El sistema recupera el estado de la tarea activa y el histórico de transiciones. 5. El sistema renderiza una línea de tiempo y un diagrama visual resaltando el nodo actual donde se encuentra el expediente, el nombre del funcionario encargado y el tiempo transcurrido.
Post Condición	El Cliente visualiza la traza completa de su trámite de forma interactiva y transparente.
Excepción	El ID del trámite ingresado es inválido o no pertenece a la cuenta (se restringe el acceso).

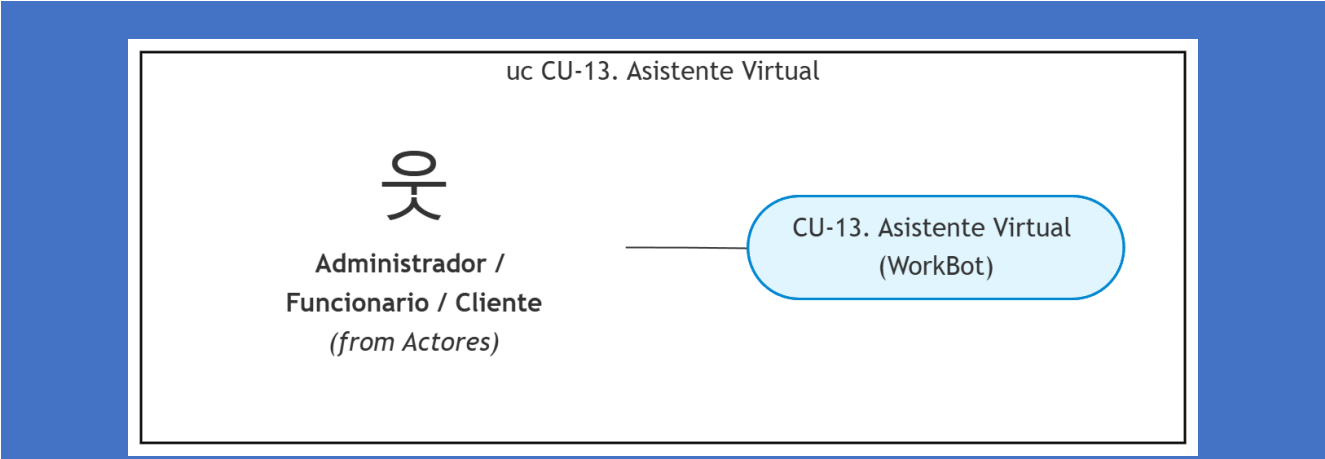
CU-12. Generación y Descarga de Resolución (PDF)



Nombre de Caso de Uso	CU-12. Generación y Descarga de Resolución (PDF)
Propósito	Permitir al Cliente descargar una copia digital en PDF oficial de la resolución, certificado o documento resolutivo que se expide al finalizar exitosamente su trámite en el workflow.
Actores	Cliente
Actor Iniciador	Cliente
Precondición	El trámite debe haber culminado con éxito (el nodo final debe haber sido alcanzado).
Flujo Principal	1. El Cliente ingresa al portal, localiza su trámite concluido y selecciona "Ver Detalles" . 2. El sistema habilita la opción de "Descargar PDF de Resolución". 3. El Cliente presiona el botón. 4. El backend procesa los datos registrados en el trámite y el histórico de firmas y genera un documento PDF estructurado en memoria. 5. El sistema entrega el flujo de bytes del archivo PDF para que el navegador o aplicación móvil inicie la descarga.
Post Condición	El Cliente descarga y almacena el documento PDF de resolución final en su almacenamiento local.

Excepción	El trámite aún no ha finalizado o fue anulado (no se genera el botón de descarga).
------------------	--

CU-13. Asistente Virtual (WorkBot)

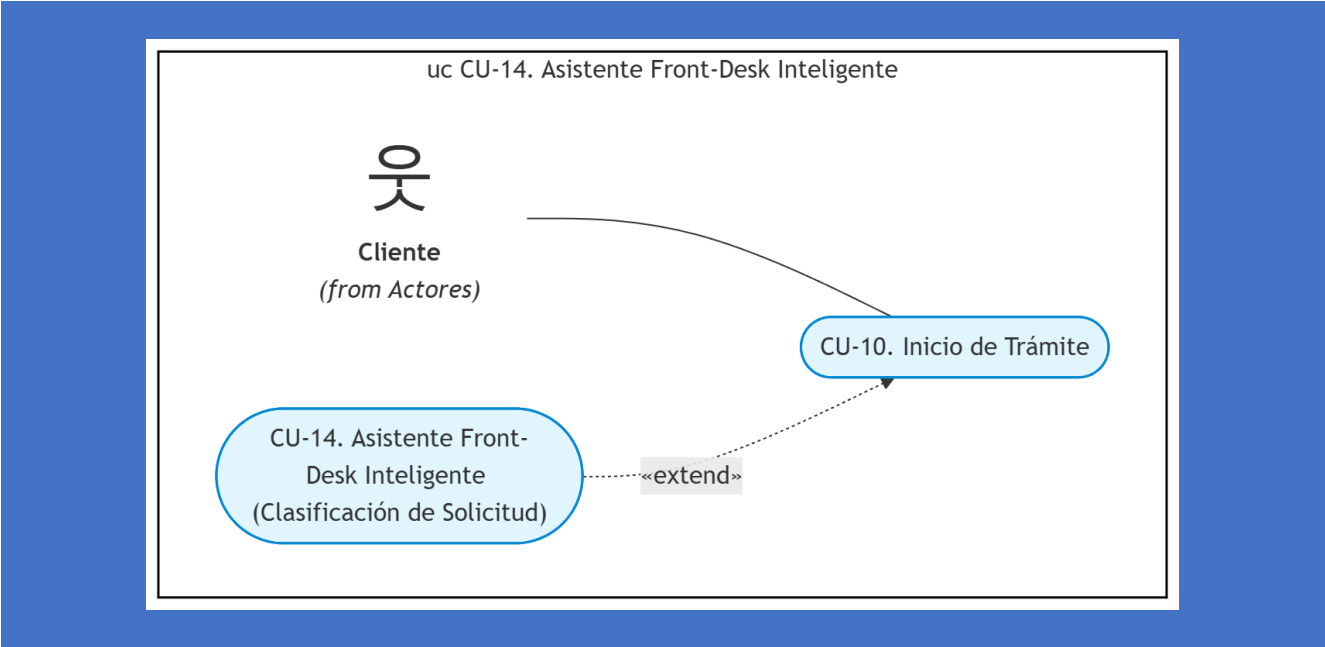


Nombre de Caso de Uso	CU-13. Asistente Virtual (WorkBot)
Propósito	Ofrecer soporte y respuestas automáticas personalizadas en lenguaje natural a los usuarios sobre cómo operar la aplicación según sus respectivos roles.
Actores	Administrador, Funcionario Operativo, Cliente
Actor Iniciador	Administrador, Funcionario Operativo, Cliente
Precondición	El usuario debe estar en la interfaz web de la aplicación y abrir el chat del WorkBot.
Flujo Principal	1. El usuario hace clic sobre el botón flotante del asistente inteligente en pantalla. 2. Se despliega una caja de chat conversacional. 3. El usuario escribe o dicta una consulta sobre el uso del sistema (ej: "<i>¿Qué hago si mi tarea está observada?</i>"). 4. El frontend envía la pregunta al servicio de IA en el backend (/api/ai/asistente). 5. La IA procesa la consulta utilizando el contexto de ayuda del sistema y genera una respuesta amigable. 6. El usuario visualiza en tiempo real la respuesta explicativa del WorkBot en el chat.
Post Condición	El usuario recibe asistencia inmediata en lenguaje natural sobre las funcionalidades del sistema.

Excepción	Interrupción en el servicio de IA (el bot ofrece disculpas con un mensaje predefinido fuera de línea).
------------------	--

Ciclo 2

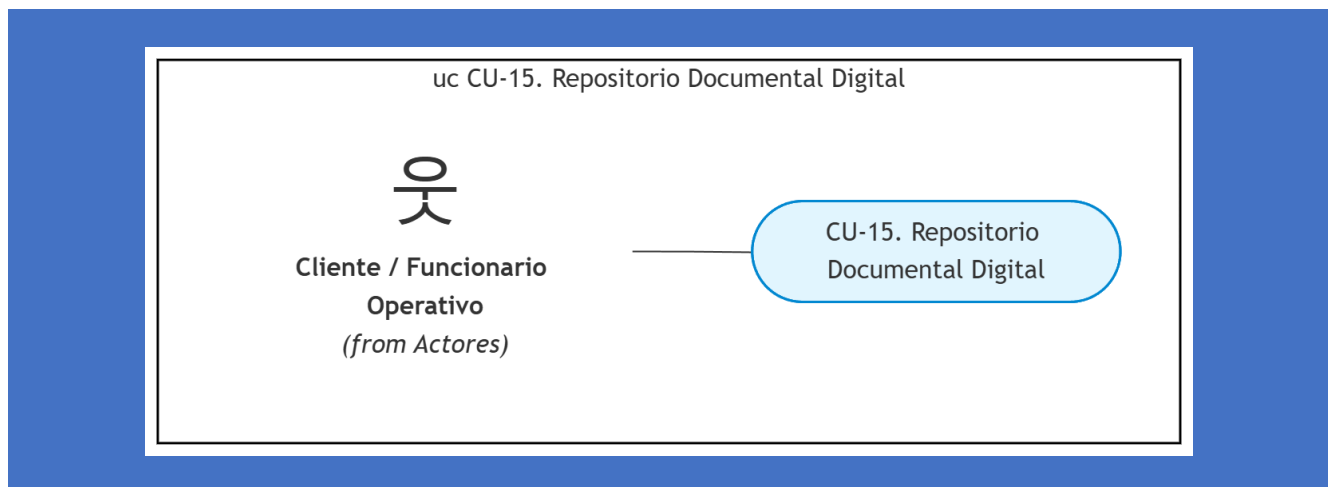
CU-14. Asistente Front-Desk Inteligente (Clasificación de Solicitud)



Nombre de Caso de Uso	CU-14. Asistente Front-Desk Inteligente (Clasificación de Solicitud)
Propósito	Analizar la solicitud del cliente escrita en lenguaje natural para clasificarla en una política activa y extraer sus datos iniciales mediante IA.
Actores	Cliente
Actor Iniciador	Cliente
Precondición	El Cliente debe estar en la interfaz de creación de trámite.
Flujo Principal	1. El Cliente escribe su solicitud en texto libre en la caja del Front-Desk. 2. El sistema envía el texto al microservicio de IA (Python FastAPI). 3. La IA analiza las políticas activas vigentes y clasifica la solicitud. 4. La IA responde con la política sugerida, nivel de confianza, requisitos y datos pre-llenados. 5. El sistema presenta de forma visual la recomendación para que el cliente la acepte.

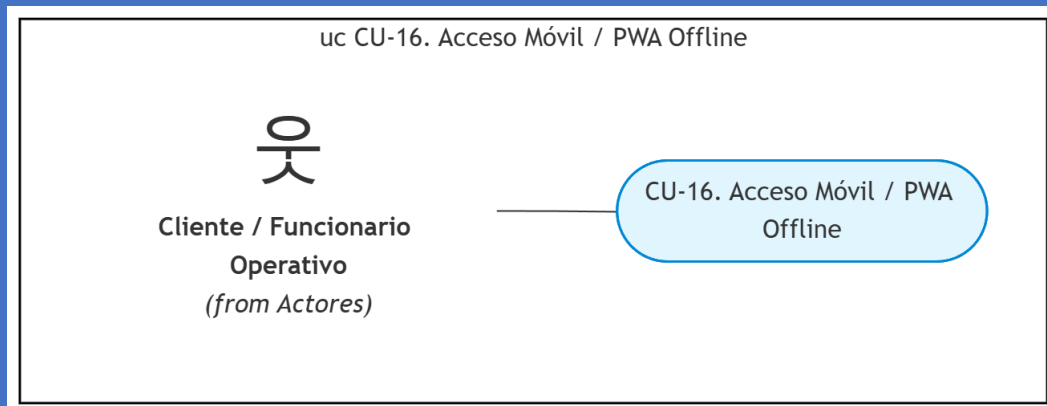
Post Condición	La solicitud queda clasificada y el formulario inicial es pre-llenado de forma predictiva.
Excepción	Confianza de clasificación demasiado baja (requiere selección manual por el cliente).

CU-15. Repositorio Documental Digital



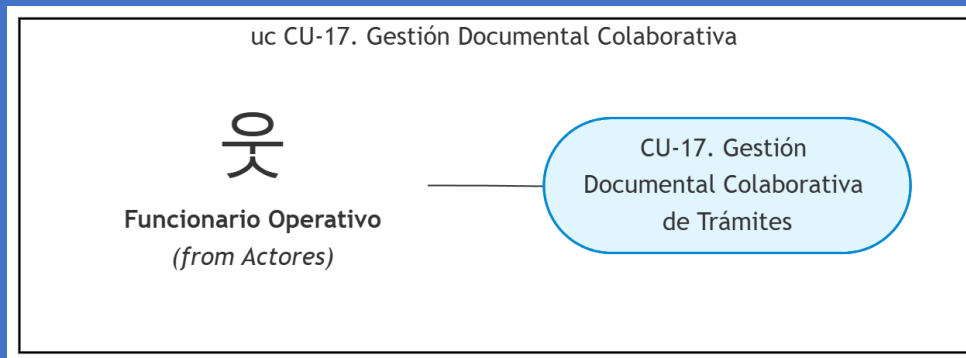
Nombre de Caso de Uso	CU-15. Repositorio Documental Digital
Propósito	Almacenar y consultar de manera centralizada y digitalizada todos los documentos asociados a un trámite en ejecución.
Actores	Cliente, Funcionario Operativo
Actor Iniciador	Cliente, Funcionario Operativo
Precondición	El trámite debe estar en ejecución y el usuario debe tener permisos sobre el mismo.
Flujo Principal	1. El usuario accede a la sección de archivos de la tarea o del trámite. 2. El sistema recupera y muestra el listado de documentos cargados en el trámite. 3. El usuario puede subir un nuevo archivo o descargar uno existente. 4. Se registra en la base de datos la fecha de carga y el usuario responsable.
Post Condición	El repositorio se actualiza y los documentos quedan vinculados al trámite.
Excepción	Tipo de archivo no permitido o límite de tamaño excedido.

CU-16. Acceso Móvil / PWA Offline



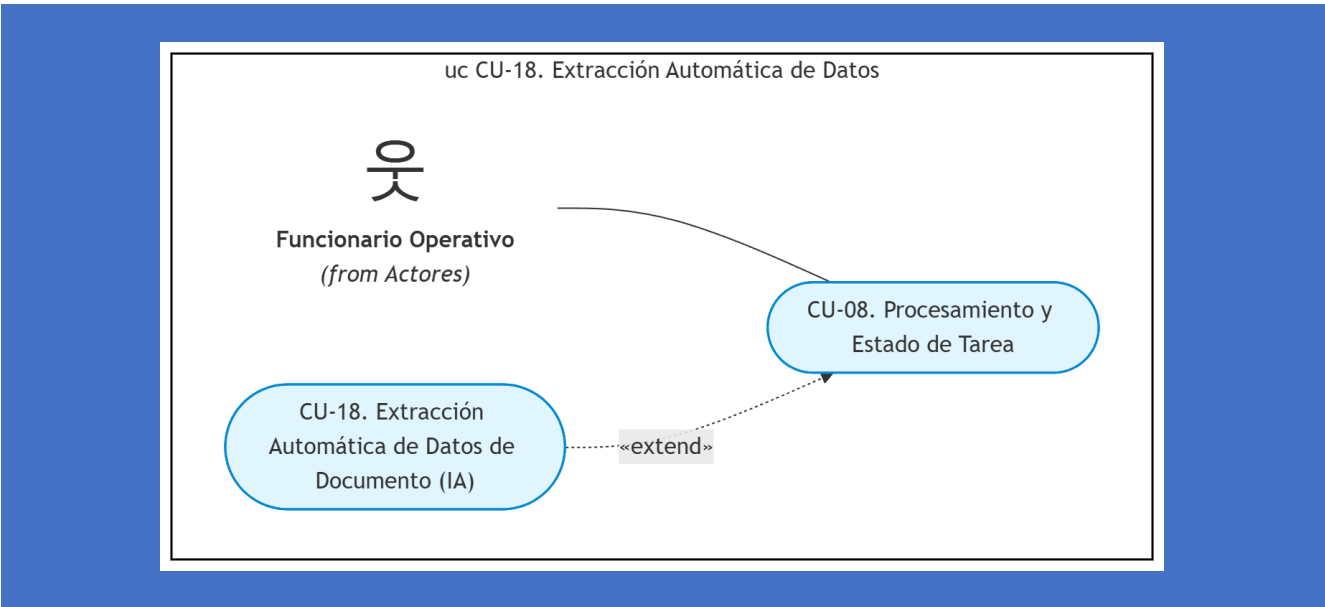
Nombre de Caso de Uso	CU-16. Acceso Móvil / PWA Offline
Propósito	Permitir que los Clientes y Funcionarios Operativos utilicen la aplicación móvil (Flutter) o web progresiva (PWA) de forma resiliente en entornos sin internet, consultando datos almacenados localmente y sincronizándolos automáticamente una vez recuperada la red.
Actores	Cliente, Funcionario Operativo
Actor Iniciador	Cliente, Funcionario Operativo
Precondición	La aplicación debe haberse descargado previamente y sincronizado por lo menos una vez estando en línea.
Flujo Principal	1. El usuario abre la aplicación móvil o PWA sin conexión activa a internet. 2. El sistema detecta el estado <i>offline</i> y desvía las consultas de red hacia el gestor de persistencia local (SQLite/IndexedDB). 3. El usuario visualiza la lista de trámites o tareas cacheadas localmente. 4. Si realiza modificaciones o redacta información, el sistema encola la petición en un búfer local (<i>Offline Storage Queue</i>). 5. Al recuperar la señal de red, un agente en segundo plano inicia el proceso de sincronización sincronizando la cola local con MongoDB.
Post Condición	El dispositivo local y el servidor central se sincronizan y unifican datos una vez recuperado el enlace de red.
Excepción	Conflicto de concurrencia al sincronizar en el backend (se notifica al usuario y se conserva la versión más reciente).

CU-17. Gestión Documental Colaborativa de Trámites



Nombre de Caso de Uso	CU-17. Gestión Documental Colaborativa de Trámites
Propósito	Facilitar a los Funcionarios la visualización, carga y edición paralela (colaborativa) de cualquier tipo de documento de soporte asociado a un trámite en ejecución, registrando autores de cambios e historial de versiones.
Actores	Funcionario Operativo
Actor Iniciador	Funcionario Operativo
Precondición	El Funcionario debe estar asignado a una tarea activa de un trámite que disponga de sección documental.
Flujo Principal	1. El Funcionario Operativo ingresa a la bandeja de archivos adjuntos del trámite. 2. El sistema recupera el repositorio de documentos vinculados al expediente actual. 3. El Funcionario edita o sube un documento nuevo en formato digital (Excel, imágenes, PDF, etc.). 4. El sistema permite el trabajo en paralelo y la sincronización colaborativa de los cambios hechos por múltiples funcionarios sobre un mismo expediente. 5. El sistema persiste el documento actualizado y registra en MongoDB la autoría, fecha, hora y el tipo de cambio efectuado en el archivo.
Post Condición	Los documentos son modificados colaborativamente y se mantiene un registro de autoría para auditoría de archivos.
Excepción	El documento se encuentra bloqueado por uso exclusivo de otro usuario (el sistema deniega la edición temporalmente).

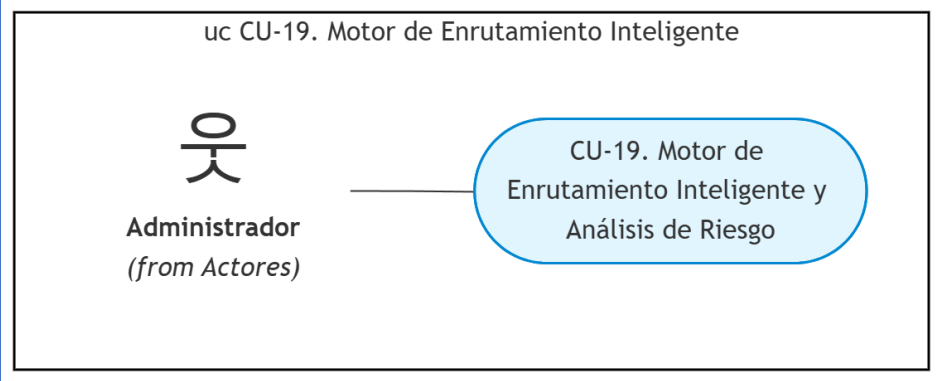
CU-18. Extracción Automática de Datos de Documento (IA)



Nombre de Caso de Uso	CU-18. Extracción Automática de Datos de Documento (IA)
Propósito	Utilizar inteligencia artificial para escanear y extraer información clave de un documento adjunto, rellenando automáticamente el formulario dinámico de la tarea para acelerar el procesamiento.
Actores	Funcionario Operativo
Actor Iniciador	Funcionario Operativo
Precondición	El Funcionario Operativo debe estar procesando una tarea (CU-08) que tenga al menos un documento digital cargado.
Flujo Principal	1. El Funcionario Operativo visualiza el documento digital adjunto en la pantalla de la tarea. 2. Hace clic en el botón "Extraer datos con IA". 3. El sistema extrae el texto del archivo y lo envía al endpoint de procesamiento de IA (/api/ai/extraer-datos) junto con el nombre del nodo. 4. El servicio de IA procesa la información y devuelve un JSON estructurado con los campos encontrados (ej: <i>observación</i>, <i>aprobado</i>, <i>monto</i>, etc.). 5. El sistema rellena automáticamente los campos vacíos del formulario dinámico con los valores sugeridos por la IA para que el funcionario los valide.
Post Condición	El formulario de la tarea queda pre-llenado a partir de los datos reales del documento de soporte.

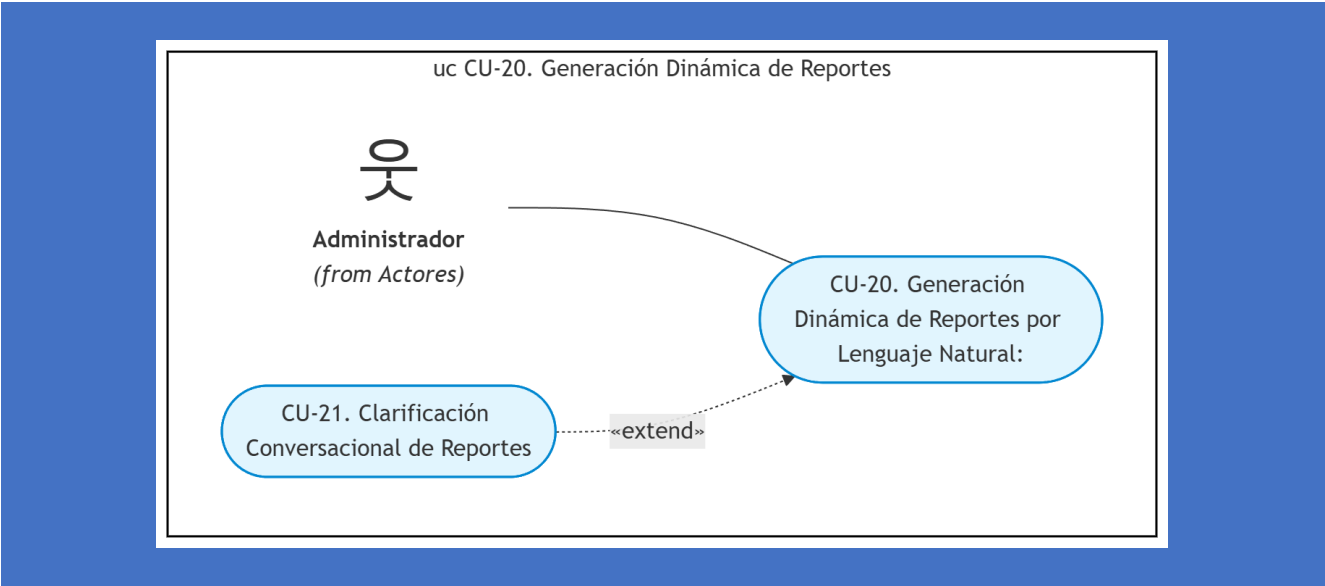
Excepción	El formato del archivo no es compatible o la IA no logra extraer datos útiles (se indica al funcionario que debe completar el formulario a mano).
------------------	---

CU-19. Motor de Enrutamiento Inteligente y Análisis de Riesgo

uc CU-19. Motor de Enrutamiento Inteligente 	
Nombre de Caso de Uso	CU-19. Motor de Enrutamiento Inteligente y Análisis de Riesgo
Propósito	Predecir de forma inteligente el nivel de riesgo de demoras en un trámite activo en base a métricas históricas de rendimiento en MongoDB y sugerir rutas de enrutamiento óptimas mediante IA.
Actores	Administrador
Actor Iniciador	Administrador
Precondición	Deben existir registros de tiempos de ejecución previos de la misma política de negocio en MongoDB.
Flujo Principal	1. El Administrador accede al monitor de trámites y selecciona un trámite activo. 2. El sistema recupera el historial de tiempos de ejecución (en horas) del nodo en el que se encuentra el trámite. 3. El backend invoca al microservicio de IA en Python (/ai/predict-risk) enviando los tiempos acumulados. 4. El servicio de IA procesa la desviación estándar, calcula el tiempo promedio real y evalúa el riesgo (Bajo, Medio, Alto) y la presencia de cuellos de botella. 5. La IA responde con la clasificación de riesgo final y un conjunto de acciones sugeridas para agilizar el trámite. 6. El Administrador visualiza la prioridad y el nivel de alerta de riesgo en pantalla.

Post Condición	El sistema provee un diagnóstico inteligente del estado y riesgo del trámite en curso.
Excepción	No hay historial de tiempos disponible (el sistema clasifica el riesgo provisionalmente como <i>Bajo</i> usando reglas heurísticas básicas).

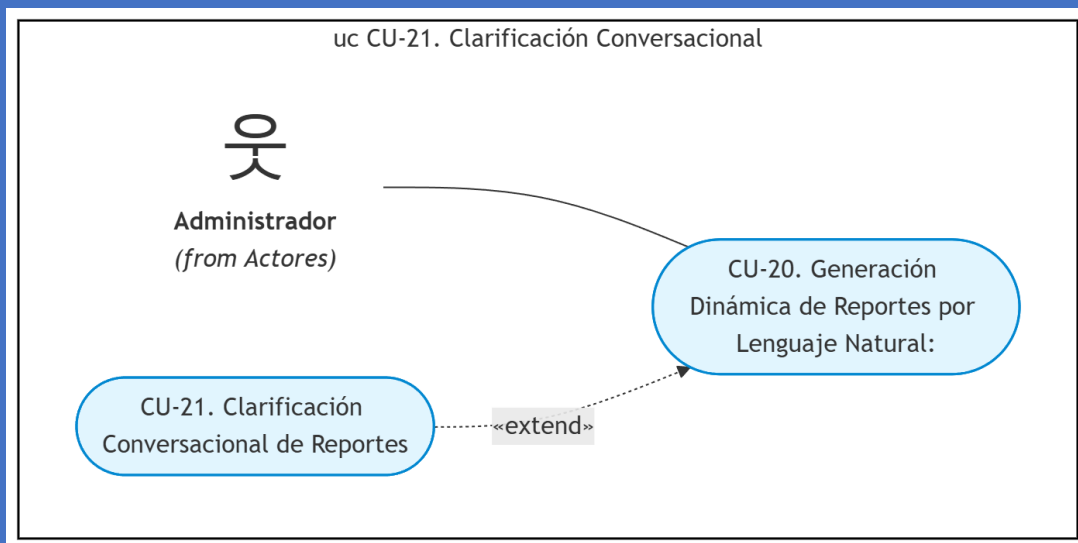
CU-20. Generación Dinámica de Reportes por Lenguaje Natural



Nombre de Caso de Uso	CU-20. Generación Dinámica de Reportes por Lenguaje Natural:
Propósito	Permitir al Administrador solicitar reportes de desempeño de los procesos institucionales mediante comandos de voz o texto libre en lenguaje natural, y presentárselos en formatos estructurados como PDF, Markdown, HTML o CSV.
Actores	Administrador
Actor Iniciador	Administrador
Precondición	El Administrador debe estar autenticado en la plataforma y contar con datos operativos cargados.
Flujo Principal	1. El Administrador accede a la sección "<i>Generador de Reportes</i>". 2. Digita o dicta su solicitud (ej: "<i>Genera un reporte del departamento Legal de los últimos 3 meses en formato PDF</i>"). 3. El sistema envía la instrucción al endpoint de IA (<i>/api/ai/generate-report</i>). 4. La IA analiza el texto y extrae las 3 variables de control: <i>departamento</i>, <i>rango_tiempo</i> y <i>formato de salida</i>. 5. Si las 3 variables están presentes, el sistema consulta las métricas

	de rendimiento en MongoDB. 6. La IA redacta el contenido ejecutivo estructurado en base a los datos reales y el sistema lo despliega formateado en pantalla para su descarga.
Post Condición	El Administrador obtiene un reporte formal detallado estructurado en el formato solicitado.
Excepción	Falta al menos una de las variables obligatorias en la solicitud (se activa el caso de uso extendido CU-21 para entablar un diálogo conversacional de aclaración).

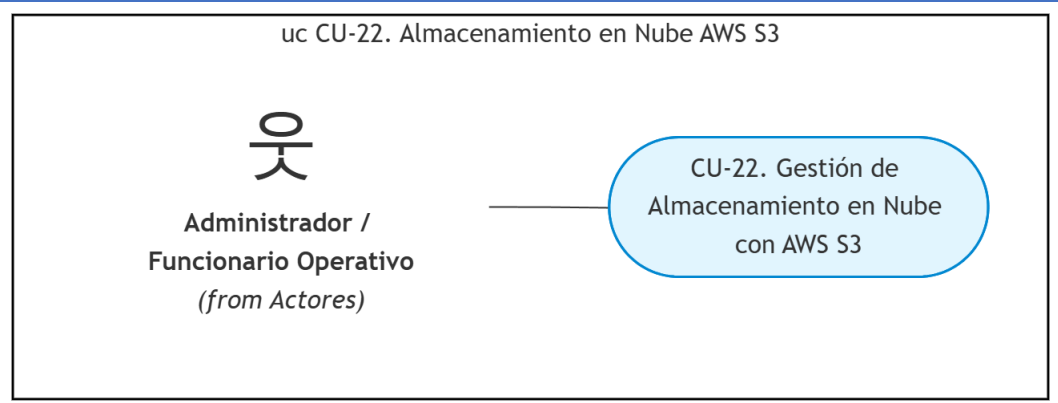
CU-21. Clarificación Conversacional de Reportes



Nombre de Caso de Uso	CU-21. Clarificación Conversacional de Reportes
Propósito	Entablar un diálogo conversacional con el Administrador para solicitar las variables obligatorias faltantes (departamento, período o formato de exportación) cuando la petición de reporte no sea del todo comprensible o esté incompleta.
Actores	Administrador
Actor Iniciador	Sistema (Microservicio de IA en Python)
Precondición	El Administrador debió iniciar la generación de un reporte (CU-20) omitiendo parámetros clave.
Flujo Principal	1. La IA del microservicio de Python detecta que la petición carece de alguno de los tres parámetros obligatorios. 2. El sistema devuelve una bandera de información faltante

	(falta_informacion: true) junto con una lista de las variables ausentes. 3. El sistema muestra de forma conversacional en la UI preguntas orientadas (ej: " <i>¿De qué departamento deseas el reporte?</i> "). 4. El Administrador responde la pregunta en el chat. 5. El sistema captura la variable y reenvía el flujo al caso de uso principal CU-20 para concluir la generación.
Post Condición	Se recopilan las variables faltantes y se permite la continuidad de la generación del reporte.
Excepción	El Administrador decide cancelar la consulta conversacional (se cierra el diálogo y no se procesa el reporte).

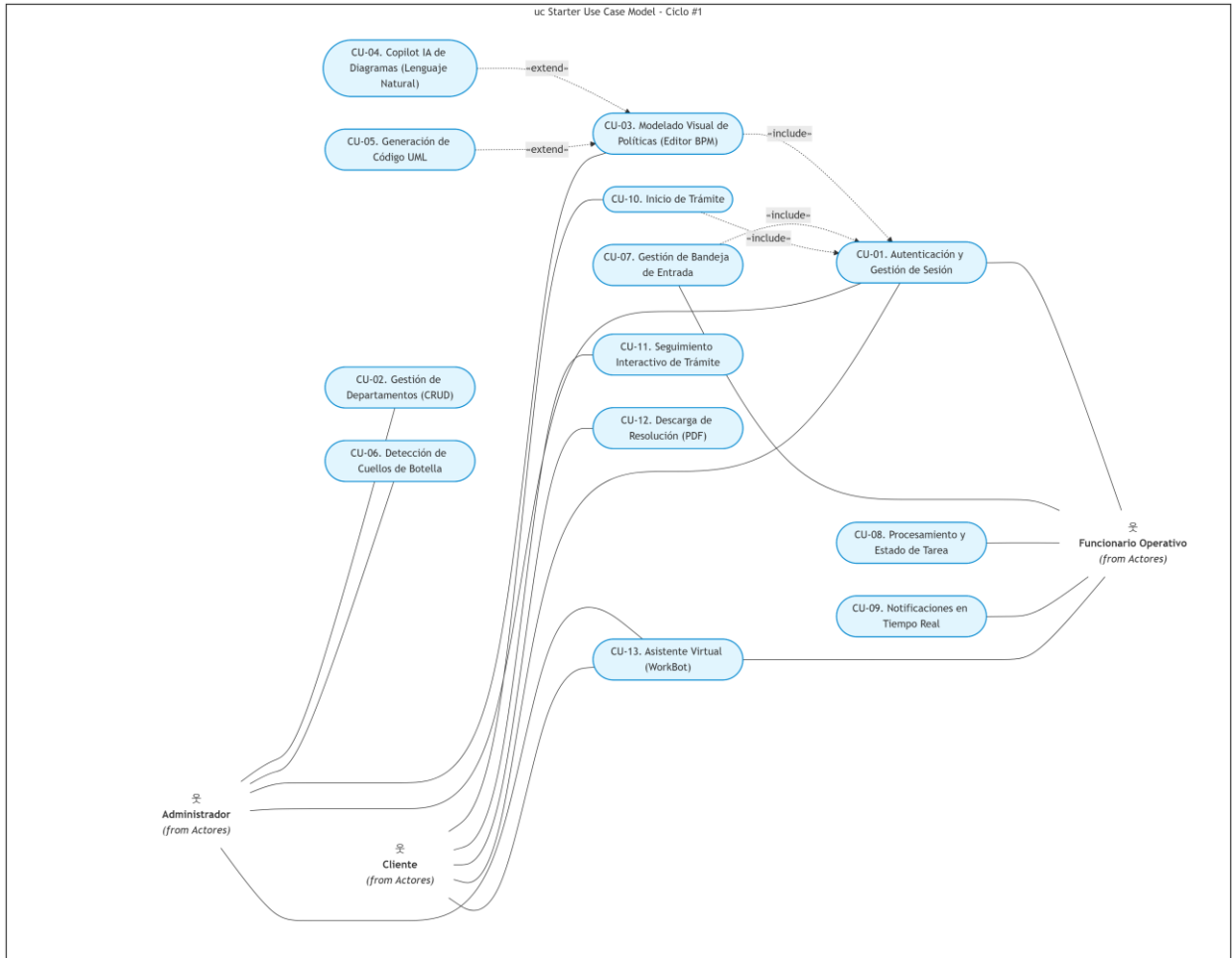
CU-22. Gestión de Almacenamiento en Nube con AWS S3

uc CU-22. Almacenamiento en Nube AWS S3 	
Nombre de Caso de Uso	CU-22. Gestión de Almacenamiento en Nube con AWS S3
Propósito	Almacenar los archivos digitales de los trámites directamente en contenedores de AWS S3, gestionando cargas y descargas seguras del lado del cliente mediante URLs prefirmadas de corta duración.
Actores	Administrador, Funcionario Operativo
Actor Iniciador	Administrador, Funcionario Operativo
Precondición	La plataforma de backend debe tener configurado el servicio S3StorageService con credenciales de AWS válidas.
Flujo Principal	1. El usuario inicia la subida de un documento desde la interfaz web o móvil. 2. El frontend solicita al backend de Spring Boot la generación de

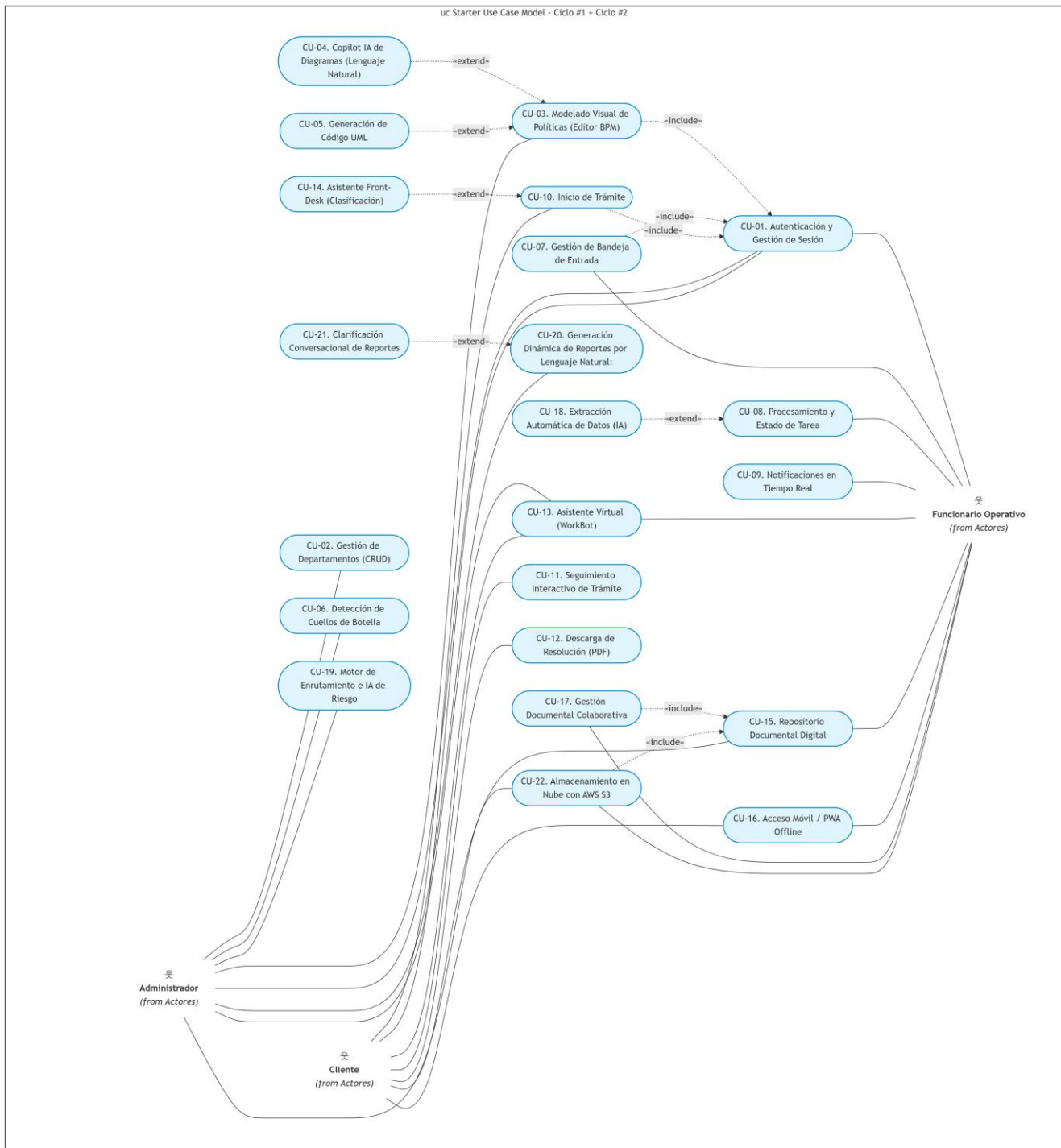
	una URL prefirmada (Presigned Upload URL). 3. El backend calcula y solicita a la API de AWS la URL firmada temporal de subida (válida por 15 minutos) y la retorna. 4. El frontend sube directamente el archivo binario mediante una petición HTTP PUT al bucket de S3 sin sobrecargar el backend. 5. Finalizada la carga, el frontend registra los datos de referencia en MongoDB. 6. Al solicitar la descarga, el backend genera una URL de descarga segura temporal (válida por 60 minutos).
Post Condición	El archivo físico queda almacenado de forma segura en la nube y su enlace lógico registrado en base de datos.
Excepción	Falla de conexión a los servicios de AWS S3 (se activa automáticamente el simulador local guardando los archivos

2.1.1.5 Estructura de modelo de caso de uso

Ciclo 1



Ciclo 2



2.2. Flujo de Trabajo: Análisis

2.2.1 Análisis de Arquitectura

2.2.1.1 Identificación de Paquetes

Paquete 1: Gestión de Acceso y Estructura Organizacional

Centraliza los servicios de seguridad, autenticación y control de acceso mediante tokens JWT para los roles del sistema (Administrador, Funcionario y Cliente). Administra además el esquema estructural de la institución a través del CRUD de departamentos, sirviendo como base lógica para la distribución de responsabilidades en los carriles (swimlanes) del motor de flujos.

Paquete 2: Modelado de Procesos y Optimización de Workflows

Agrupar las herramientas de diseño gráfico y optimización de las políticas de negocio del sistema. Gobierna el editor visual interactivo basado en la semántica de grafos de UML/BPM y coordina los servicios de asistencia inteligente encargados de la generación de diagramas por lenguaje natural, el análisis de tiempos de ejecución y la detección automatizada de cuellos de botella.

Paquete 3: Ejecución de Trámites y Operaciones en Tiempo Real

Encargado de la orquestación transaccional y del ciclo de vida operativo de cada trámite. Controla la persistencia de estados de las tareas, la bandeja semafórica de los funcionarios, el seguimiento de expedientes, el chatbot WorkBot y la emisión de alertas asíncronas por WebSockets. Incorpora el soporte de sincronización y almacenamiento local (SQLite/Hive) para la resiliencia Offline-First en la aplicación móvil Flutter.

Paquete 4: Gestión Documental Digital y Almacenamiento en la Nube

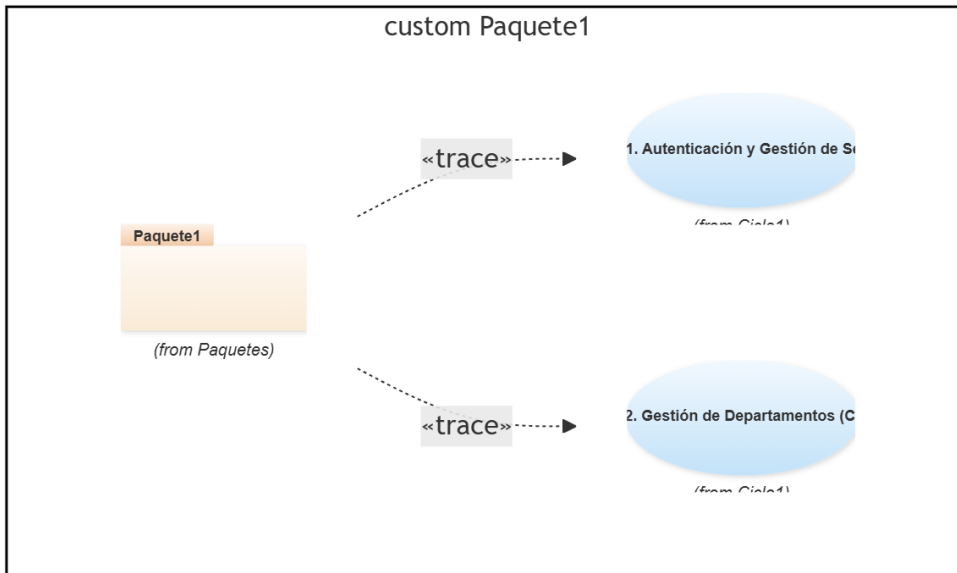
Sustenta la persistencia binaria y la estructura documental de los expedientes bajo una política de cero papel. Gestiona la transferencia segura y directa de archivos heterogéneos hacia la nube mediante un puente con Amazon S3 utilizando Presigned URLs. Regula los mecanismos de concurrencia para la edición colaborativa síncrona y almacena las pistas de auditoría y metadatos.

Paquete 5: Inteligencia Analítica, Enrutamiento Predictivo y Reportes NLP

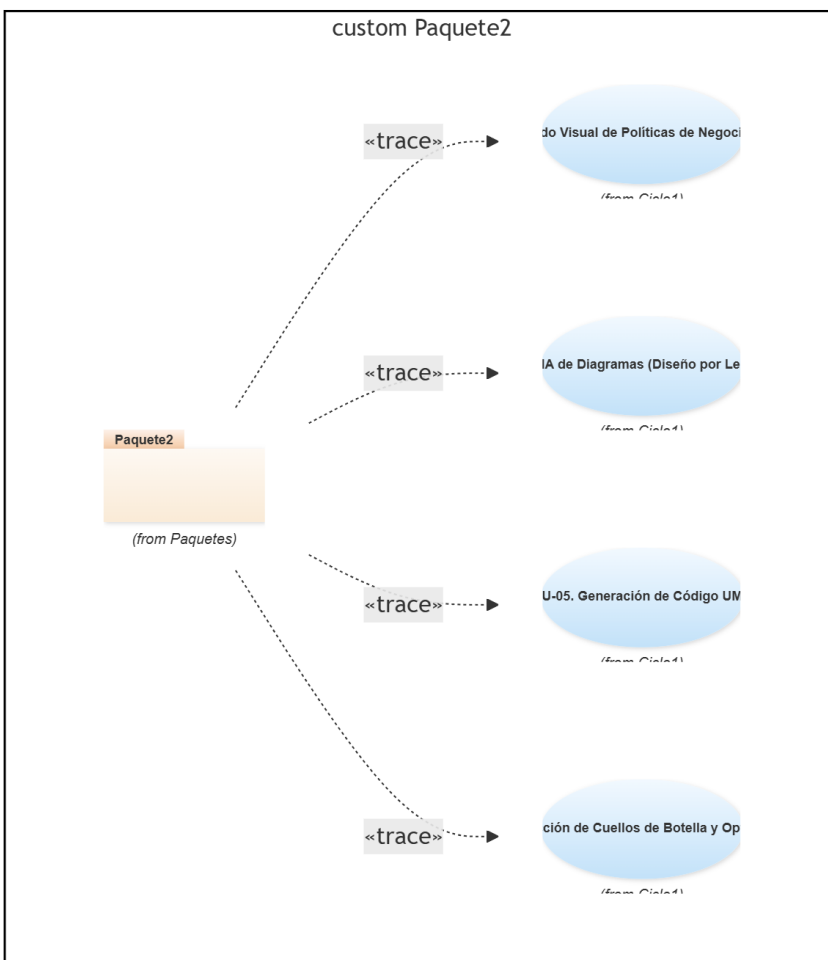
Centraliza los servicios cognitivos avanzados de Inteligencia Artificial implementados en el microservicio en Python (FastAPI). Integra el agente Front-Desk de clasificación automatizada de solicitudes, los modelos de Deep Learning para extracción de datos en documentos y el algoritmo de análisis predictivo de riesgo de demoras. Asimismo, procesa comandos de voz para la generación dinámica de reportes gerenciales estructurados a través de procesamiento de lenguaje natural (NLP).

2.2.1.2 Relación de Paquetes y Casos de Uso

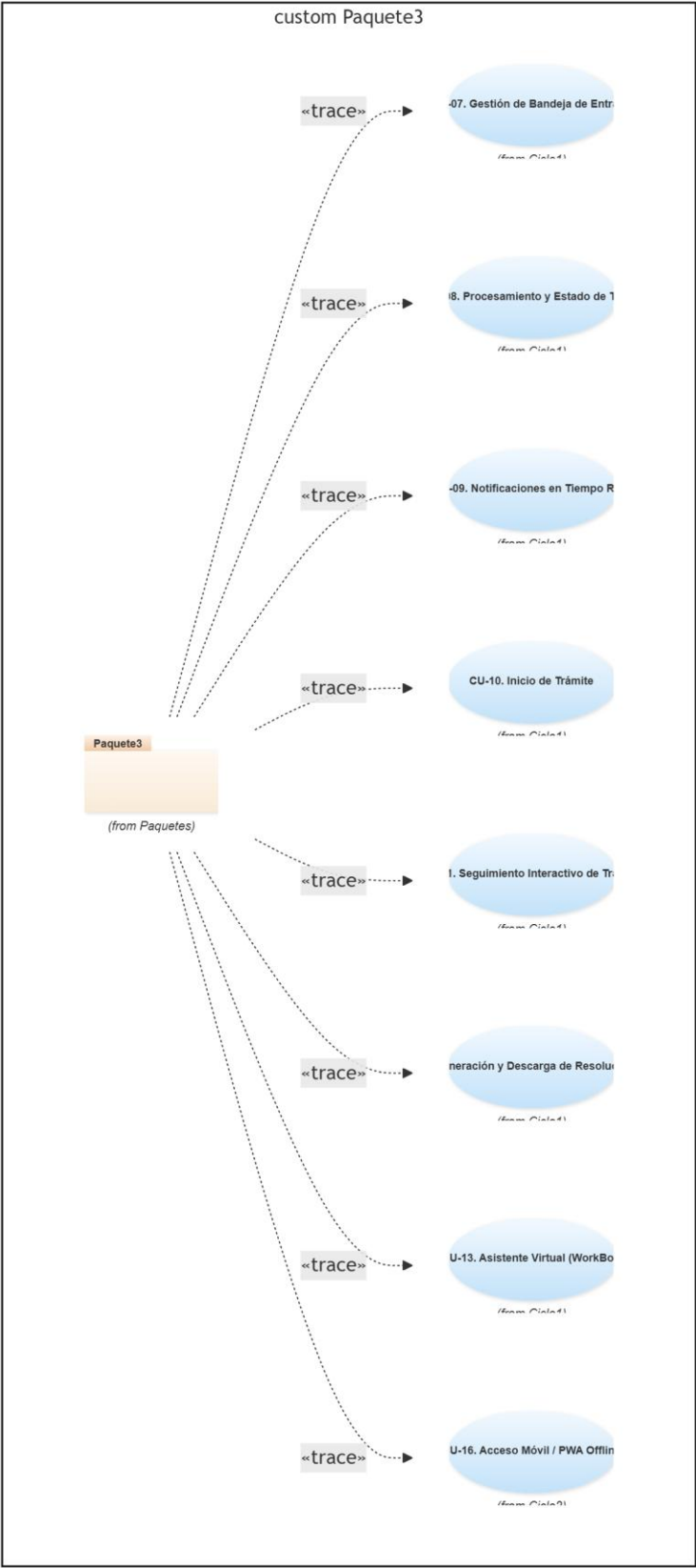
Paquete 1: Gestión de Acceso y Estructura Organizacional



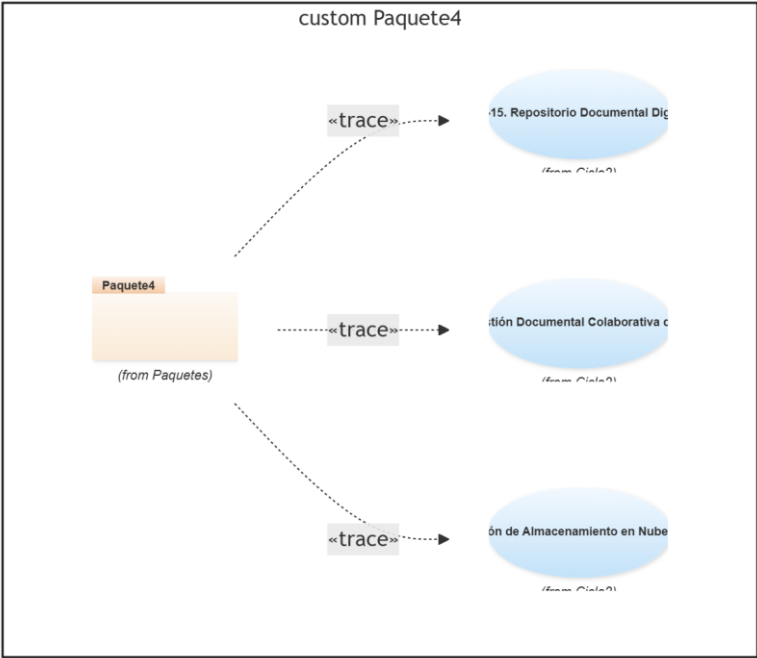
Paquete 2: Modelado de Procesos y Optimización de Workflows



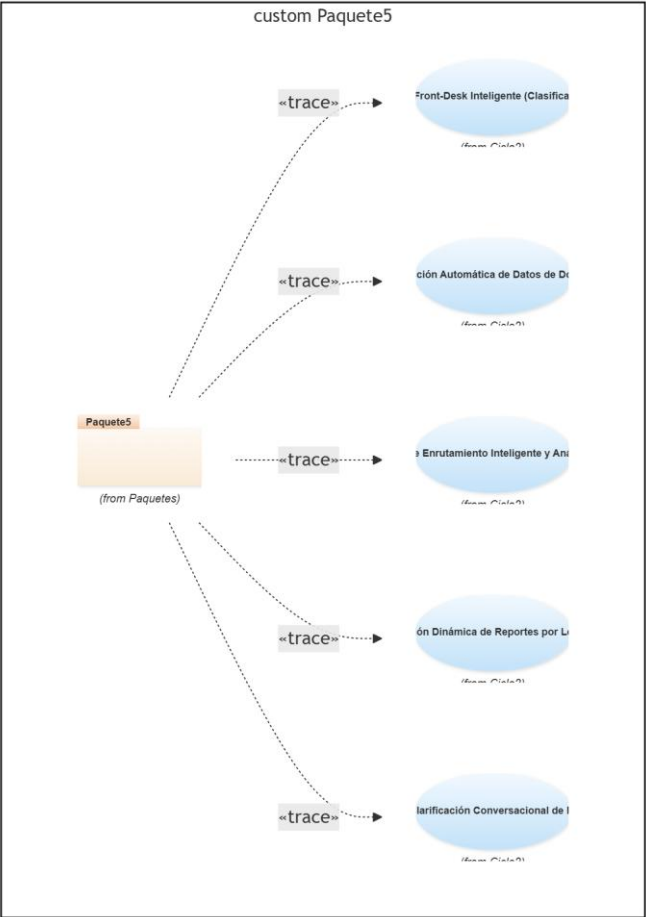
Paquete 3: Ejecución de Trámites y Operaciones en Tiempo Real



Paquete 4: Gestión Documental Digital y Almacenamiento en la Nube

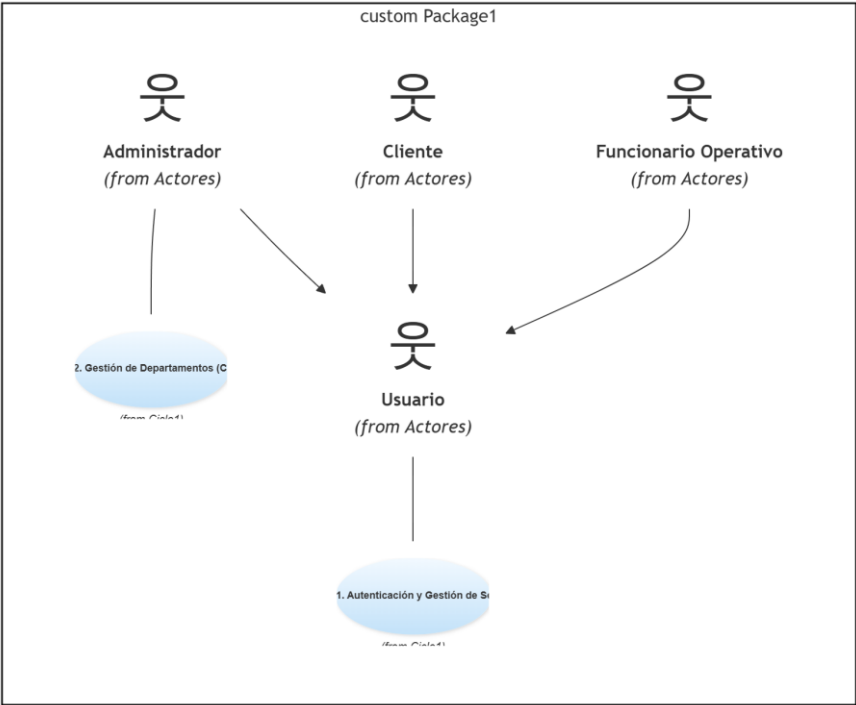


Paquete 5: Inteligencia Analítica, Enrutamiento Predictivo y Reportes NLP

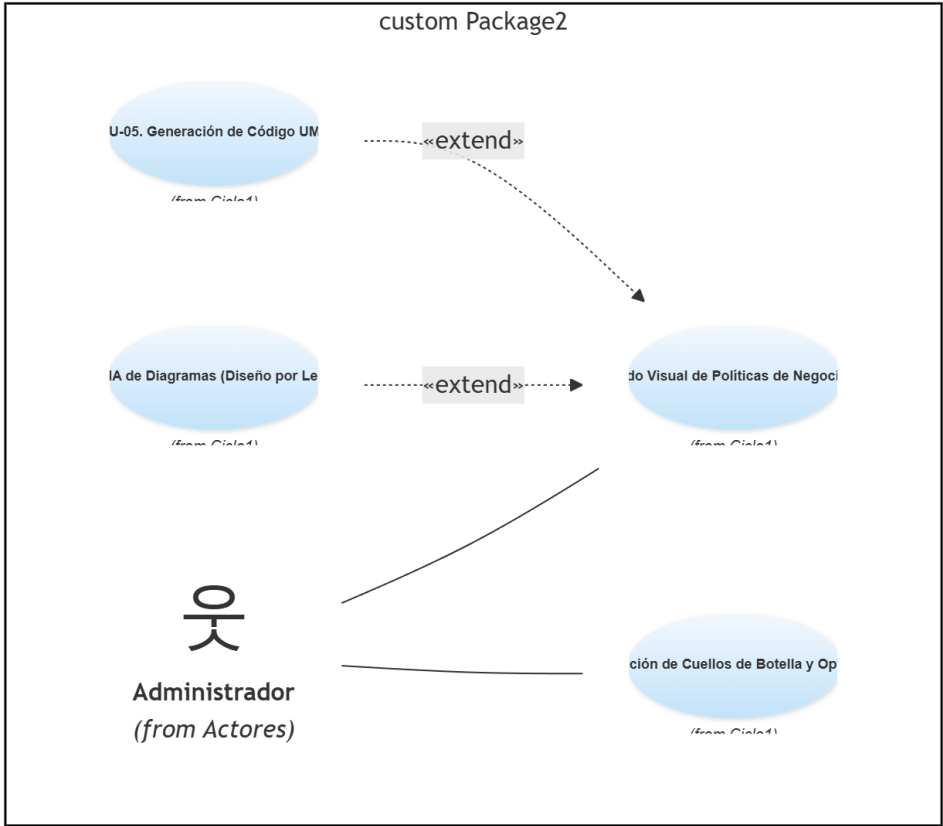


2.2.1.3 Vista de Paquetes

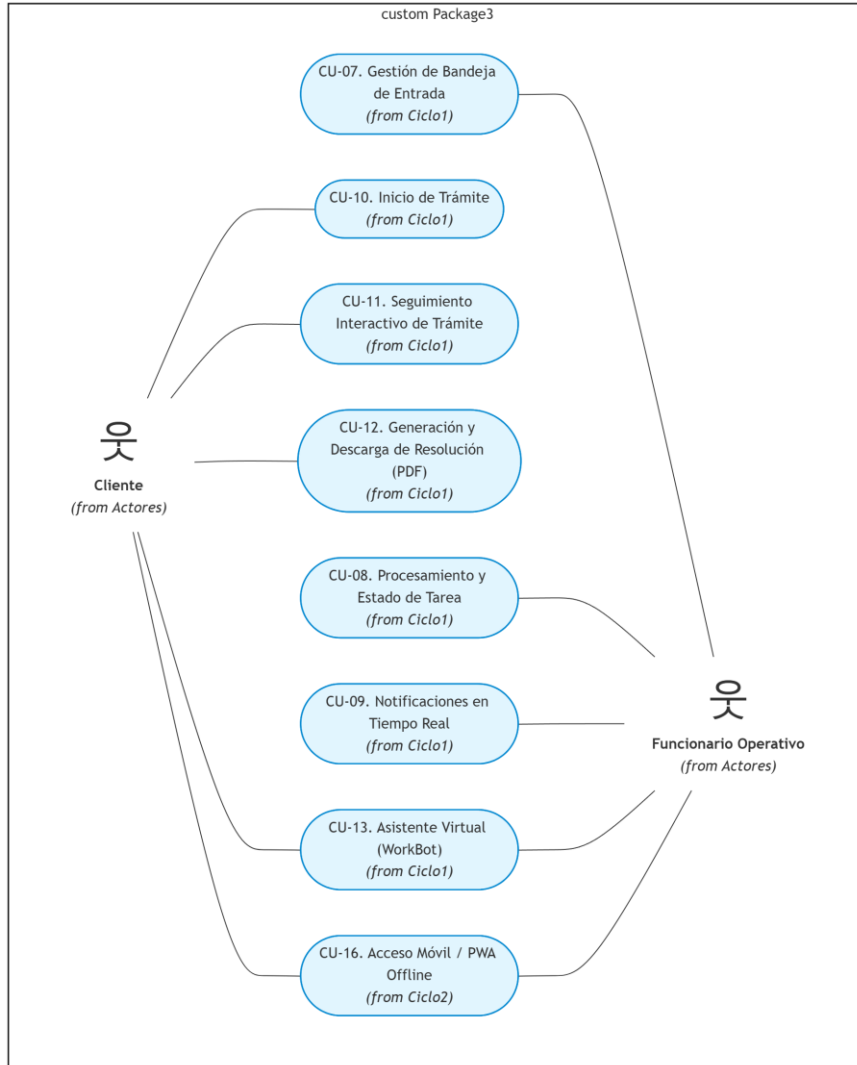
Paquete 1: Gestión de Acceso y Estructura Organizacional



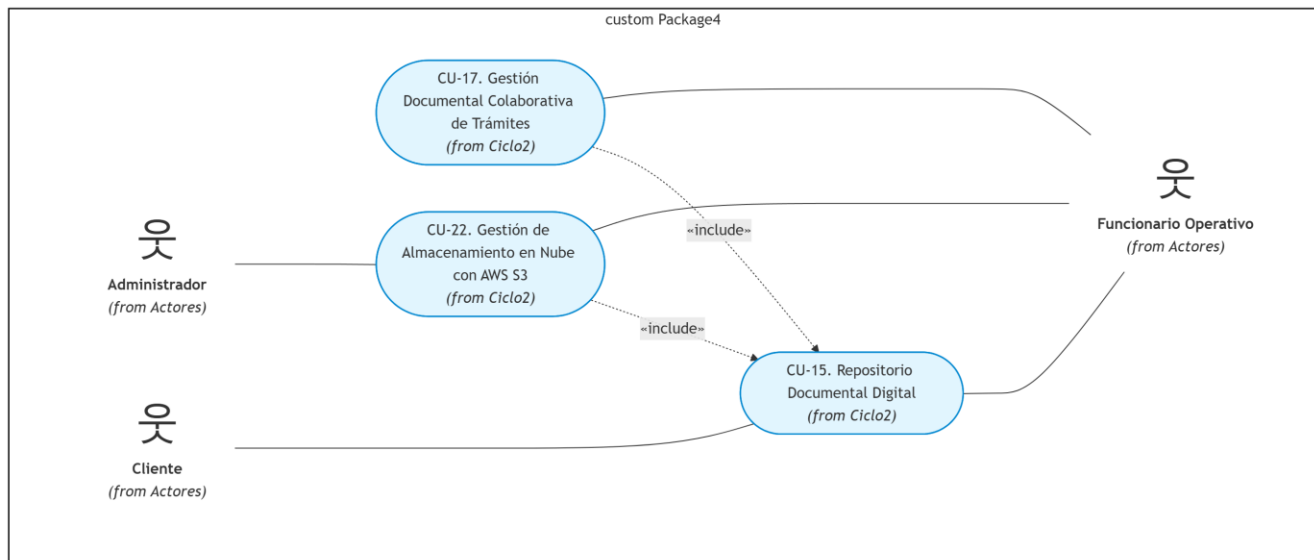
Paquete 2: Modelado de Procesos y Optimización de Workflows



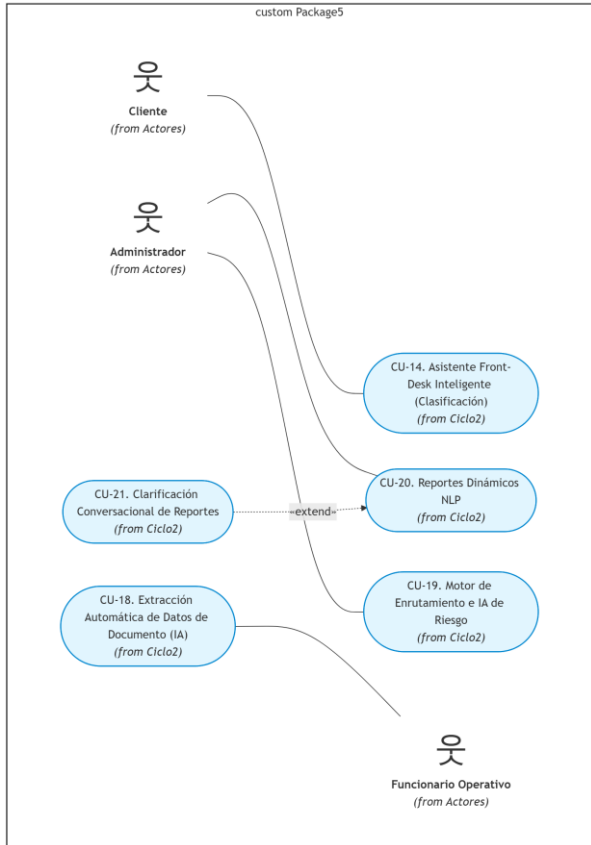
Paquete 3: Ejecución de Trámites y Operaciones en Tiempo Real



Paquete 4: Gestión Documental Digital y Almacenamiento en la Nube



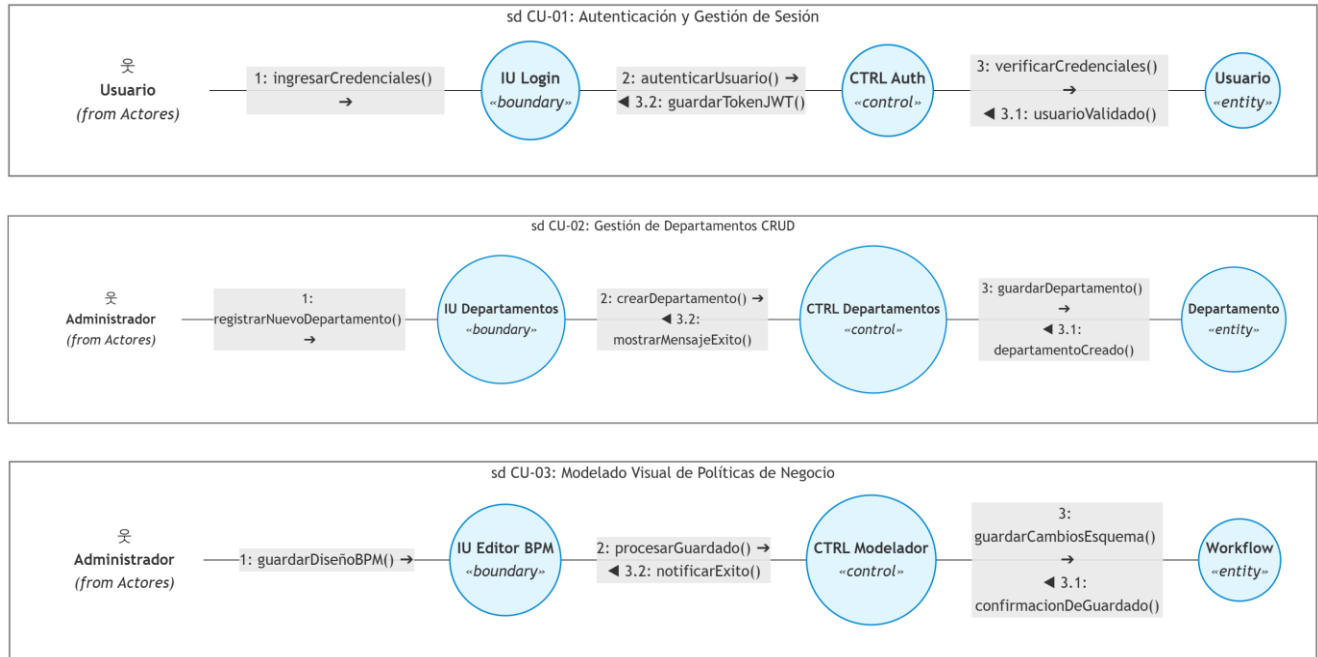
Paquete 5: Inteligencia Analítica, Enrutamiento Predictivo y Reportes NLP

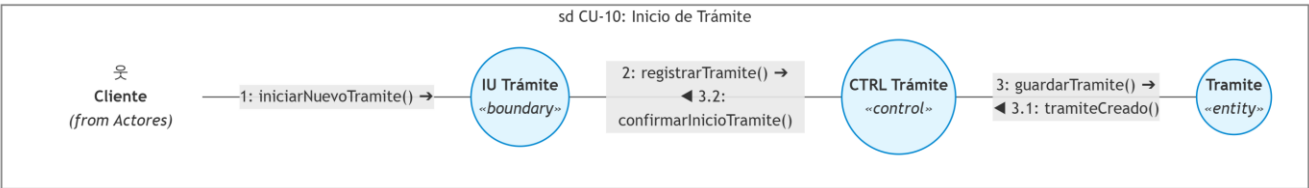
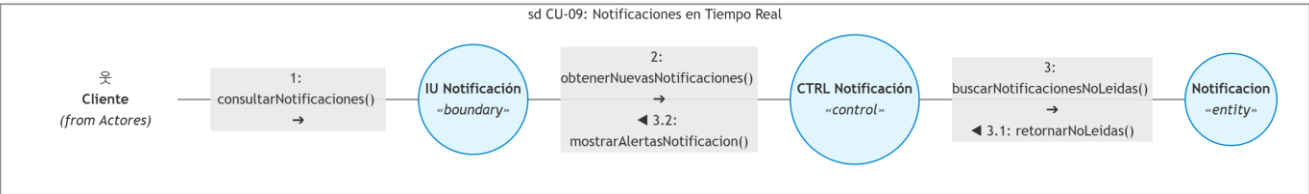
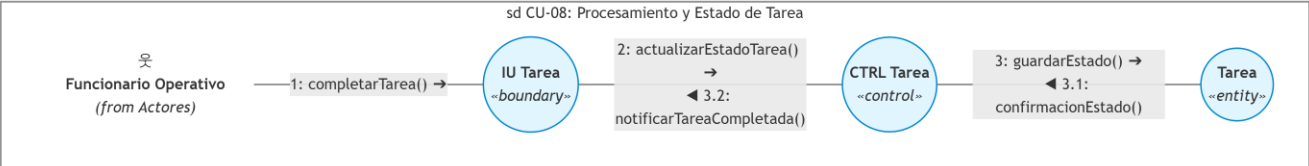
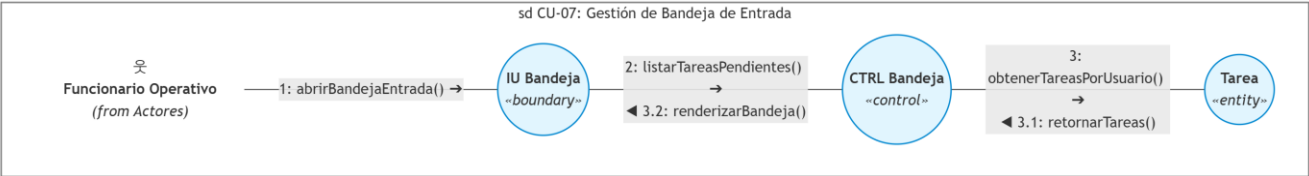
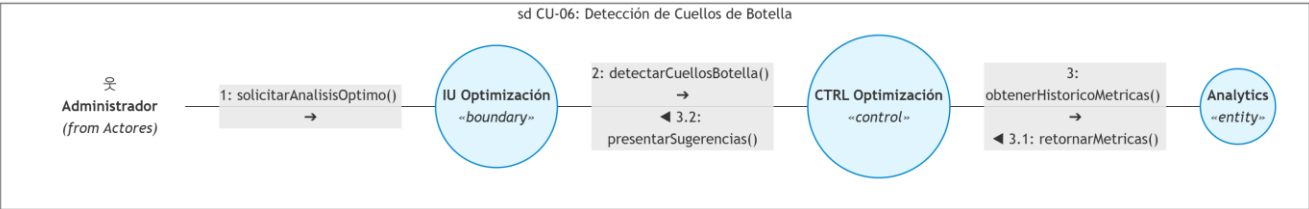
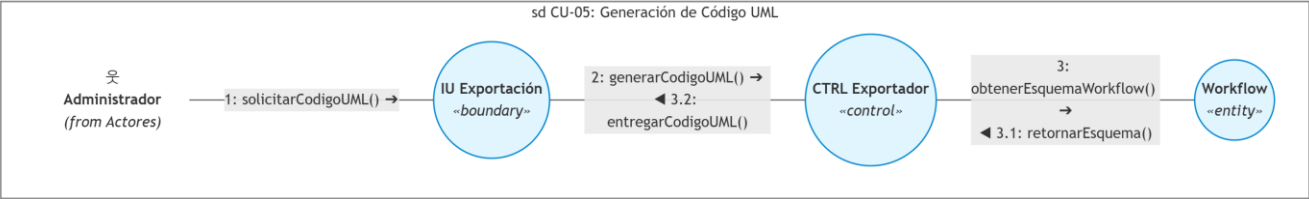
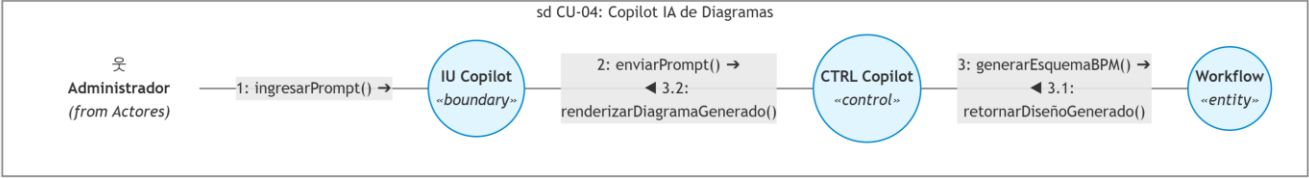


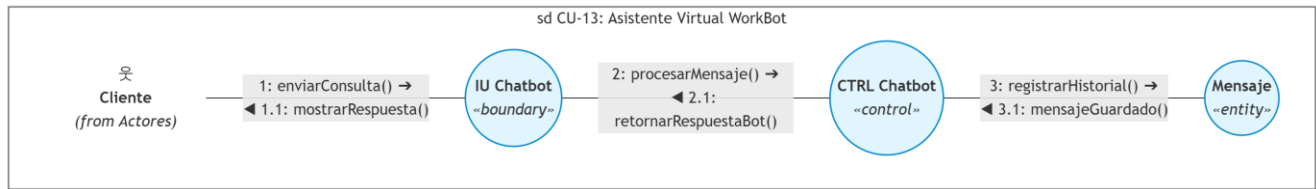
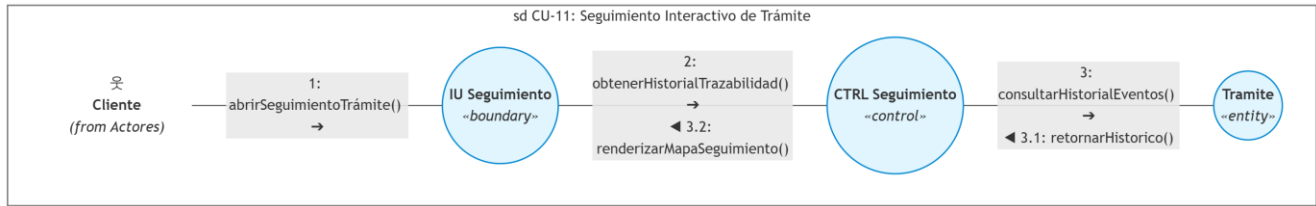
2.2.1.4 Análisis de Casos de Uso

2.2.1.4.1 Diagramas de Comunicación

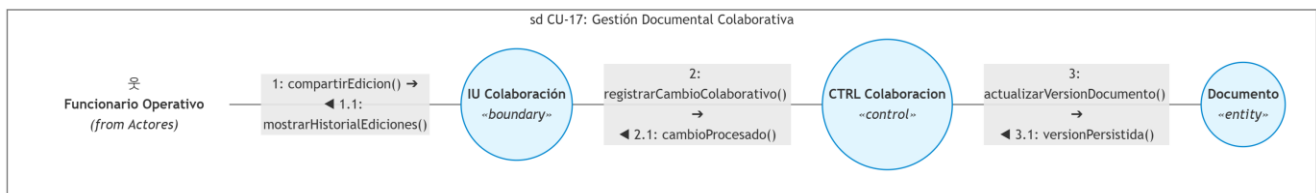
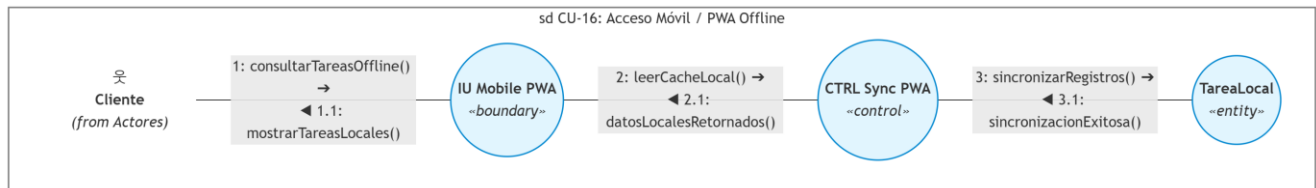
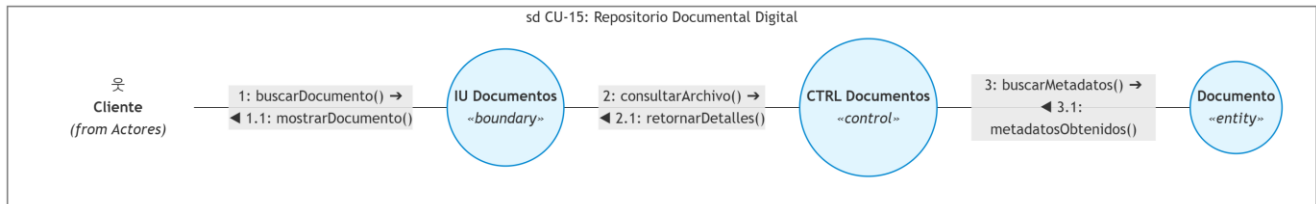
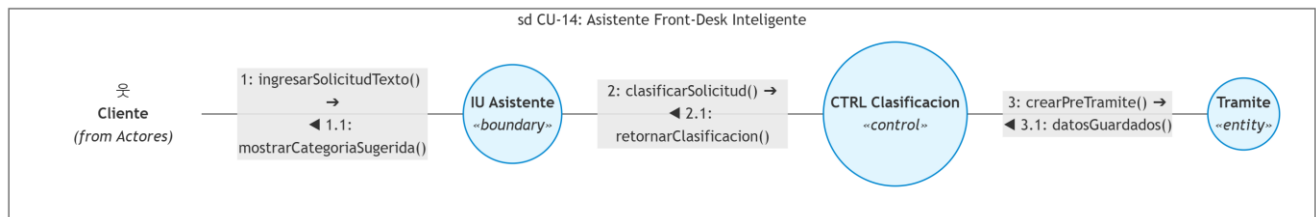
Ciclo 1

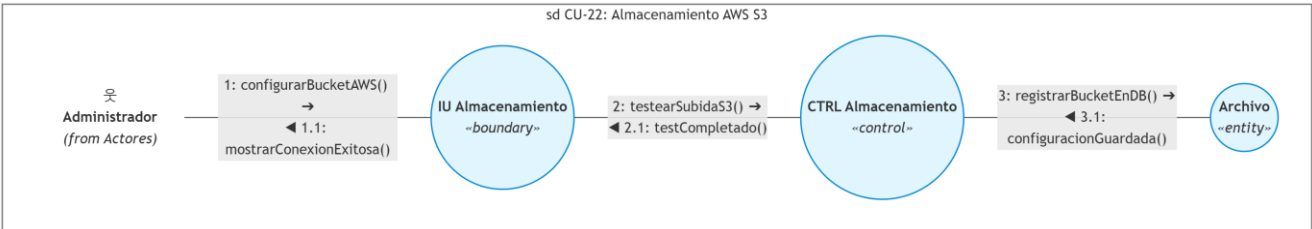
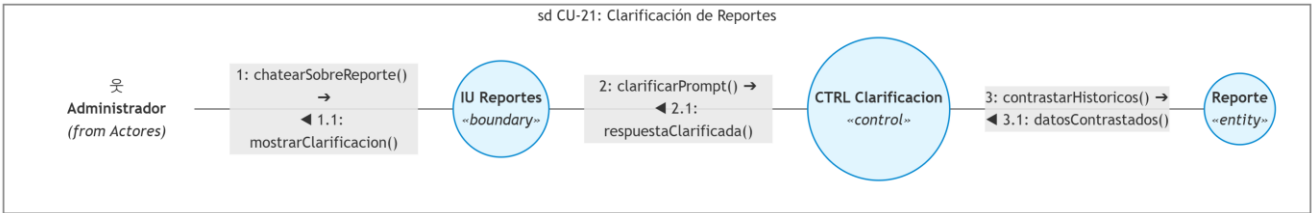
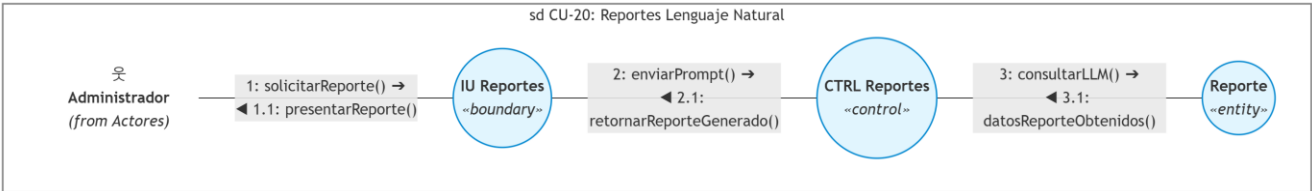
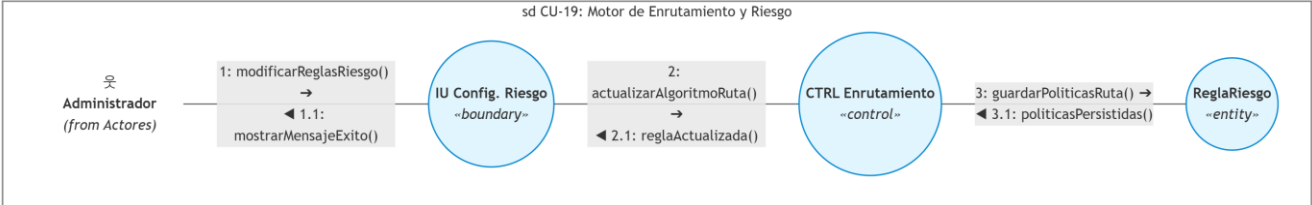
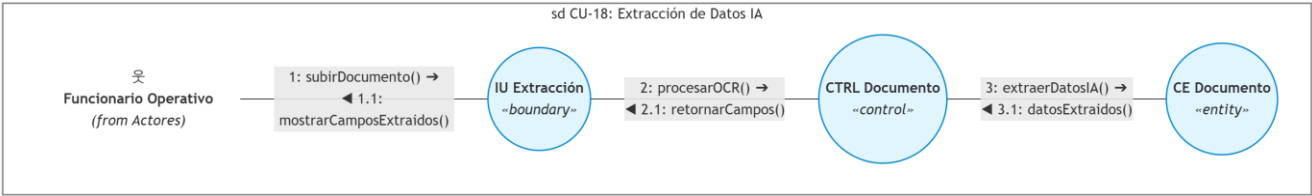






Ciclo 2

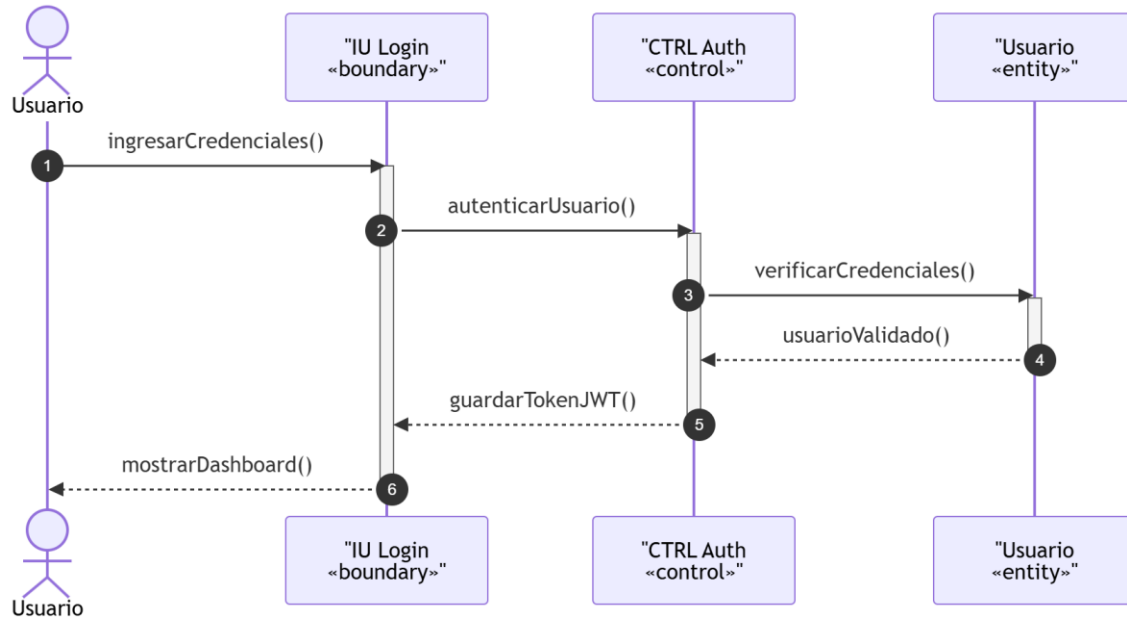




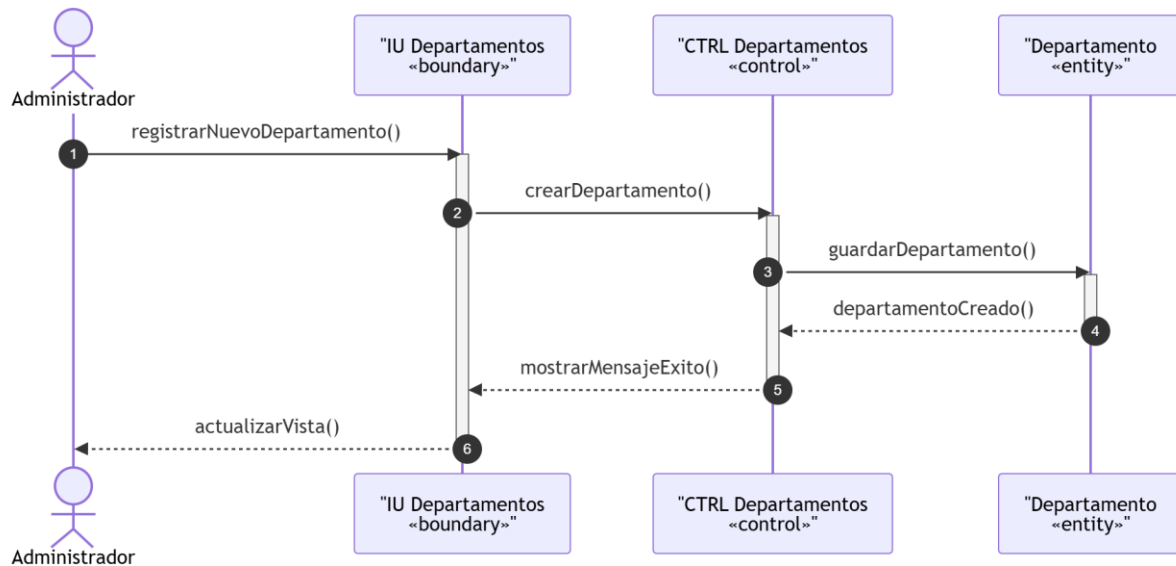
2.2.1.4.2 Diagrama de Secuencia

Ciclo 1

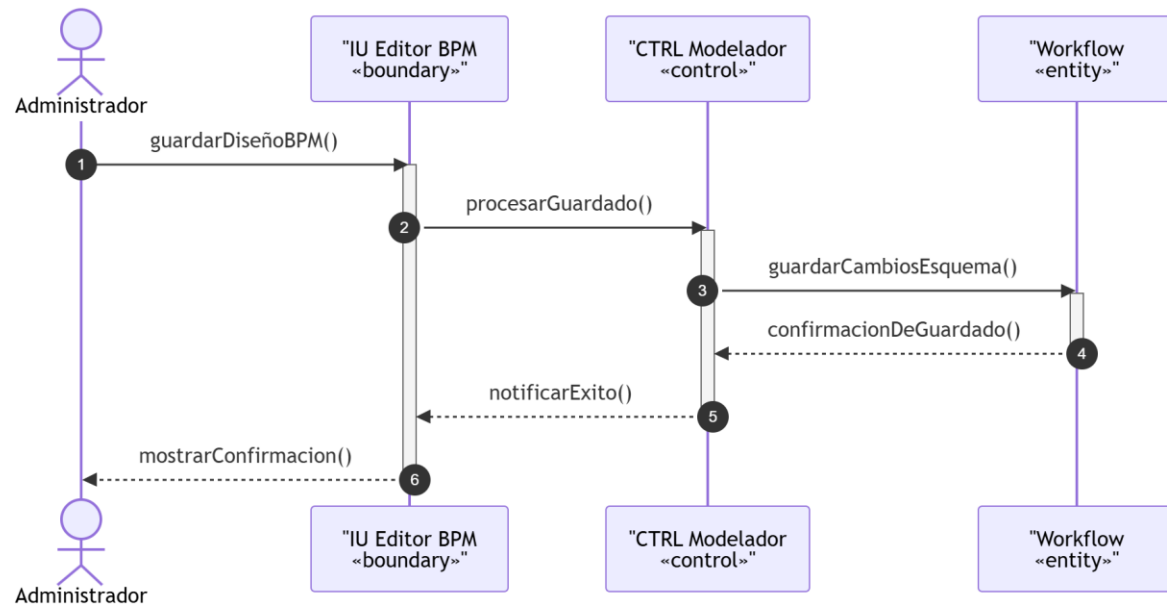
CU-01. Autenticación y Gestión de Sesión



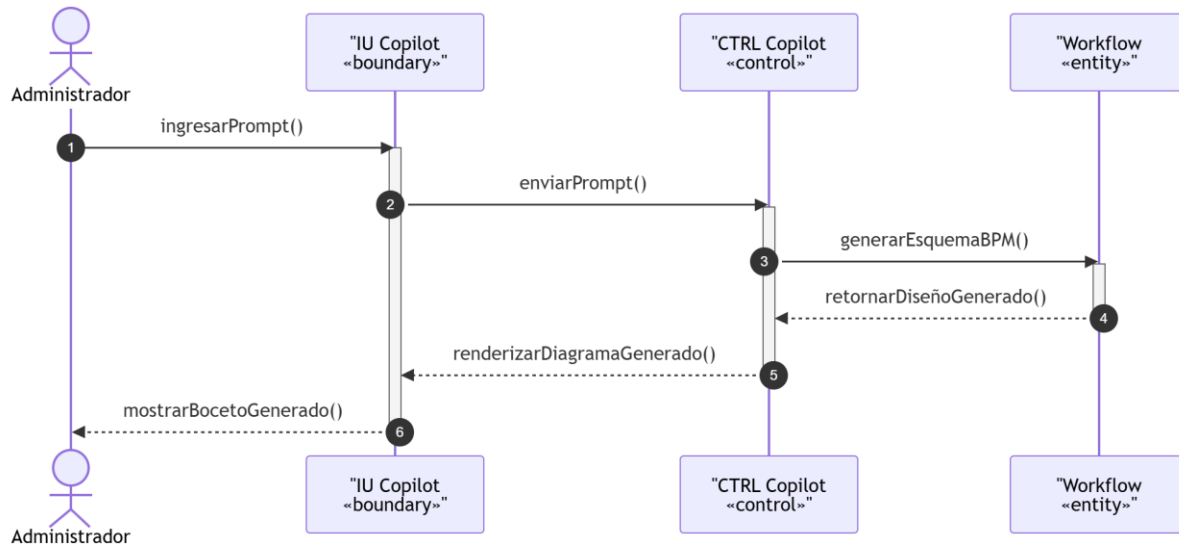
CU-02. Gestión de Departamentos (CRUD)



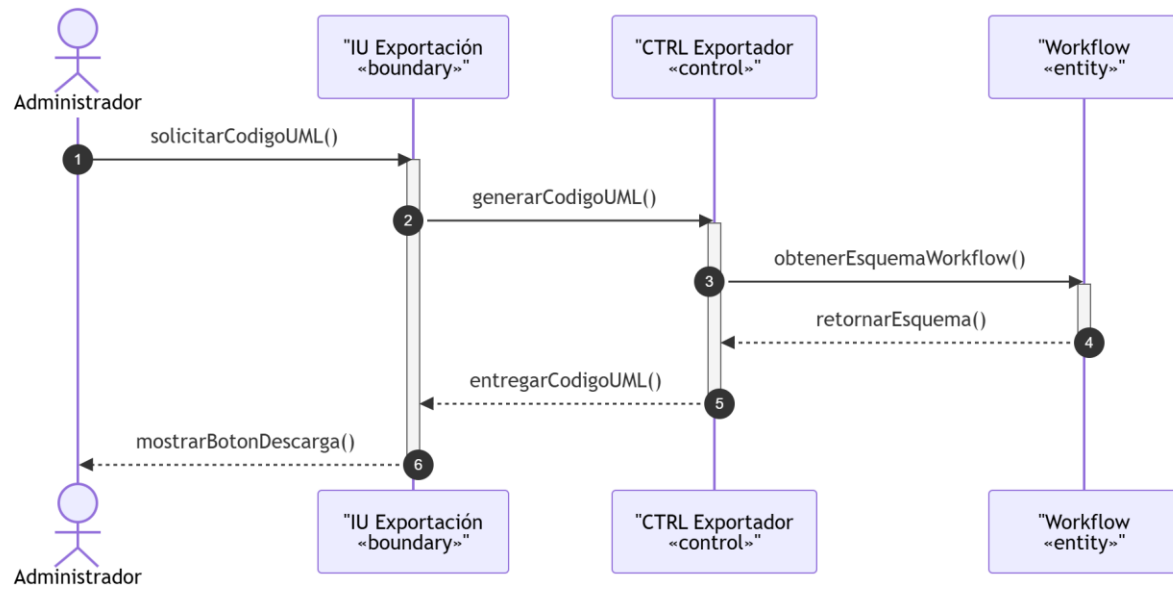
CU-03. Modelado Visual de Políticas de Negocio (Editor BPM)



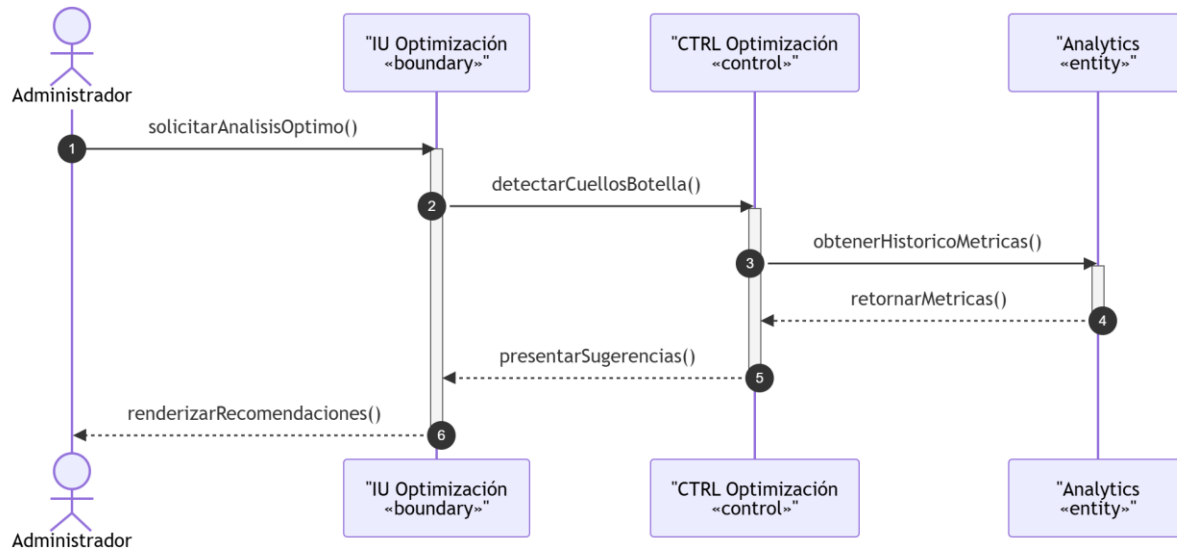
CU-04. Copilot IA de Diagramas (Diseño por Lenguaje Natural)



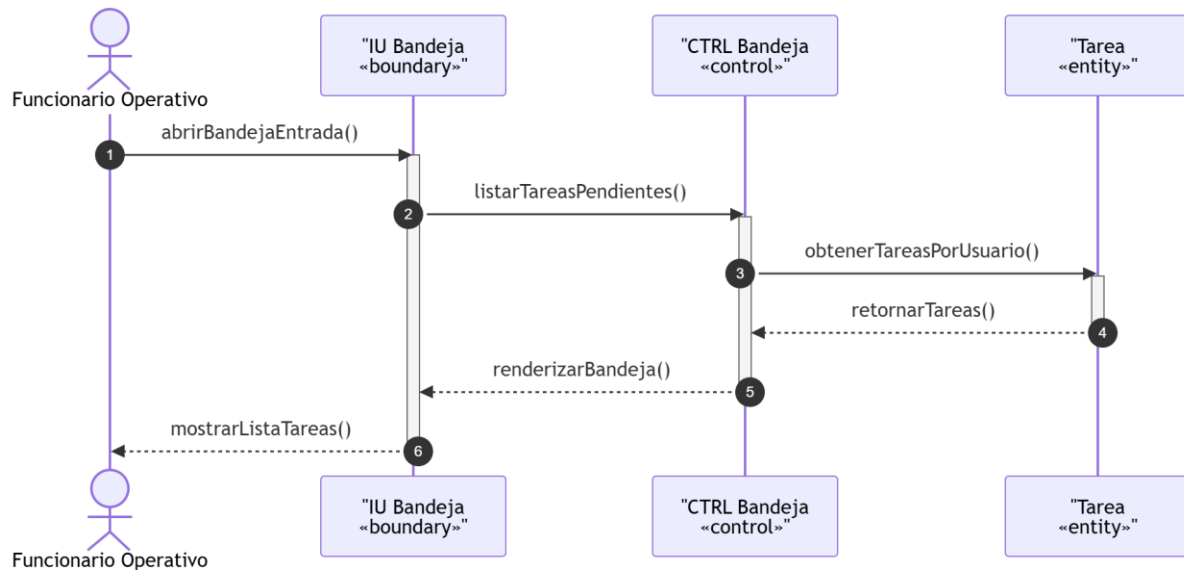
CU-05. Generación de Código UML



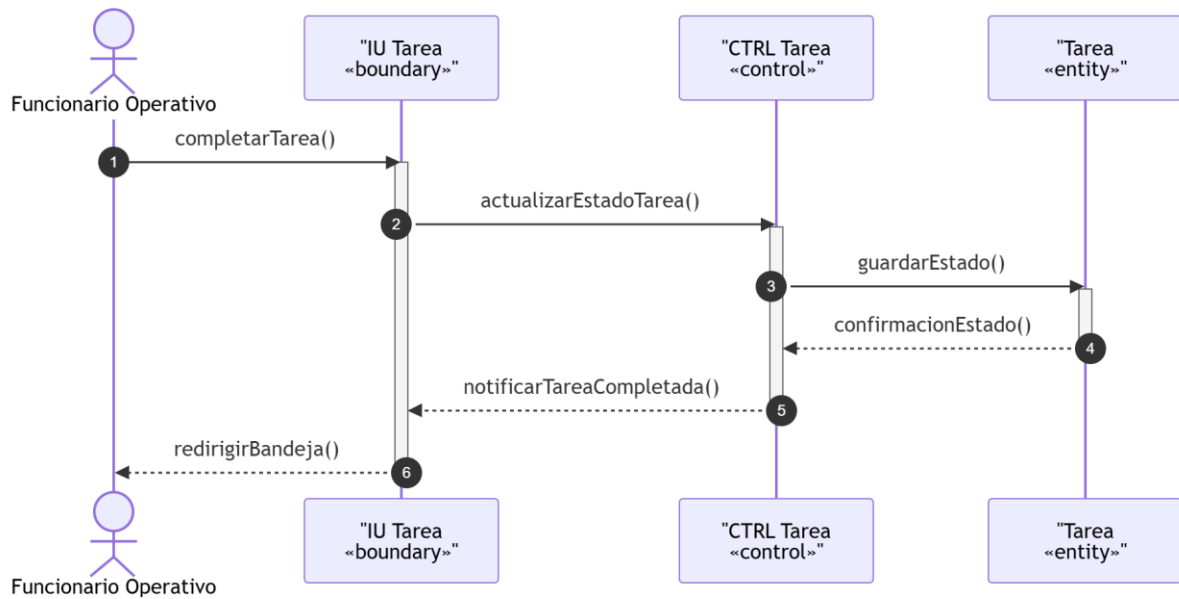
CU-06. Detección de Cuellos de Botella y Optimización IA



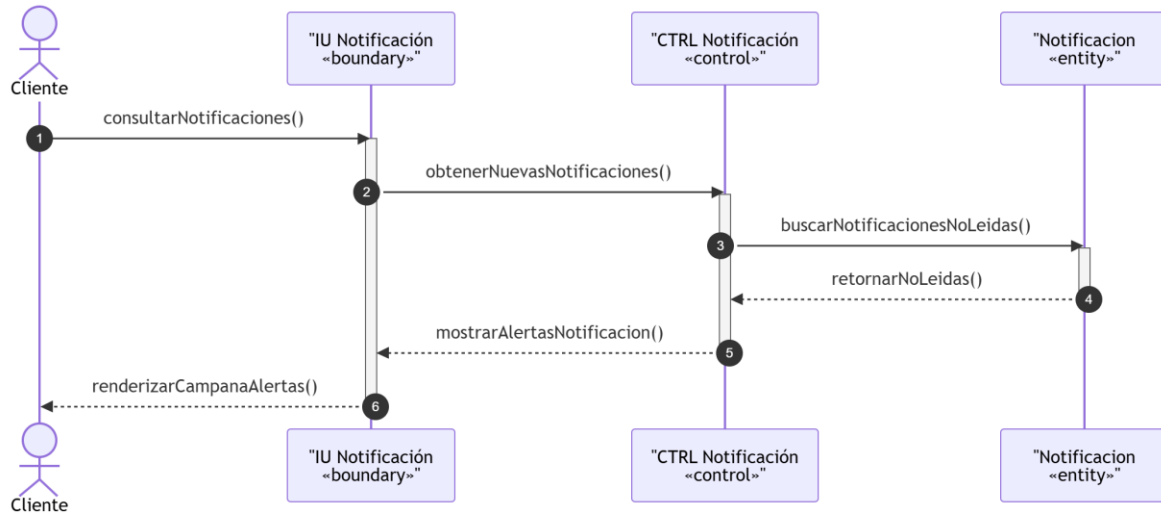
CU-07. Gestión de Bandeja de Entrada



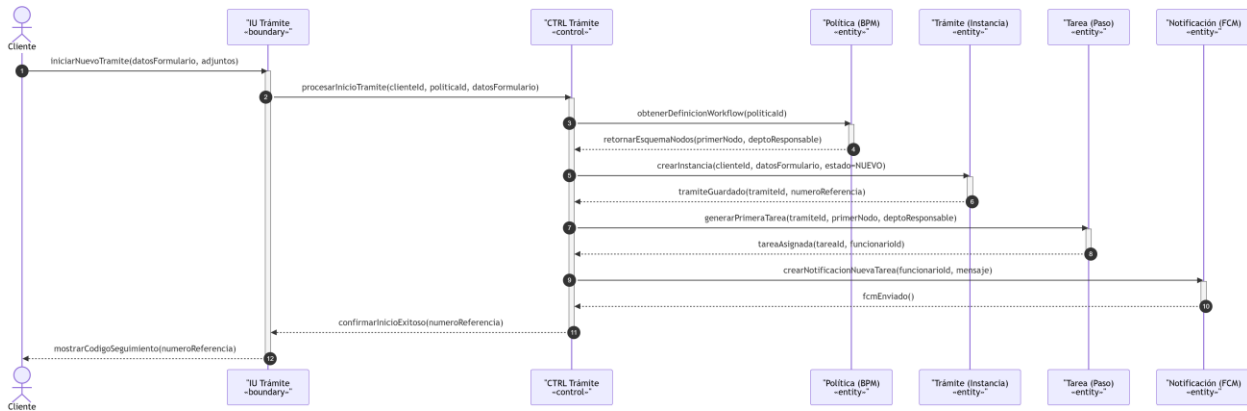
CU-08. Procesamiento y Estado de Tarea



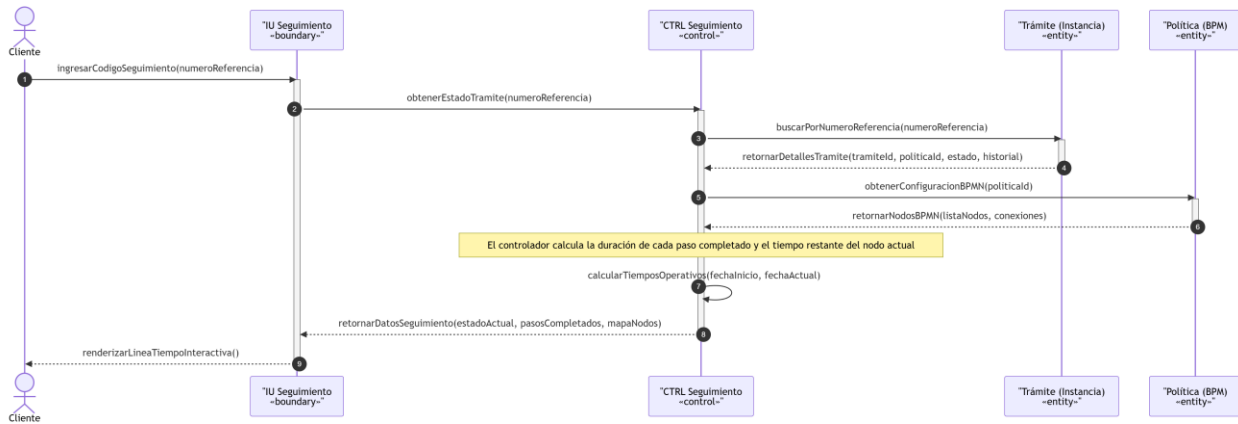
CU-09. Notificaciones en Tiempo Real



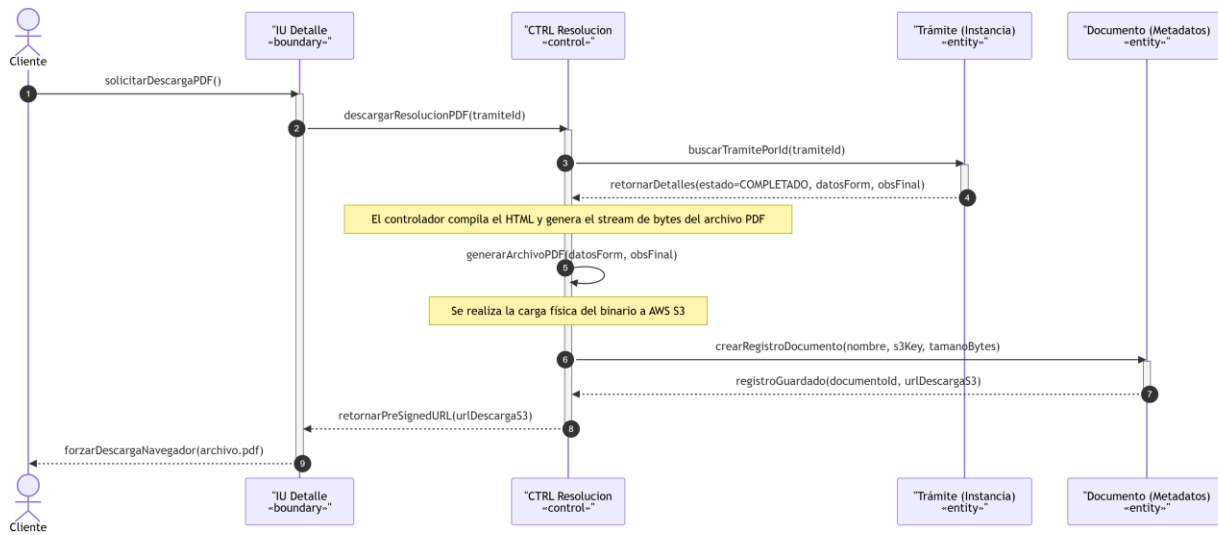
CU-10. Inicio de Trámite



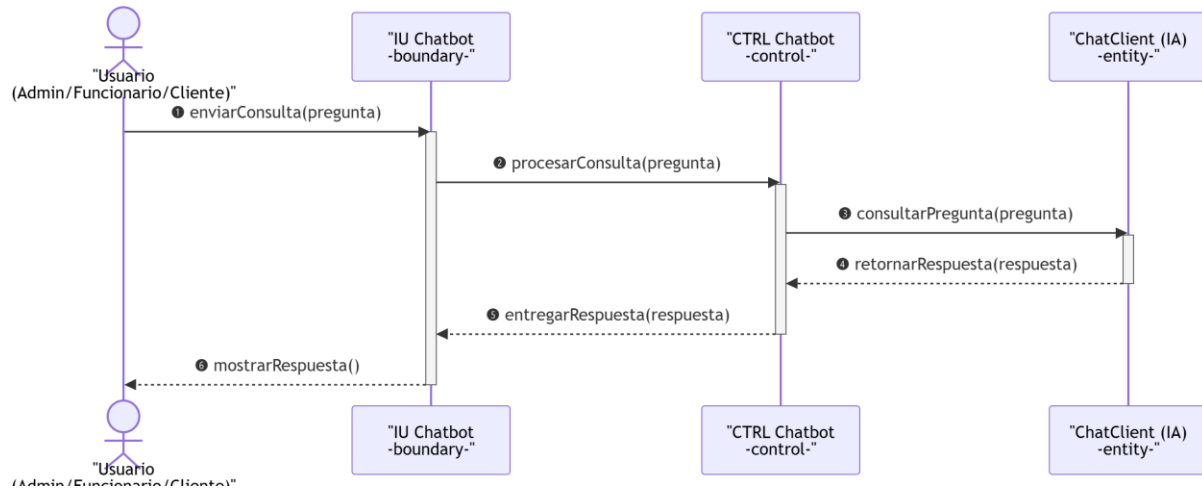
CU-11. Seguimiento Interactivo de Trámite



CU-12. Generación y Descarga de Resolución (PDF)

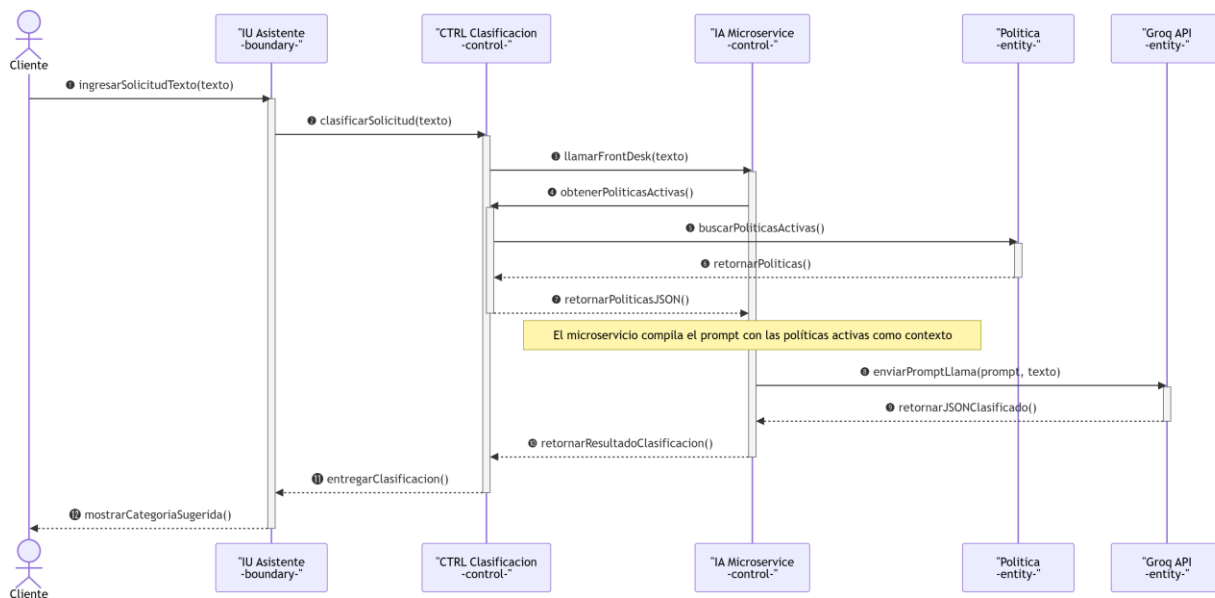


CU-13. Asistente Virtual (WorkBot)

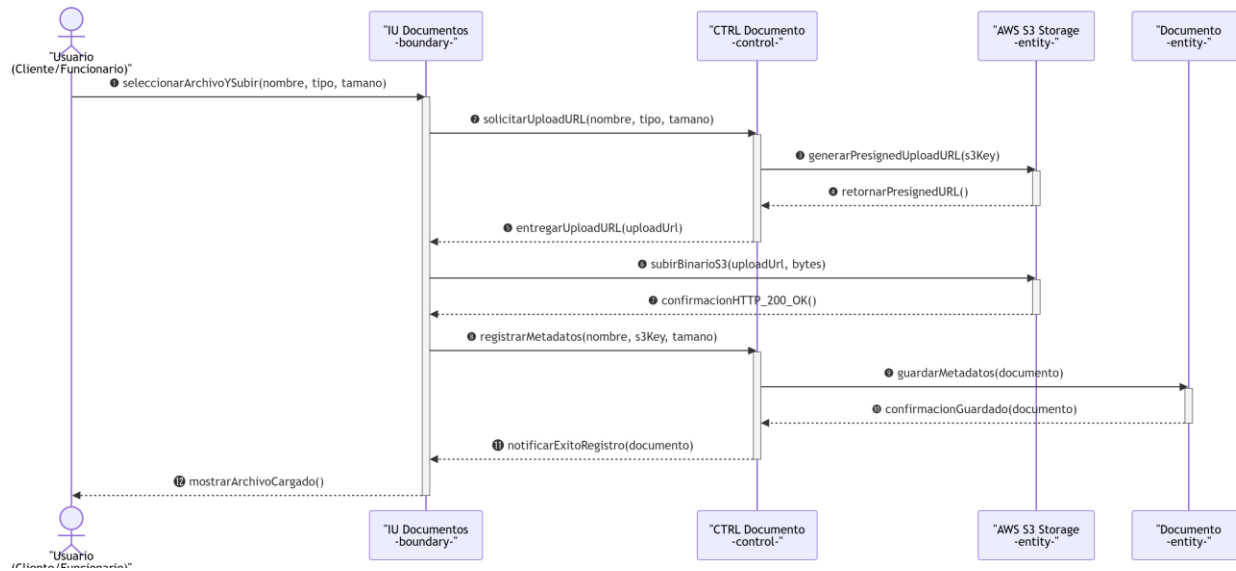


Ciclo 2

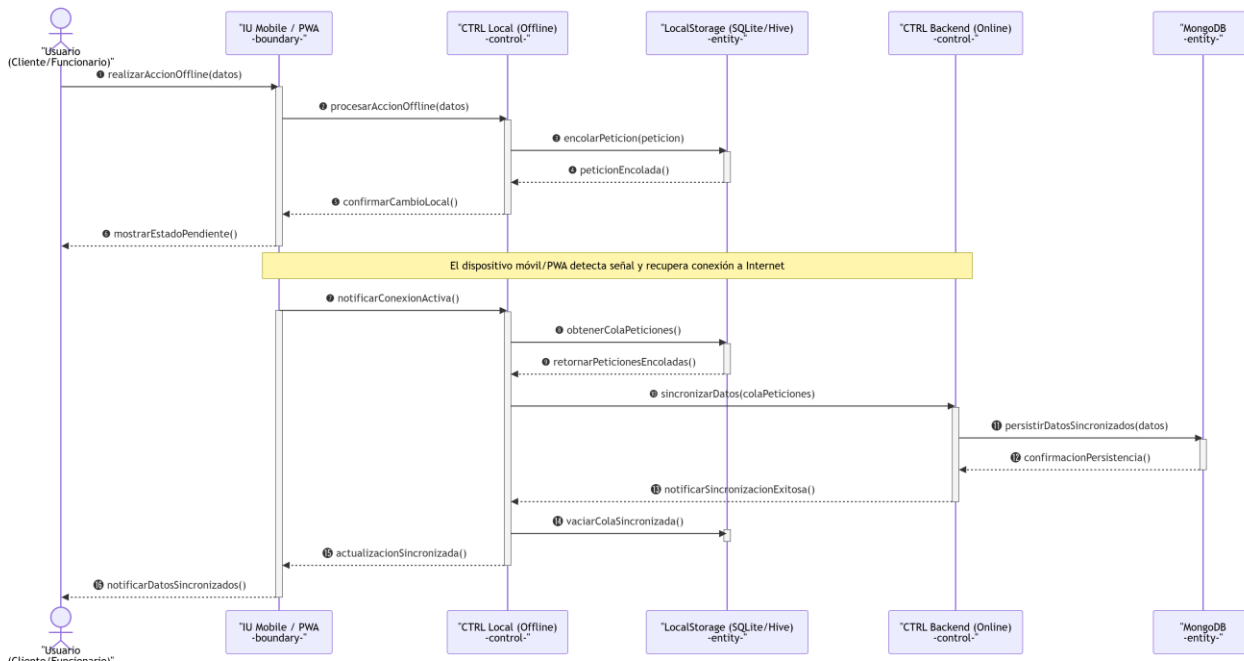
CU-14. Asistente Front-Desk Inteligente (Clasificación de Solicitud)



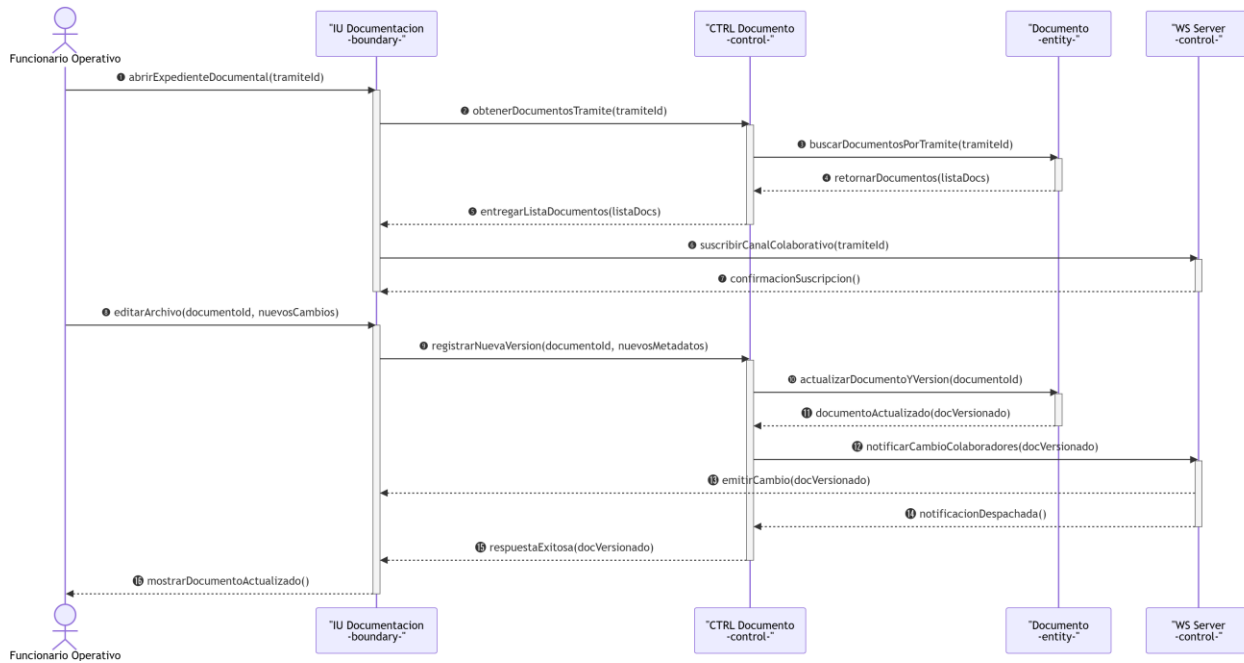
CU-15. Repositorio Documental Digital



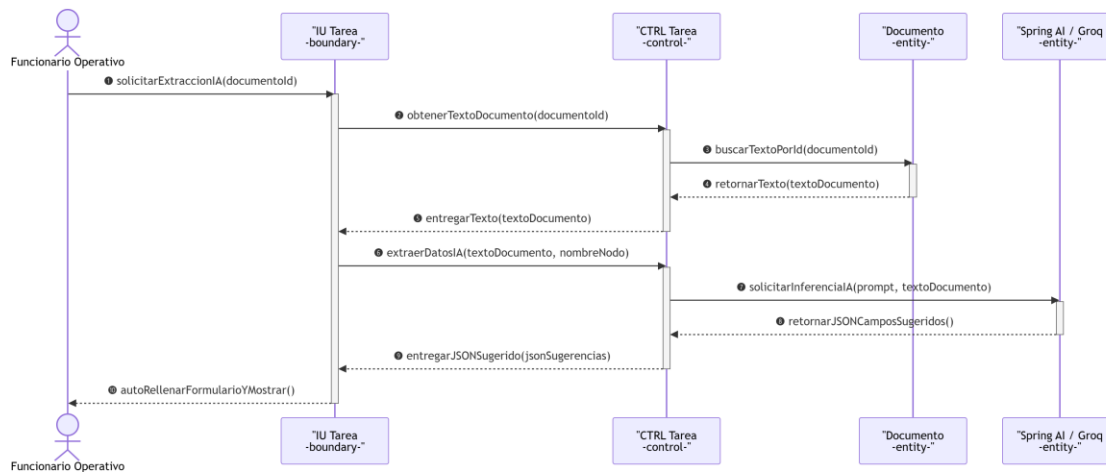
CU-16. Acceso Móvil / PWA Offline



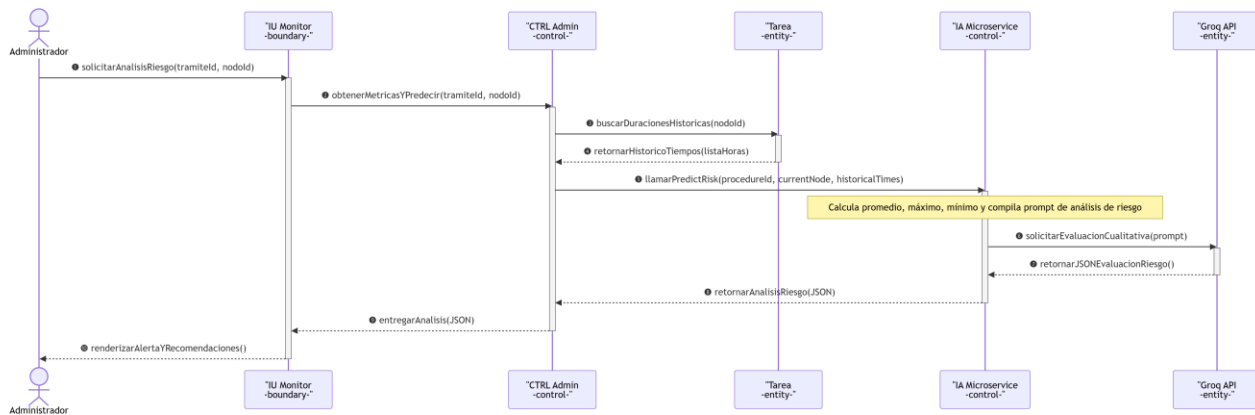
CU-17. Gestión Documental Colaborativa de Trámites



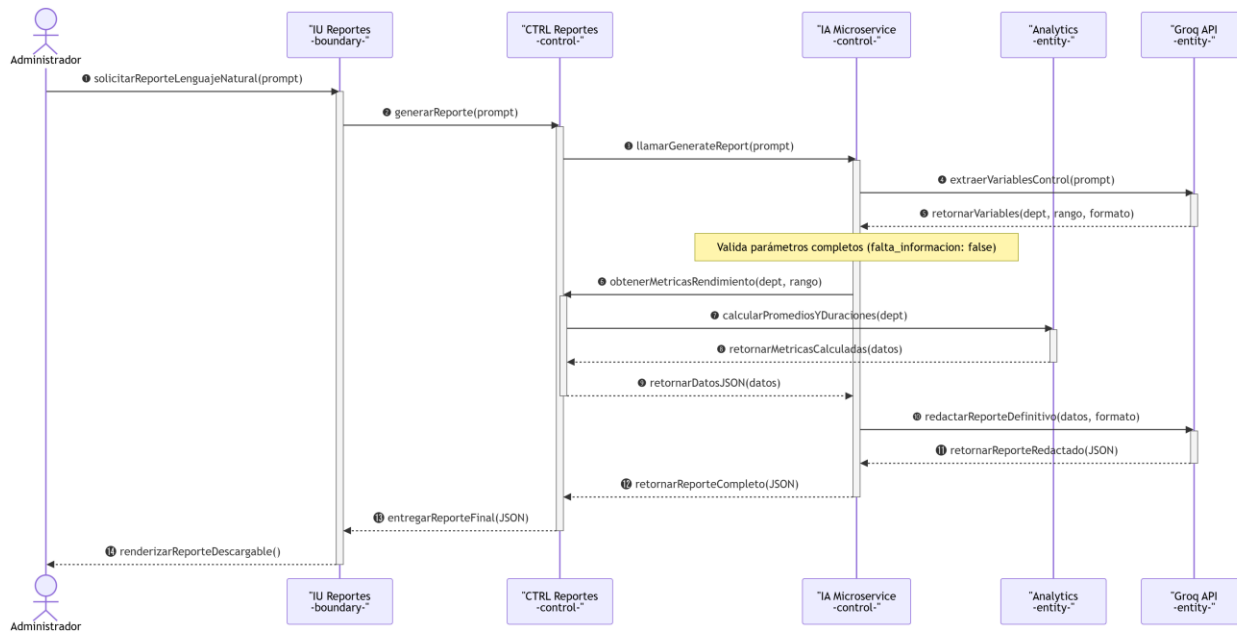
CU-18. Extracción Automática de Datos de Documento (IA)



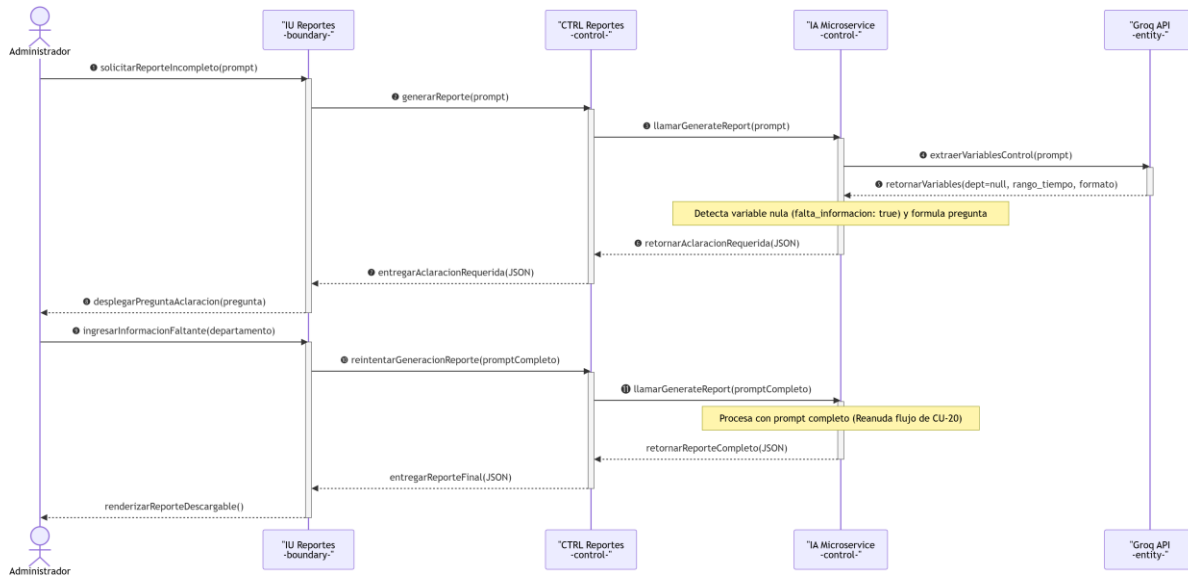
CU-19. Motor de Enrutamiento Inteligente y Análisis de Riesgo



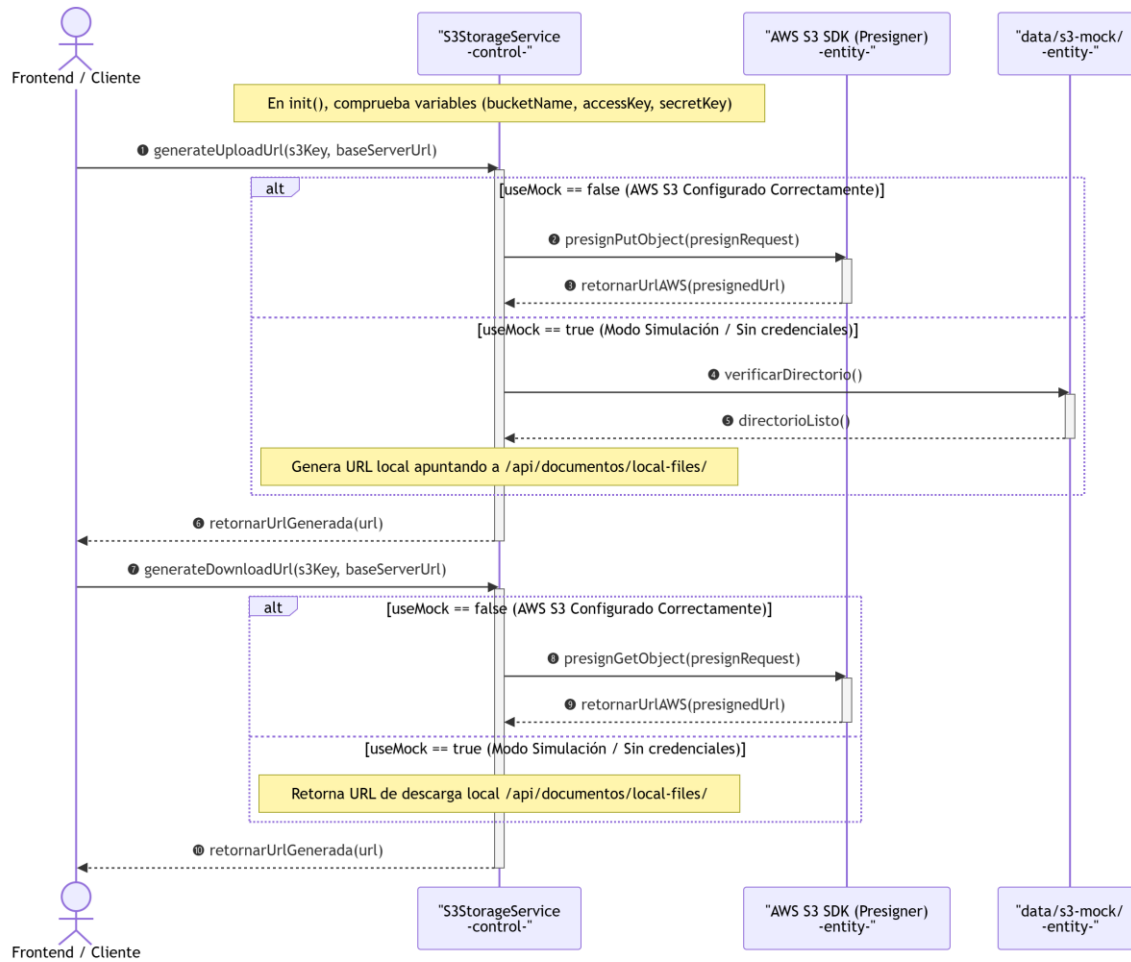
CU-20. Generación Dinámica de Reportes por Lenguaje Natural:



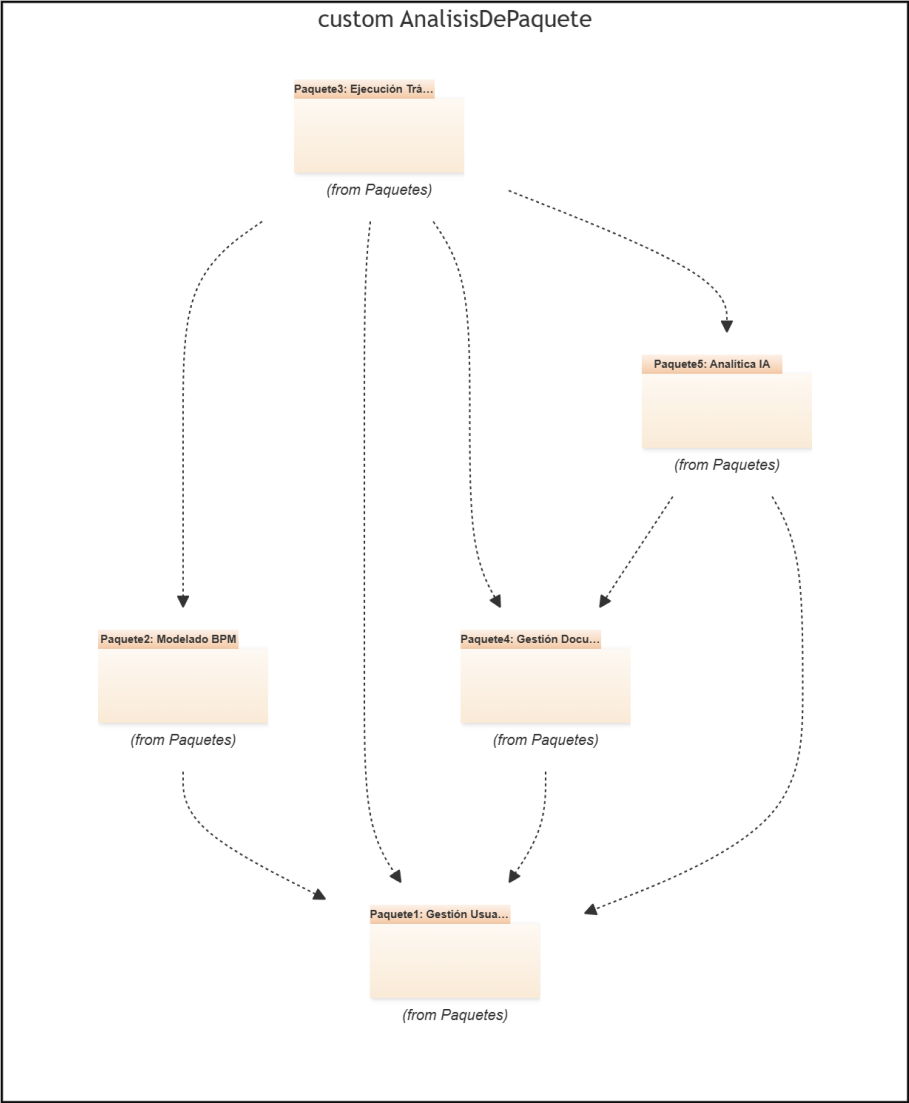
CU-21. Clarificación Conversacional de Reportes



CU-22. Gestión de Almacenamiento en Nube con AWS S3



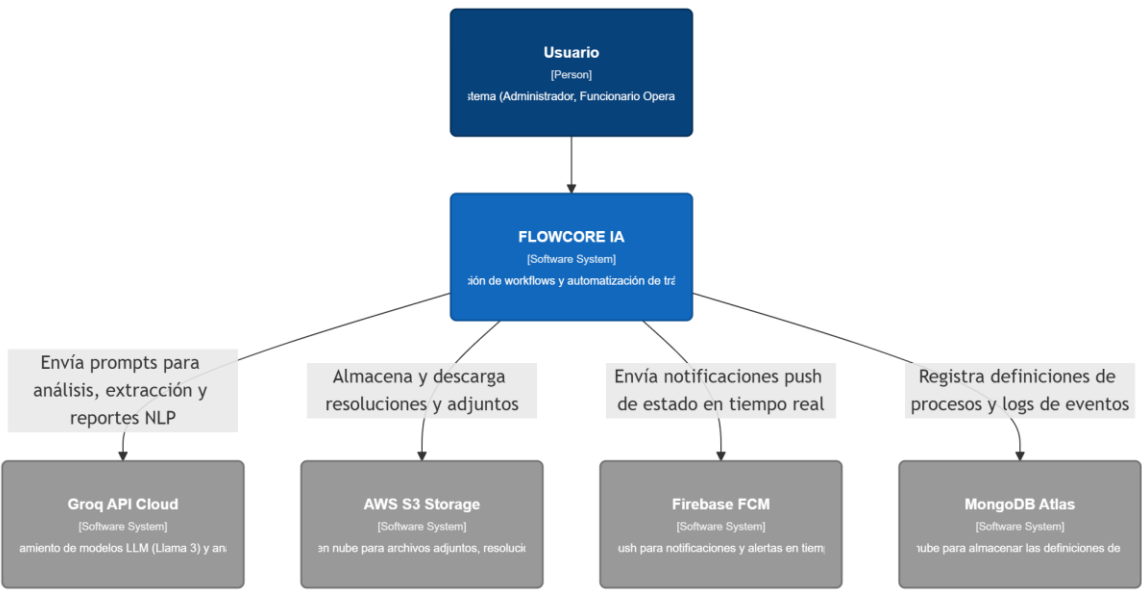
2.2.1.5 Análisis de Paquetes



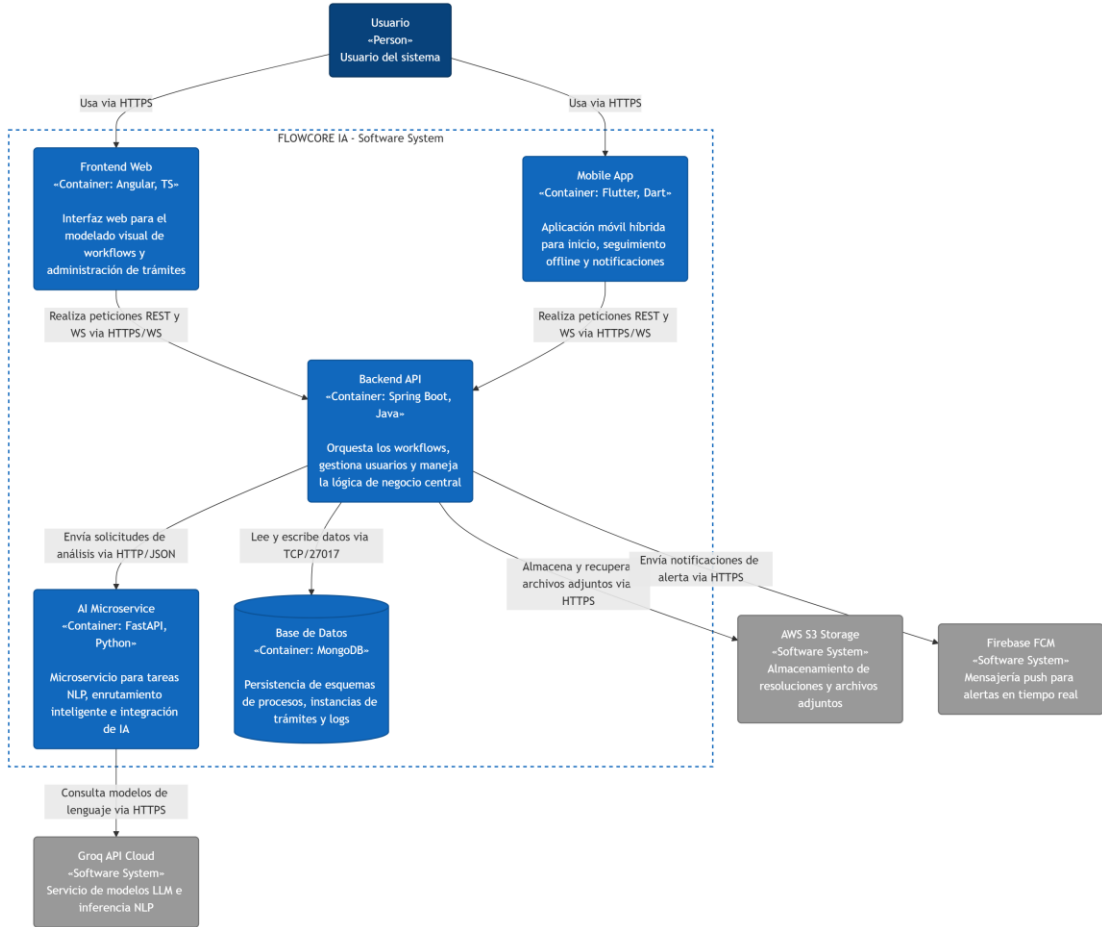
2.3 Flujo de Trabajo: Diseño

2.3.1 Diseño de arquitectura (Modelo C4)

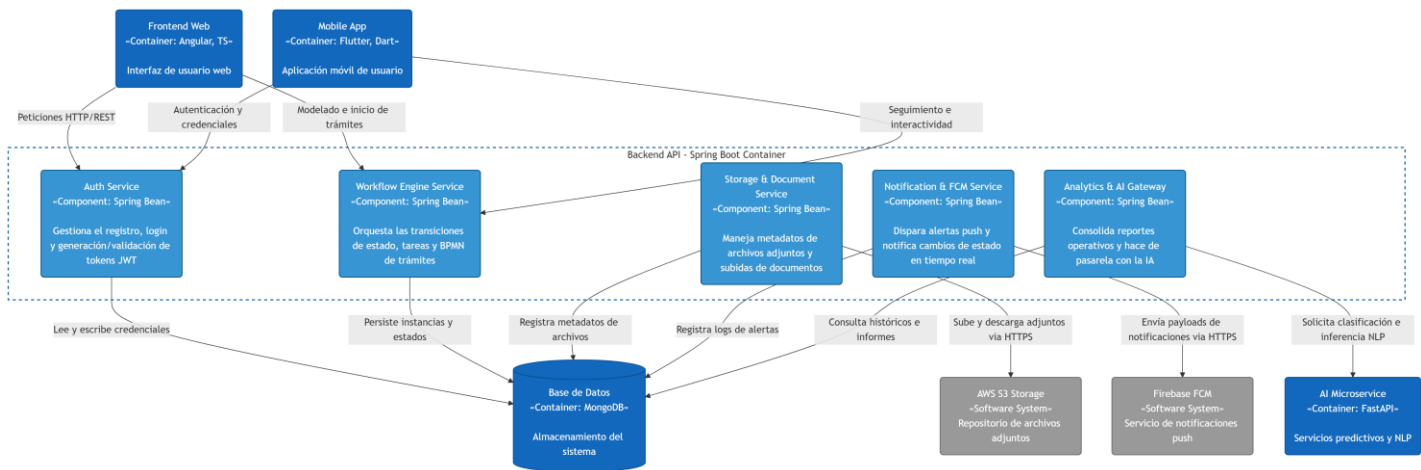
3.3.1.1 Diagrama de Contexto



3.3.1.2 Diagrama de Contenedores



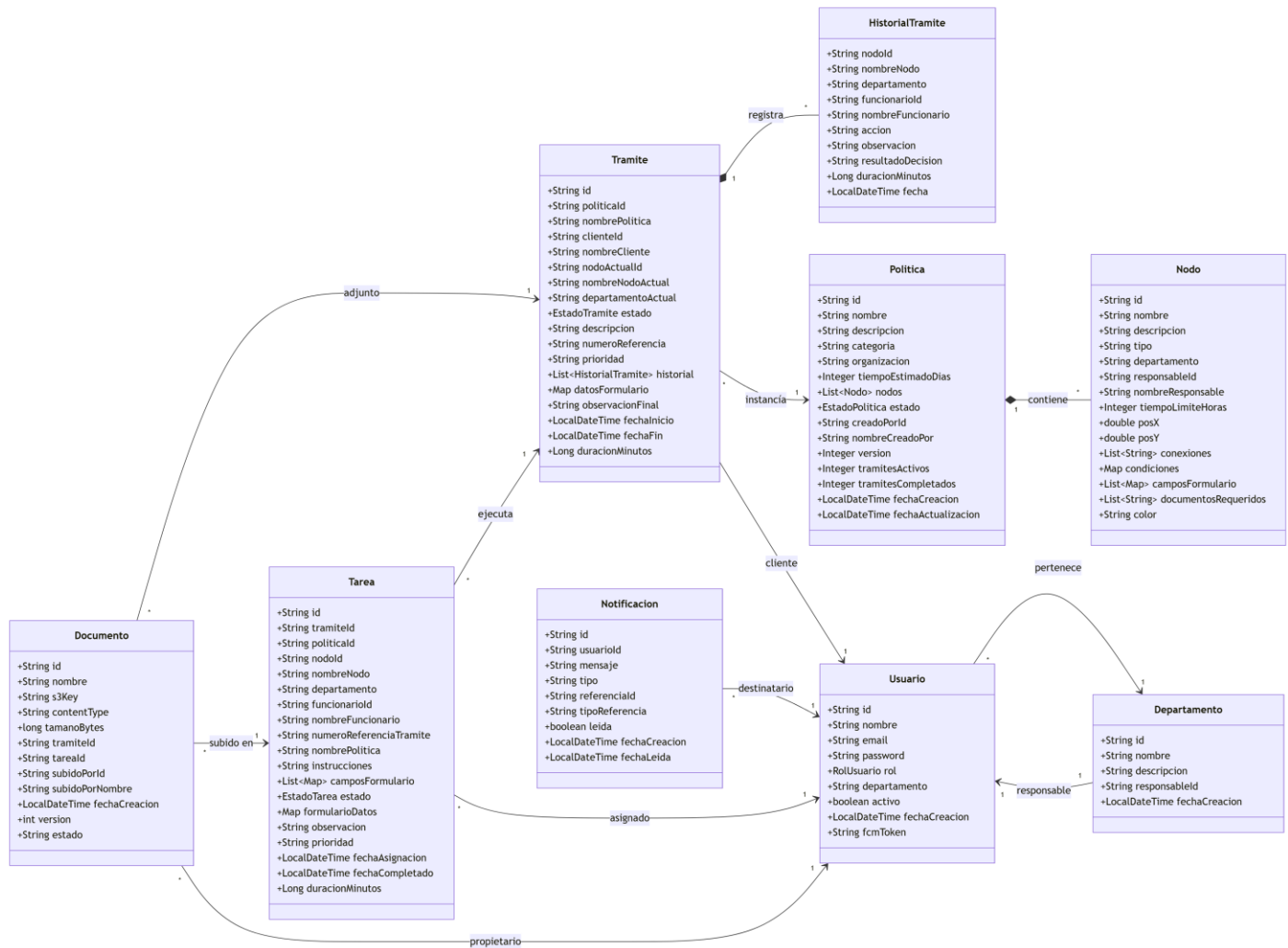
3.3.1.3 Diagrama de Componentes



2.3.2 Diseño físico (Modelo de Despliegue)

2.3.2.1 Diseño de Datos Logico

Diagrama de Clases



Mapeo

Usuario:

id	nombre	email	password	rol	departamento	activo	fechaCreacion	fcmToken
----	--------	-------	----------	-----	--------------	--------	---------------	----------

Departamento:

id	nombre	descripcion	responsableId	fechaCreacion
----	--------	-------------	---------------	---------------

Politica:

id	nombre	descripcion	categoria	organizacion	tiempoEstimadoDias	nodos	estado	creadoPor	nombreCreadorPor	version	tramitesActivos	tramitesCompletados	fechaCreacion	fechaActualizacion	fechaActivacion
----	--------	-------------	-----------	--------------	--------------------	-------	--------	-----------	------------------	---------	-----------------	---------------------	---------------	--------------------	-----------------

Nodo (Embebido en Politica):

id	nombre	descripcion	tipo	departamento	responsableId	nombreResponsable	tiempoLimiteHoras	posX	posY	conexiones	condiciones	camposFormula	documentosRequeridos	color
----	--------	-------------	------	--------------	---------------	-------------------	-------------------	------	------	------------	-------------	---------------	----------------------	-------

Tramite:

id	politicaId	nombrePolitica	clienteId	nombreCliente	nodoActualId	nombreNodoActual	departamentoActual	estado	descripcion	numeroReferencia	prioridad	historial	datosFormula	observacionFinal	fechaInicio	fechaFin	fechaUltimaActualizacion	duracionMinutos
----	------------	----------------	-----------	---------------	--------------	------------------	--------------------	--------	-------------	------------------	-----------	-----------	--------------	------------------	-------------	----------	--------------------------	-----------------

HistorialTramite

nodoId	nombreNodo	departamento	funcionarioId	nombreFuncionario	accion	observacion	resultadoDecision	duracionMinutos	fecha
--------	------------	--------------	---------------	-------------------	--------	-------------	-------------------	-----------------	-------

Tarea:

id	tramitad	politicaId	nodoId	nombreNodo	departamento	funcionarioId	nombreFuncionario	numeroReferenciaTramite	nombrePolitica	instrucciones	camposFormula	estado	formularioDatos	observacion	prioridad	fechaAsignacion	fechaCompletado	duracionMinutos
----	----------	------------	--------	------------	--------------	---------------	-------------------	-------------------------	----------------	---------------	---------------	--------	-----------------	-------------	-----------	-----------------	-----------------	-----------------

Documento:

id	nombre	s3Key	contentType	tamanoBytes	tramiteId	tareaId	subidoPor	subidoPorNombre	fechaCreacion	version	estado
----	--------	-------	-------------	-------------	-----------	---------	-----------	-----------------	---------------	---------	--------

Notificacion:

id	usuarioId	mensaje	tipo	referencialId	tipoReferencia	leida	fechaCreacion	fechaLeida
----	-----------	---------	------	---------------	----------------	-------	---------------	------------

2.3.3.2 Diseño de Datos Fisico

Tabla de Volumen

Usuario

Atributo	Tipo de dato	Descripción	Tamaño	Nulo	Tipo de Llave
id	String	Identificador único del usuario (MongoDB ObjectId)	24 bytes	no	primaria
nombre	String	Nombre completo del usuario	100 char	no	
email	String	Correo electrónico institucional único	150 char	no	
password	String	Contraseña del usuario cifrada con BCrypt	60 char	no	
rol	String (Enum)	Rol asignado (ADMIN, FUNCIONARIO, CLIENTE)	20 char	no	
departamento	String	Nombre del departamento del funcionario	100 char	no	
activo	Boolean	Estado de actividad de la cuenta de usuario	1 byte	no	
fechaCreacion	Datetime	Fecha y hora de registro de la cuenta	8 bytes	no	
fcmToken	String	Token de Firebase para notificaciones push	Variable	si	

Departamento

Atributo	Tipo de dato	Descripción	Tamaño	Nulo	Tipo de Llave
id	String	Identificador único del departamento	24 bytes	no	primaria
nombre	String	Nombre único del departamento o área	100 char	no	
descripcion	String	Descripción de funciones del departamento	255 char	si	

responsableId	String	ID del usuario jefe o administrador del área	24 bytes	no	
fechaCreacion	Datetime	Fecha y hora de creación del área	8 bytes	no	

Politica

Atributo	Tipo de dato	Descripción	Tamaño	Nulo	Tipo de Llave
id	String	Identificador único de la política	24 bytes	no	primaria
nombre	String	Nombre descriptivo de la política/proceso	100 char	no	
descripcion	String	Propósito y reglas del flujo de negocio	500 char	si	
categoria	String	Rubro o categoría del proceso de negocio	50 char	no	
organizacion	String	Organización o entidad propietaria	100 char	no	
tiempoEstimadoDias	Integer	Límite máximo estimado para el trámite	4 bytes	no	
nodos	List (JSON)	Estructura interna de los nodos (BPMN)	Variable	no	
estado	String (Enum)	Estado actual de la regla (BORRADOR, ACTIVA)	20 char	no	
creadoPorId	String	ID del administrador creador	24 bytes	no	

nombreCreadoPor	String	Nombre del administrador creador	100 char	no	
version	Integer	Versión numérica de control de cambios	4 bytes	no	
tramitesActivos	Integer	Cantidad de ejecuciones activas simultáneas	4 bytes	no	
tramitesCompletados	Integer	Histórico de ejecuciones exitosas cerradas	4 bytes	no	
fechaCreacion	Datetime	Fecha de alta en el sistema del workflow	8 bytes	no	
fechaActualizacion	Datetime	Fecha de última modificación de la regla	8 bytes	no	
fechaActivacion	Datetime	Fecha de puesta en producción	8 bytes	si	

Tramite

Atributo	Tipo de dato	Descripción	Tamaño	Nulo	Tipo de Llave
id	String	Identificador único del trámite	24 bytes	no	primaria
politicald	String	ID del workflow/política que rige el trámite	24 bytes	no	
nombrePolitica	String	Nombre de la política que rige el trámite	100 char	no	

clienteId	String	ID del usuario cliente que inició el trámite	24 bytes	no	
nombreCliente	String	Nombre completo del cliente iniciador	100 char	no	
nodoActualId	String	ID del nodo de la actividad en ejecución	50 char	si	
nombreNodoActual	String	Nombre de la actividad en ejecución	100 char	si	
departamentoActual	String	Área asignada responsable del nodo actual	100 char	si	
estado	String (Enum)	Estado (NUEVO, EN_PROCESO, COMPLETADO)	20 char	no	
descripcion	String	Comentarios o detalles extras del cliente	500 char	no	
numeroReferencia	String	Código legible único (ej: TRM-2026-0001)	30 char	no	
prioridad	String	Prioridad heredada (BAJA, MEDIA, ALTA)	10 char	no	
historial	List (JSON)	Log secuencial de pasos y actividades completadas	Variable	no	

datosFormulario	Map (JSON)	Datos y campos rellenos en los pasos	Variable	no	
observacionFinal	String	Comentarios finales del cierre	500 char	si	
fechaInicio	Datetime	Fecha de radicación del trámite	8 bytes	no	
fechaFin	Datetime	Fecha de cierre del trámite	8 bytes	si	
fechaUltimaActualizacion	Datetime	Fecha del último movimiento de estado	8 bytes	no	
duracionMinutos	Long	Tiempo total de ejecución (minutos)	8 bytes	si	

Tarea

Atributo	Tipo de dato	Descripción	Tamaño	Nulo	Tipo de Llave
id	String	Identificador único de la tarea asignada	24 bytes	no	primaria
tramiteld	String	ID del trámite contenedor de la tarea	24 bytes	no	
politicald	String	ID del workflow del proceso	24 bytes	no	
nodold	String	ID del nodo de actividad origen	50 char	no	

nombreNodo	String	Nombre del paso (ej: "Revisión Documental")	100 char	no	
departamento	String	Área responsable de procesar la tarea	100 char	no	
funcionarioId	String	ID del funcionario asignado al paso	24 bytes	si	
nombreFuncionario	String	Nombre del funcionario procesador	100 char	si	
numeroReferenciaTramite	String	Código único del trámite para vinculación	30 char	no	
nombrePolitica	String	Nombre del proceso para contextualización	100 char	no	
instrucciones	String	Indicaciones y directrices para el funcionario	500 char	si	
camposFormulario	List (JSON)	Definición de inputs a completar	Variable	no	
estado	String (Enum)	Estado (PENDIENTE, COMPLETADO, RECHAZADO)	20 char	no	
formularioDatos	Map (JSON)	Valores recolectados en el formulario	Variable	no	

observacion	String	Glosa o justificación escrita por el funcionario	500 char	si	
prioridad	String	Nivel de urgencia de la tarea	10 char	no	
fechaAsignacion	Datetime	Fecha y hora en que se creó la tarea	8 bytes	no	
fechaCompletado	Datetime	Fecha y hora en que se firmó y resolvió	8 bytes	si	
duracionMinutos	Long	Minutos requeridos por el funcionario	8 bytes	si	

Documento

Atributo	Tipo de dato	Descripción	Tamaño	Nulo	Tipo de Llave
id	String	Identificador único del archivo	24 bytes	no	primaria
nombre	String	Nombre original del archivo subido	150 char	no	
s3Key	String	Clave única del objeto en AWS S3	255 char	no	
contentType	String	Tipo MIME de datos (ej: application/pdf)	50 char	no	
tamanoBytes	Long	Tamaño físico en bytes del archivo	8 bytes	no	
tramiteId	String	ID del trámite en el cual se adjuntó	24 bytes	no	

tareald	String	ID de la tarea específica en que se subió	24 bytes	si	
subidoPorId	String	ID del usuario que cargó el documento	24 bytes	no	
subidoPorNombre	String	Nombre del usuario que cargó el documento	100 char	no	
fechaCreacion	Datetime	Fecha de alta y subida del adjunto a S3	8 bytes	no	
version	Integer	Control de versión del archivo cargado	4 bytes	no	
estado	String	Estado físico de validez (ACTIVO, ELIMINADO)	20 char	no	

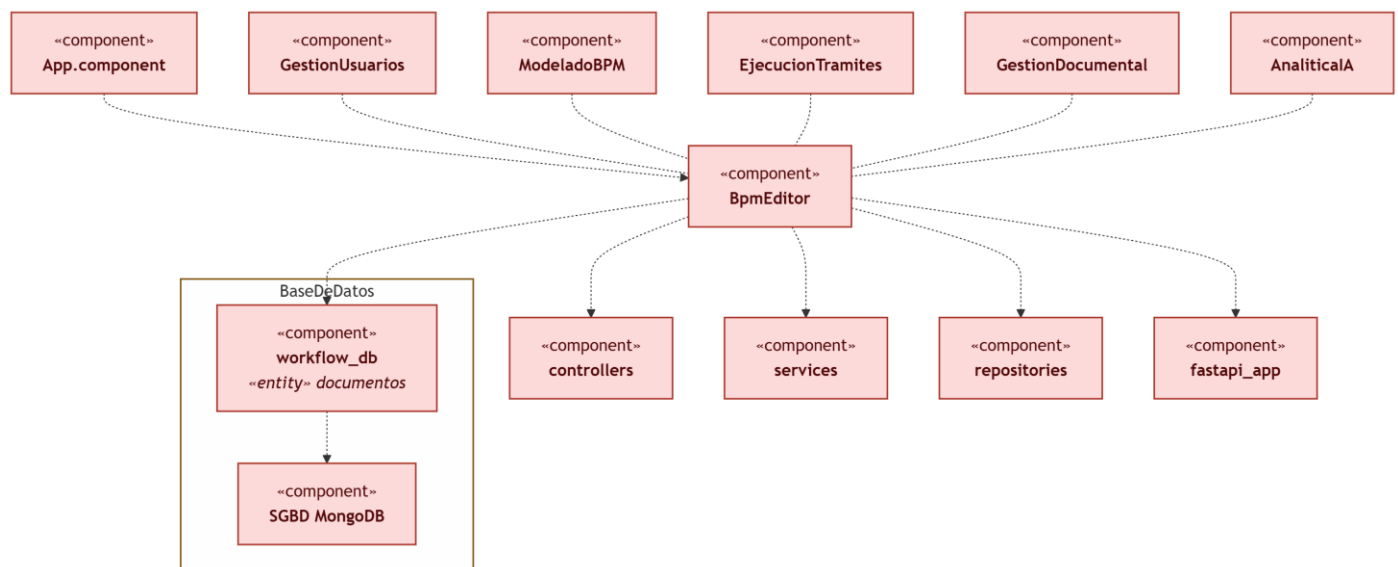
Notificacion

Atributo	Tipo de dato	Descripción	Tamaño	Nulo	Tipo de Llave
id	String	Identificador único de la notificación	24 bytes	no	primaria
usuarioid	String	ID del destinatario de la alerta push/in-app	24 bytes	no	
mensaje	String	Contenido corto o glosa de la notificación	255 char	no	
tipo	String	Tipo (NUEVA_TAREA, CUELLO_BOTELLA, etc.)	30 char	no	
referenciald	String	ID del recurso origen asociado (tarea o trámite)	24 bytes	no	

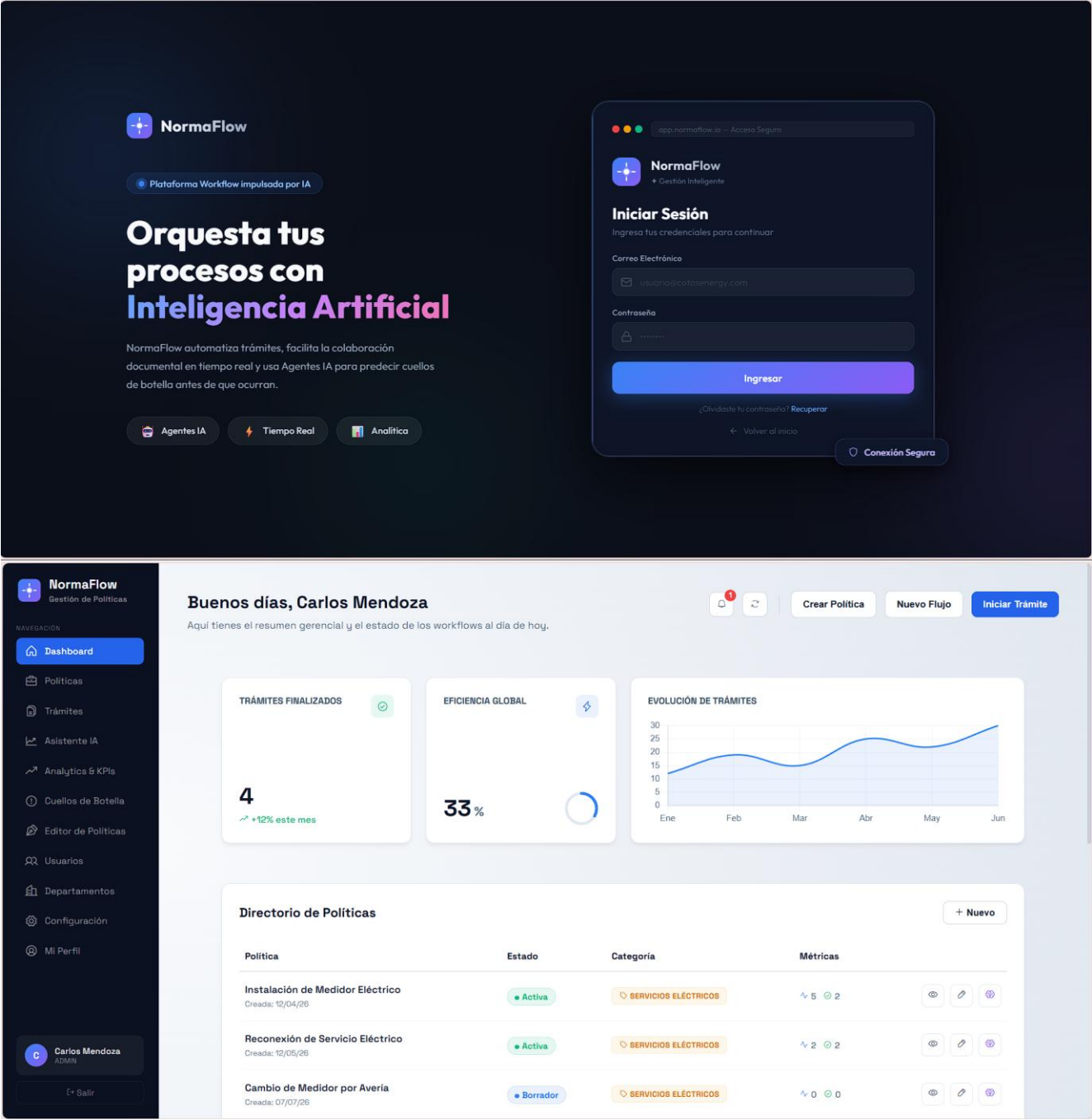
tipoReferencia	String	Tipo de recurso origen (TAREA, TRAMITE)	20 char	no	
leida	Boolean	Indicador si la alerta ya fue vista	1 byte	no	
fechaCreacion	Datetime	Fecha y hora de emisión del push	8 bytes	no	
fechaLeida	Datetime	Fecha y hora de apertura de la alerta	8 bytes	si	

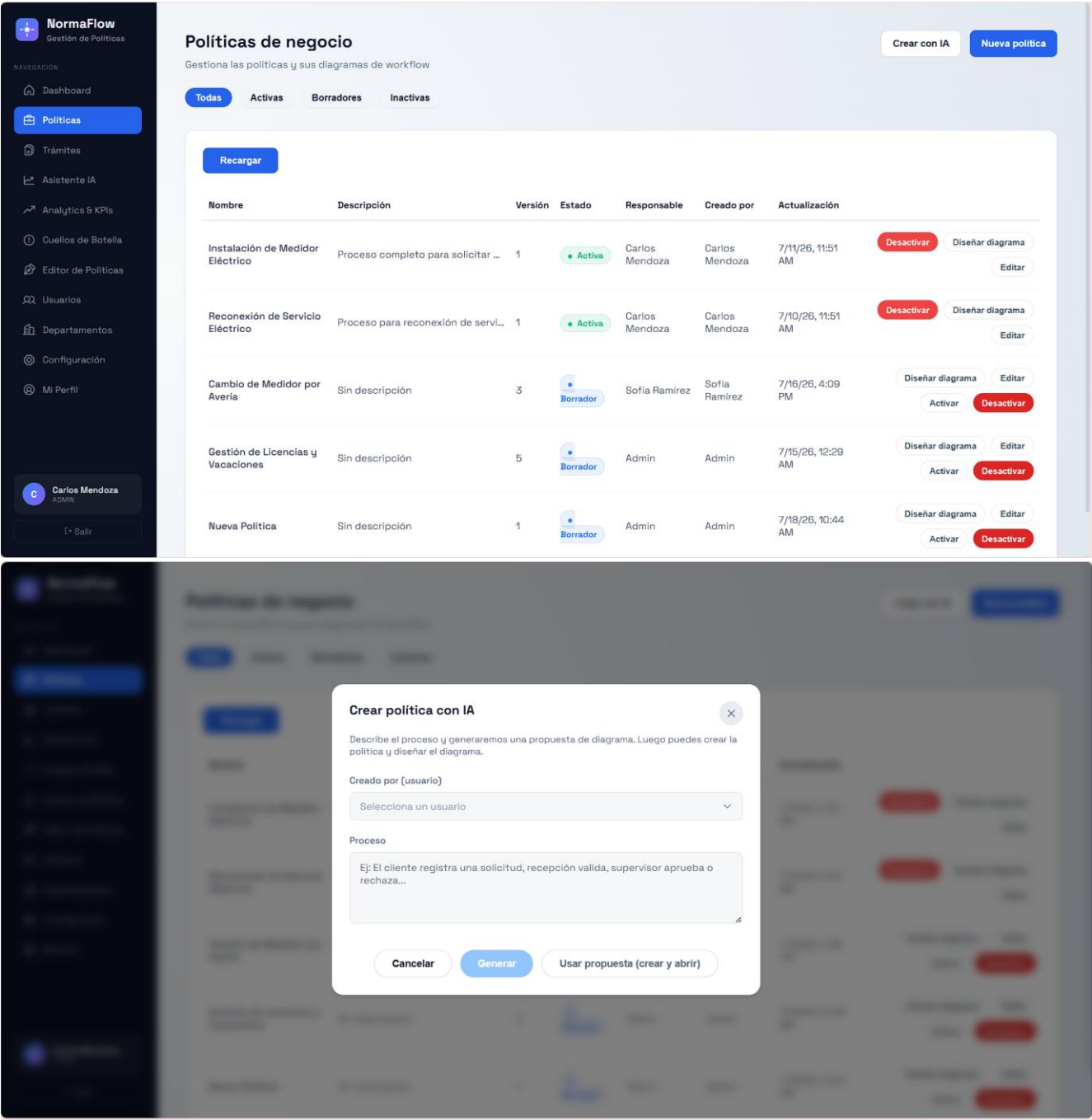
2.4 Flujo de Trabajo: Implementación

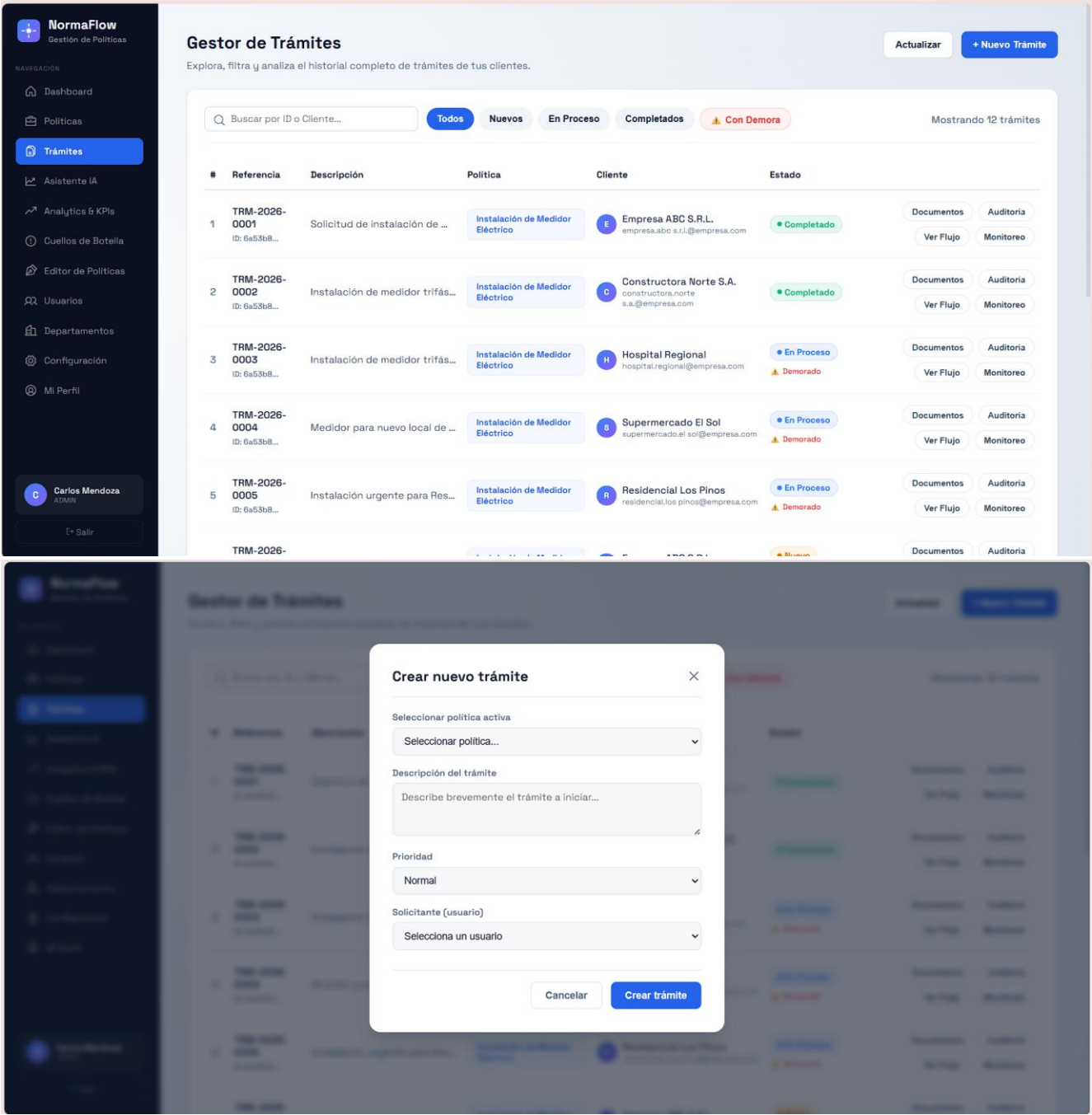
2.4.1 Implementación de la arquitectura del Sistema



2.5 Flujo de Trabajo: Pruebas







NormaFlow

Gestión de Políticas

NAVIGACIÓN

Dashboard

Políticas

Trámites

Asistente IA

Analytics & KPIs

Cuellos de Botella

Editor de Políticas

Usuarios

Departamentos

Configuración

Mi Perfil

Carlos Mendoza

ADMIN

Salir

Editor de diagramas

Selecciona una política en borrador para abrir el editor UML.

+ Crear Nueva Política

Unirme por link/código (colaborativo)

Pega aquí el link que te compartieron (.../policies/<id>/diagram?collab=1&who=...) o el ID

Unirme

Tip: si solo te pasan el ID de la política, también sirve pegarlo (ej: 6620...).

Políticas en estado BORRADOR

Nombre	Versión	Acción
Cambio de Medidor por Avería	3	Abrir editor
Gestión de Licencias y Vacaciones	5	Abrir editor
Nueva Política	1	Abrir editor

←

Gestión de Licencias y Vac

Categoría

3 calles5 nodos5 enlaces1 editor(es)

+ Calle

UML

Guardar

Activar

Propiedades del Nodo

Selecciona un nodo en el lienzo para editar sus propiedades y formulario.

ELEMENTOS

Inicio

Fin

Tarea

Decisión

Paralelo

GoJS 4.0 evaluation
(c) 1998-2020 Northwoods Software
Not for distribution or production use
gojs.net

General

Recursos Humanos

Dept. Correspondiente

¿Aprobación de Vacaciones?

FIN

Solicitud en RRHH

Inicio

Revisión del Jefe de Departamento

NormaFlow

Gestión de Políticas

NAVIGACIÓN

Dashboard

Políticas

Trámites

Asistente IA

Analytics & KPIs

Cuellos de Botella

Editor de Políticas

Usuarios

Departamentos

Configuración

Mi Perfil

C

Carlos Mendoza

ADMIN

Salir

Asistente IA

Consulta datos organizacionales y genera reportes dinámicos

¡Hola! Soy el Asistente Analítico de NormaFlow.
Puedes pedirme reportes en lenguaje natural o usar tu voz.

Por ejemplo:

- "Muéstrame la eficiencia de los funcionarios"

- "Trámites con riesgo de demora"

Ej. Trámites con riesgo de demora...

NormaFlow Analista utiliza modelos avanzados y puede cometer errores. Verifica los datos.

CONVERSACIONES

+ Nuevo chat

Nueva conversación

3:53 PM - 10/07

Volver

Reconexión de Servicio Elé

Servicios Electr

3 calles | 0 nodos | 0 enlaces

Calle

Guardar

Desactivar

UML

Inicio

Fin

Tarea

Decisión

Paralelo

Haz clic en un nodo para configurar sus detalles y variables.

Anastra elementos desde la paleta hasta el lienzo.

Anastra desde el borde de un nodo para crear conexiones de flujo.

Usa "x Calle" para definir departamentos responsables.

Reconexión de Servicio Eléctrico

Inicio

Verificar Deuda

¿Deuda pagada?

FIN

Reconexión Técnica

Notificar Rechazo

Verificar Deuda

¿Deuda pagada?

FIN

Reconexión Técnica

Notificar Rechazo

Verificar Deuda

¿Deuda pagada?

FIN

Reconexión Técnica

Notificar Rechazo

ASISTENTE COPILOT IA

Nota: Describe los cambios que quieres hacer al diagrama. Puedo agregar nodos, calles, conexiones y mas.

Política "Reconexión de Servicio Eléctrico" cargada. 0 nodos, estado: ACTIVA

Calle Legal

Tarea

Decisión

Secuenciar

Pídele a la IA diseñar

FLOWCORE IA

NAVEGACIÓN

Panel Principal

Editor de Políticas

Analytics

Reportes IA

Usuarios

Mi Perfil

Carlos Mendoza

ADMIN

Salir

Generador de Reportes con IA

Generación conversacional de reportes ejecutivos usando procesamiento NLP

Solicitar Reporte Ejecutivo

La IA analizará tu solicitud y validará que especifiques tres variables obligatorias: Departamento, Proceso, Período de Evaluación y Formato de Salida.

Ej: Genera un reporte de Finanzas de los últimos 3 meses en formato Markdown...

Generar

NormaFlow

Gestión de Políticas

NAVEGACIÓN

Dashboard

Políticas

Trámites

Asistente IA

Analytics & KPIs

Cuellos de Botella

Editor de Políticas

Usuarios

Departamentos

Configuración

Mi Perfil

Carlos Mendoza

ADMIN

Salir

Analytics & IA

Panel de control, métricas, cuellos de botella y rendimiento de la organización.

General y Anomalías

Análisis por Política

Por Departamento/Funcionario

Anomalías Detectadas (IA LSTM)

Actualizar

No se han detectado anomalías graves recientes. ¡Todo marcha bien!

KPIs Globales de la Empresa

TOTAL TRÁMITES

12

TASA COMPLETADO

33.3%

ACTIVOS AHORA

5

DURACIÓN PROM.

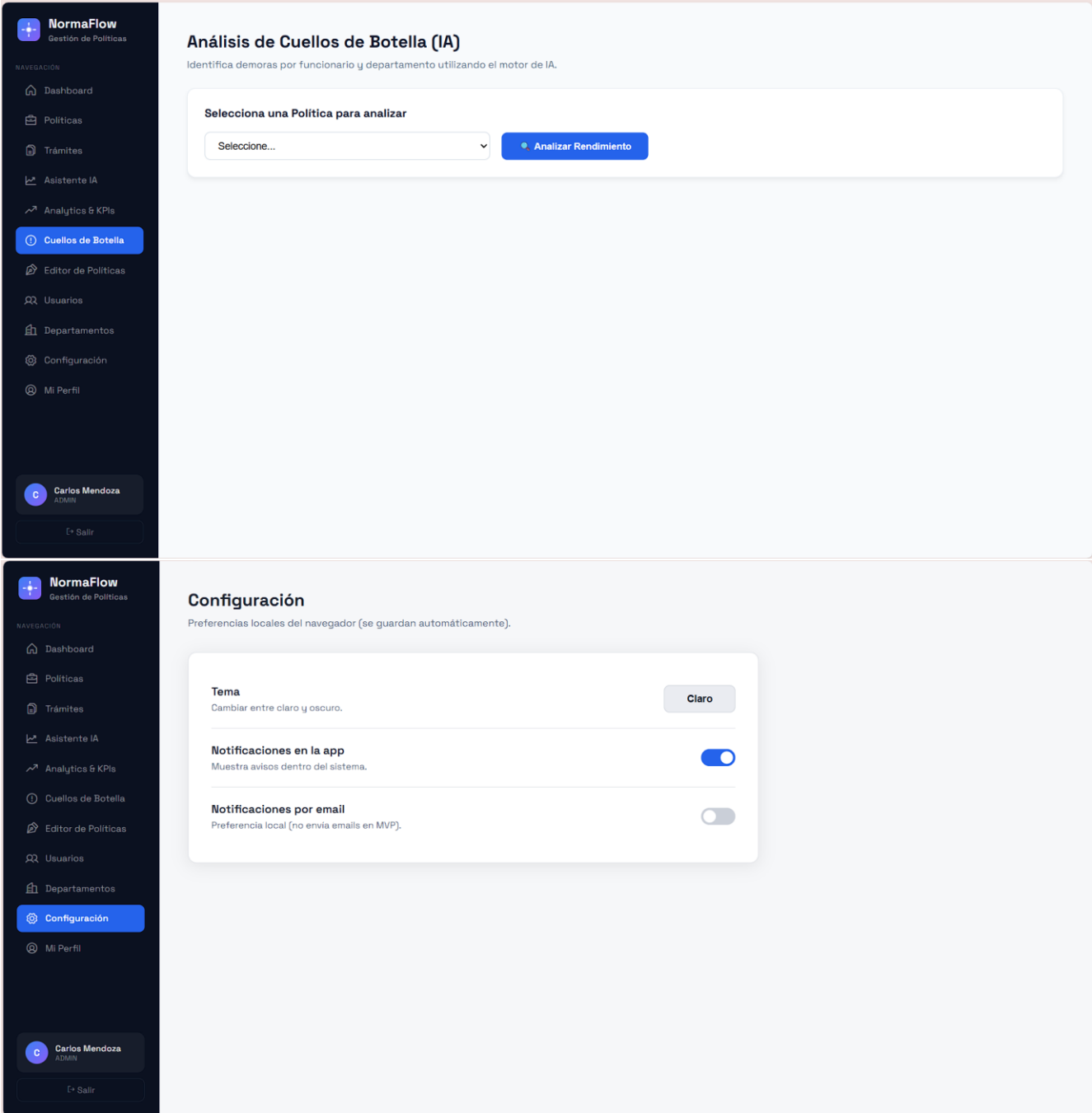
5483 min

TAREAS PENDIENTES

5

RECHAZADOS

0



NormaFlow

Gestión de Políticas

NAVEGACIÓN

Dashboard

Políticas

Trámites

Asistente IA

Analytics & KPIs

Cuellos de Botella

Editor de Políticas

Usuarios

Departamentos

Configuración

Mi Perfil

Carlos Mendoza

ADMIN

Salir

Mi Perfil

Gestiona tu información personal

C

Carlos Mendoza

ADMIN

Administración

admin@workflow.com

Editar Información

Nombre completo

Carlos Mendoza

Correo electrónico

admin@workflow.com

Departamento

Administración

Cancelar

Guardar cambios

Parte III

Bibliografía

Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language User Guide* (2.^a ed.). Addison-Wesley.

Cáceres, M. (2019). *Herramientas CASE en el desarrollo de software: Análisis y aplicación práctica*. Editorial Académica Española.

Cooper, A., Reimann, R., Cronin, D., & Noessel, C. (2014). *About Face: The Essentials of Interaction Design* (4.^a ed.). Wiley.

ISO/IEC. (2011). *ISO/IEC 25010:2011 - Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE)*. International Organization for Standardization.

Larman, C. (2012). *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development* (3.^a ed.). Pearson.

OMG. (2017). *OMG Unified Modeling Language (UML), Version 2.5.1*. Object Management Group. <https://www.omg.org/spec/UML/2.5.1/>

Preece, J., Rogers, Y., & Sharp, H. (2015). *Interaction Design: Beyond Human-Computer Interaction* (4.^a ed.). Wiley.

Sommerville, I. (2016). *Software Engineering* (10.^a ed.). Pearson.

Tognazzini, B., & Nielsen, J. (2020). *Guidelines for User Experience Practitioners*. Nielsen Norman Group. <https://www.nngroup.com/articles/>

Tucker, L. (2021). *Designing with Figma: Collaborative UI Design and Prototyping*. O'Reilly Media.

Anexos

Estándares de codificación utilizados

Para mantener un código uniforme, legible y fácil de mantener, el proyecto NormaFlow-IA aplica convenciones de nomenclatura de acuerdo con la tecnología y el tipo de elemento. Los siguientes estándares fueron identificados en el backend Spring Boot, el frontend Angular, la aplicación móvil Flutter y las colecciones almacenadas en MongoDB.

Estándar	Dónde se aplica	Ejemplos reales del proyecto
camelCase	Variables, atributos, propiedades y métodos en Java, TypeScript y Dart.	clienteId, politicaRepository, mensajeInput, iniciarSesion(), procesarMensaje(), enviarMensaje()
PascalCase	Clases, interfaces, enumeraciones, componentes Angular y widgets de Flutter.	AgenteService, AgenteSession, MensajeDialogo, DemoAgentelaComponent, CollaborativeEditorScreen
kebab-case	Archivos, selectores y rutas de componentes en Angular.	demo-agente-ia.component.ts, agente.service.ts, app-demo-agente-ia, /demo-agente-ia
snake_case	Colecciones de MongoDB y nombres de archivos o módulos en Flutter.	agente_sesiones, app_config.dart, collaborative_editor_screen.dart, auth_service.dart
UPPER_CASE	Estados, roles y valores fijos utilizados por el flujo de negocio.	IDENTIFICACION, REQUISITOS, CONFIRMACION, COMPLETADA, ABANDONADA, USUARIO, AGENTE

Ejemplos de aplicación en el código

A continuación se muestran fragmentos representativos del repositorio que permiten comprobar la aplicación de los estándares descritos.

PascalCase y camelCase en TypeScript

```
export interface AgenteSession {  
  id: string;  
  clienteId: string;
```

```

nombrePolitica?: string;
}
export class DemoAgenteIaComponent implements OnInit, OnDestroy {
mensajeInput = '';
cargando = false;
iniciarSesion(): void {
this.agenteService.iniciarSesionDemo();
}
}
}

```

En este fragmento, AgenteSession y DemoAgentelaComponent utilizan PascalCase por tratarse de una interfaz y una clase. En cambio, clienteld, nombrePolitica, mensajeInput e iniciarSesion() emplean camelCase.

snake_case en MongoDB y Flutter

```

@Document(collection = "agente_sesiones")
public class AgenteSession {
private String clienteId;
private String politicaId;
}
app_config.dart
collaborative_editor_screen.dart
auth_service.dart

```

La colección agente_sesiones y los archivos app_config.dart, collaborative_editor_screen.dart y auth_service.dart utilizan snake_case, separando las palabras mediante guiones bajos. En la imagen de referencia este grupo aparece rotulado como kebab-case, pero técnicamente corresponde a snake_case.

Repositorio:

<https://github.com/maddieflores/NormaFlow-IA.git>



Web:

<http://98.81.152.138/>



Manual de Usuario:

<https://drive.google.com/file/d/1k42P06TME0jmI0VXp4HYOzP10zVFXOgi/view?usp=sharing>

